



FACULTY OF ELECTRONICS,
TELECOMMUNICATIONS
AND INFORMATICS

Student's name and surname: Michał Szyfelbein

ID: 193307

Cycle of studies: bachelor's degree studies

Mode of study: full-time studies

Field of study: Informatics

Profile: Algorithms and Systems Modeling

ENGINEERING DIPLOMA THESIS

Title of the thesis: Experimental analysis of binary search models in graphs.

Title of the thesis (in Polish): Analiza eksperymentalna modeli wyszukiwania binarnego w grafach.

Supervisor: prof. dr hab. inż. Dariusz Dereniowski

FACULTY OF ELECTRONICS, TELECOMMUNICATIONS AND
INFORMATICS
GDAŃSK UNIVERSITY OF TECHNOLOGY, POLAND

Experimental Analysis of Binary Search Models in Graphs

Michał Szyfelbein

Supervisor: prof. dr hab. inż. Dariusz Dereniowski

December 9, 2025

Abstract

In this work, we conduct an experimental analysis of the generalized binary search problem in tree graphs. The study is structured into two main sections:

The first part focuses on the theoretical foundations of the problem. It introduces key definitions, concepts, and pseudocodes of the analyzed procedures, along with a formal analysis of their parameters, including approximation ratios and time complexities.

The second part presents experimental verification of the theoretical claims established in the previous chapter. It includes a practical comparison of the quality of the solutions produced by the algorithms, both against each other and relative to the optimum. We also measure the computation time required by each algorithm to obtain these results. We evaluate the performance of the algorithms on randomly generated graphs of various classes: general random trees, random trees with bounded degree, and random trees with bounded diameter. Additionally, we employ different methods for generating cost functions, including uniform, binomial, and exponential distributions.

Keywords and phrases Trees, Graph Searching, Binary Search, Decision Trees, Ranking Colorings, Graph Theory, Approximation Algorithm, Combinatorial Optimization, Experimental Analysis of Algorithms

Acknowledgements The author wants to thank prof. dr hab. inż. Dariusz Dereniowski for the preliminary discussions, guidance throughout the development of this thesis and many insightful suggestions regarding this work.

Gratitude is also expressed towards the Department of Algorithms and System Modeling for providing computational resources required to conduct the experiments presented in this thesis. Special thanks are extended to dr inż. Krzysztof Manuszewski for his assistance with setting up the experimental environment and maintaining the computational infrastructure which was essential to conduct the extensive amount of experiments required for this work.

Streszczenie

W niniejszej pracy przeprowadzamy eksperymentalną analizę uogólnionego problemu wyszukiwania binarnego w grafach drzewiastych. Badanie podzielone jest na dwie główne sekcje:

Pierwsza część koncentruje się na teoretycznych podstawach problemu. Wprowadza kluczowe definicje, koncepcje i pseudokody analizowanych procedur, wraz z formalną analizą ich parametrów, w tym współczynników aproksymacji i złożoności czasowych.

Druga część przedstawia eksperymentalną weryfikację twierdzeń teoretycznych zaprezentowanych w poprzednim rozdziale. Obejmuje ona praktyczne porównanie jakości rozwiązań generowanych przez algorytmy, zarówno ze sobą nawzajem, jak i w stosunku do optimum. Mierzymy również czas obliczeń wymagany przez każdy algorytm do uzyskania tych wyników. Oceniamy wydajność algorytmów na losowo generowanych grafach różnych klas: ogólne drzewa losowe, drzewa losowe o ograniczonym stopniu, oraz drzewa losowe o ograniczonej średnicy. Dodatkowo stosujemy różne metody generowania funkcji kosztów, w tym rozkłady jednorodny, dwumianowy i wykładniczy.

Słowa kluczowe Drzewa, Przeszukiwanie Grafów, Wyszukiwanie Binarne, Drzewa Decyzyjne, Kolorowania Rankingowe, Teoria Grafów, Algorytm Aproksymacyjny, Optymalizacja Kombinatoryczna, Eksperymentalna Analiza Algorytmów

Podziękowania Autor chce podziękować prof. dr hab. inż. Dariuszowi Dereniowskiemu za wstępne dyskusje, pomoc w opracowywaniu oraz wiele cennych sugestii dotyczących niniejszej pracy.

Wyrazy wdzięczności kierowane są również w stronę Katedry Algorytmów i Modelowania Systemów za udostępnienie zasobów obliczeniowych niezbędnych do przeprowadzenia eksperymentów przedstawionych w tej pracy. Szczególne podziękowania należą się dr inż. Krzysztofowi Manuszewskiemu za pomoc w konfiguracji środowiska eksperymentalnego oraz utrzymanie infrastruktury obliczeniowej, która była niezbędna do przeprowadzenia obszernej liczby eksperymentów wymaganych do tej pracy.

Contents

Glossary	5
1 Introduction	7
1.1 The search problem	7
1.2 Problem statement	8
1.3 Motivations and applications	9
1.4 The three field notation for the search problem	11
1.5 Many names, one problem	12
1.6 Related work	12
1.7 The aim of the thesis	13
1.8 Organization of the work	14
2 Notions and Definitions	15
2.1 Graph theory	15
2.2 Optimization problems	15
2.3 The Graph Search Problem	16
2.3.1 Additional input parameters	16
2.3.2 Decision trees, optimization criteria and the Graph Search Problem	16
3 Theoretical Analysis	18
3.1 Trees, Worst Case, Uniform Costs	19
3.2 Trees, worst case, non-uniform costs	21
3.2.1 A warm up: $O(\log n / \log \log n)$ -approximation algorithm for $T V, c C_{max}$	21
3.2.2 An $O(\sqrt{\log n})$ -approximation algorithm for $T V, c C_{max}$	24
3.2.3 QPTAS for $T V, c C_{max}$	25
3.2.4 An $O(\log \log n)$ -approximation algorithm parametrized by the k -up-modularity of the cost function	35
4 Experimental Results	43
4.1 Overview of the experiments	44
4.2 Trees with Uniform Cost ($T V C_{max}$)	47
4.2.1 Random Trees ($T_R V C_{max}$)	47
4.2.2 Random Bounded Degree Trees ($T_\Delta V C_{max}$)	48
4.2.3 Random Bounded Diameter Trees ($T_D V C_{max}$, $D = 6$)	49
4.3 Trees with Non-Uniform Costs ($T V, c C_{max}$)	52
4.3.1 Random Trees with Uniform Cost Distribution ($T_R V, c \sim \mathcal{U} C_{max}$)	52
4.3.2 Random Trees with Binomial Cost Distribution ($T_R V, c \sim \mathcal{B} C_{max}$)	58
4.3.3 Random Trees with Exponential Cost Distribution ($T_R V, c \sim \mathcal{E} C_{max}$)	64
4.3.4 Random bounded degree trees with Uniform Cost Distribution ($T_\Delta V, c \sim \mathcal{U} C_{max}$)	71
4.3.5 Random bounded degree trees with Binomial Cost Distribution ($T_\Delta V, c \sim \mathcal{B} C_{max}$)	77
4.3.6 Random bounded degree trees with Exponential Cost Distribution ($T_\Delta V, c \sim \mathcal{E} C_{max}$)	83
4.3.7 Random bounded diameter trees with Uniform Cost Distribution ($T_D V, c \sim \mathcal{U} C_{max}$)	90
4.3.8 Random bounded diameter trees with Binomial Cost Distribution ($T_D V, c \sim \mathcal{B} C_{max}$)	97

4.3.9	Random bounded diameter trees with Exponential Cost Distribution ($T_D V, c \sim \mathcal{E} C_{max}$)	103
4.3.10	Exponential Runtime Growth Analysis of the Exact Algorithm	109
5	Summary	110

Glossary

G A graph.

$V(G)$ The set of vertices of a graph.

$E(G)$ The set of edges of a graph.

n The number of vertices, $n = |V(G)|$.

m The number of edges, $m = |E(G)|$.

T A tree (connected acyclic graph).

$N_G(v)$ The set of neighbors of vertex v in graph G , $N_G(v) = \{u \in V(G) \mid uv \in E(G)\}$.

$\deg_G(v)$ The degree of vertex v in graph G (number of incident edges).

$\Delta(G)$ Maximum degree of vertices in graph G .

$d(u, v)$ The distance between vertices u and v (length of shortest path).

$\text{diam}(T)$ Diameter of tree T (maximum distance between any two vertices).

c Query cost function, $c : V \rightarrow \mathbb{R}^+$, assigning a cost to each vertex.

$c(v)$ The cost of querying vertex v .

D A decision tree used to identify target vertices through queries.

$Q_D(G, x)$ The sequence of queries made to locate target x in graph G using decision tree D .

$C_G(D, c)$ Worst-case search cost of decision tree D , $C_G(D, c) = \max_{v \in V} \sum_{q \in Q_D(G, v)} c(q)$.

$\text{OPT}(G, c)$ Optimal search cost, $\text{OPT}(G, c) = \min_D C_G(D, c)$.

T_R Random tree generated via Prüfer sequence.

T_Δ Bounded degree tree with maximum degree Δ .

T_{diam} Bounded diameter tree with diameter at most diam .

$\mathcal{U}(a, b)$ Uniform distribution: all values in $[a, b]$ equally likely.

$\mathcal{B}(n, p)$ Binomial distribution: number of successes in n trials with probability p .

$\mathcal{E}(\lambda)$ Exponential distribution with rate parameter λ .

$c \sim \mathcal{U}$ Query costs drawn from uniform distribution.

$c \sim \mathcal{B}$ Query costs drawn from binomial distribution.

$c \sim \mathcal{E}$ Query costs drawn from exponential distribution.

$\alpha || \beta || \gamma$ Three-field notation: α = search space, β = query characteristics, γ = objective.

Aligned Cost A modified cost measure for aligned decision trees that ignores light queries with down responses.

Approximation Ratio The ratio $\frac{\text{ALG}}{\text{OPT}}$ between algorithm's solution and optimal solution.

Binary Search A search strategy that repeatedly divides the search space by repeatedly querying the median vertex.

Connected Component A maximal connected subgraph where every pair of vertices has a path between them.

Cycle A closed path where the first and last vertices are the same.

Decision Tree A rooted tree whose nodes represent ordering of queries to be made depending on the previous responses.

Down Response A response to query v in rooted tree T where the returned component does not contain the root $r(T)$.

EPTAS Efficient Polynomial-Time Approximation Scheme: PTAS with running time $f(\epsilon) \cdot \text{poly}(n)$ where f depends only on ϵ .

Forest An acyclic graph; a disjoint union of trees.

FPTAS Fully Polynomial-Time Approximation Scheme: PTAS with running time $O(\text{poly}(n, 1/\epsilon))$ time.

Heavy Module A maximal connected subset $H \subseteq V(T)$ where every vertex in H is heavy.

Heavy Vertex A vertex v with query cost $c(v) > pk$ for some parameter p and box size k .

Induced Subgraph For $S \subseteq V(G)$, the subgraph $G[S]$ with vertex set S and all edges from $E(G)$ with both endpoints in S .

Light Vertex A vertex v with query cost $c(v) \leq pk$ for some parameter p and box size k .

Path A sequence of vertices where each adjacent pair is connected by an edge.

PTAS Polynomial-Time Approximation Scheme: $(1 + \epsilon)$ -approximation algorithm running in polynomial time for fixed ϵ .

QPTAS Quasi-Polynomial-Time Approximation Scheme: $(1 + \epsilon)$ -approximation algorithm running in $n^{O(\text{poly}(\log n/\epsilon))}$ time.

Query Cost The cost $c(v)$ of querying vertex v .

Rooted Tree A tree with a designated root vertex r that establishes parent-child relationships.

Search Cost The total cost of all queries performed to identify a target vertex.

Subtree A connected subgraph of a tree.

Up Response A response to query v in rooted tree T where the returned component contains the root $r(T)$.

Vertex Ranking A labeling $l : V \rightarrow \{1, 2, \dots, k\}$ where for each pair u, v with $l(u) = l(v)$, there exists z on the path between u and v with $l(z) > l(v)$.

Worst Case The maximum search cost over all possible target vertices.

Chapter 1

Introduction

1.1 The search problem

The Binary Search is a classical algorithm used to efficiently locate a hidden target element in a linearly ordered set. To do so, the searcher repeatedly picks the median element of such set, performs a comparison operation and in constant time learns whether it is the target and if not, whether it lays above or below the median (For example see Figure 1.1).

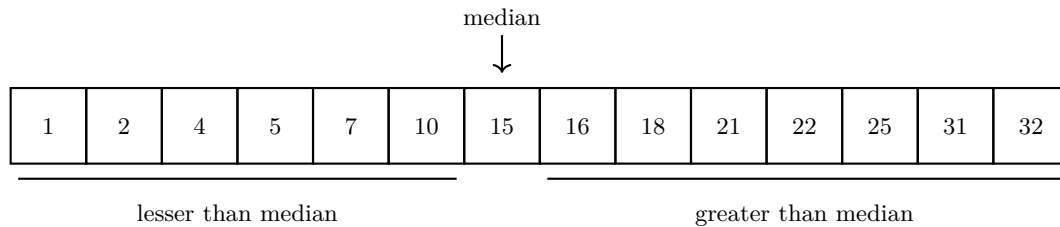


Figure 1.1: Example of a sorted array containing 14 elements. The subarrays with elements lesser than and greater than the median (15) are underlined. If the hidden element is for example 2, then the result of the comparison operation is "below" and the searcher can immediately discard all elements of value above 10. If the target were to be 15, then the comparison operation would yield "equal" meaning that the median element is in fact the target.

The study of searching was initiated by D. Knuth in his seminal book [Knu73] in which he discussed its various variants. However, the origins of the search problem reach the famous Rényi-Ulam game of twenty one questions in which a player is required to guess an unnamed object by asking yes-or-no questions¹. Throughout the years, the searching and its variants have been continuously rediscovered under various definitions and names. This hints that the intuitions behind this problem resurface among multiple use cases and research domains. In fact, the search problem in its many variants is deeply connected with many other algorithmic notions including: parallelization of the Cholesky factorization, scheduling join operations in database queries, VLSI-layouts, learning theory, data clustering, graph cuts and parallel assembly of multi-part products. This work aims to serve as a survey of the results obtained for the problem and an experimental analysis of algorithms aimed at solving it.

The importance of searching is also due to its various practical applications. For example consider the following scenario: a complex procedure contains a hidden bug required to be fixed. The procedure is composed of multiple (often nested) blocks of code. In order to find this hidden bug the searcher can perform tests which allow them to check whether a given block of code contains the bug. After performing each such test they learn whether the bug is in or outside of the tested block. This process then continues, until the bug is found. The problem is to find the best testing strategy for the tester in order to find the bug efficiently.

¹Note that in the twenty-one questions game one answer to a question may be a lie.

1.2 Problem statement

Tree Search More formally, we model the search space as a tree T . The *Vertex Tree Search Problem* is as follows: Among vertices of T there is a hidden target vertex x which is required to be located². During the search process, the searcher is allowed to perform queries, each about a chosen vertex $v \in V(T)$. In constant time the oracle responds whether the target is v and if not, it identifies which connected component of $T - v$ contains x . Upon learning this information the searcher then iteratively picks the next vertex to query until the target is found. The goal is to create a strategy of searching, which is an *adaptive* algorithm that provides the next query based on the previous responses, which optimizes a given criterion (e.g. the worst case or average case search time). One may also define an analogous process in which the queries concern edges. After a query to an edge e the searcher learns which connected component of $T - e$ contains the target. We will call this problem the *Edge Tree Search Problem*. For a visual example for both query models see Fig 1.5.

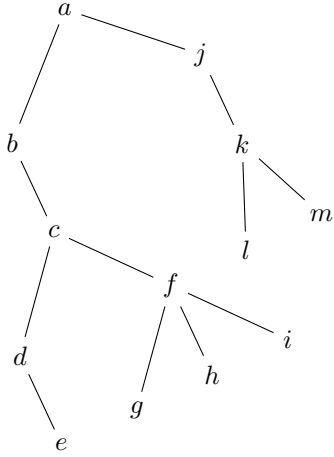


Figure 1.2

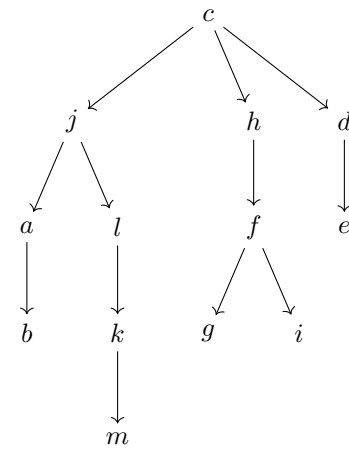


Figure 1.3

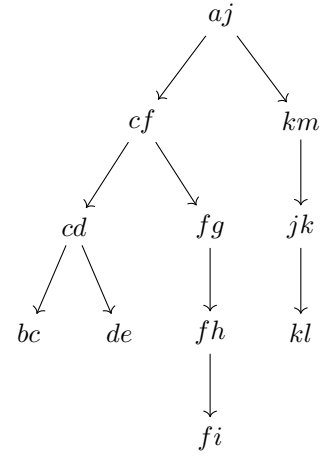


Figure 1.4

Figure 1.5: Sample input tree T (Figure 1.2) and two decision trees for T : one for the Vertex Tree Search Problem (Figure 1.3) and one for the Edge Tree Search Problem (Figure 1.4).

When the input tree is a path both problems become the classical binary search in the linearly ordered set.

Decision Trees and their cost A natural way to encode a strategy is to represent it as a decision tree. A *decision tree* D is a rooted tree in which each vertex represents a query and each edge represents a possible response. The search is conducted by choosing as the next query the root of D . After receiving the response (if the search is not terminated) the searcher moves along the edge $e = (r, r_e)$ incident to r associated with the response. The process then recurses in D_{r_e} until the target is found. It should be noted, that this is far from the only viable way of encoding the search strategy. The choice of the data structure used is a matter of taste and often leads to a simpler design or analysis of the algorithms.

In order to sensibly talk about the quality of such strategy we need to measure its cost. The cost of locating a vertex x using a strategy $\mathcal{A}$ is the amount of queries required to be performed to find x using $\mathcal{A}$. The two most intuitive ways to measure the cost of $\mathcal{A}$ are:

- The worst case search time which is maximum of costs of $\mathcal{A}$ over all vertices.
- The average case which is the sum of costs of $\mathcal{A}$ over all vertices³.

Even though similar, these two criteria usually differ in their analysis and algorithms constructed for them usually exploit different properties of the input. It is often the case that greedy

²It should be pointed out that the target vertex might not always be the same across multiple searches.

³Note that the average search time and the sum of search times are equivalent up to a constant factor of n .

heuristics perform much better when we measure the average case cost of the decision trees created by them. In contrast, for the worst case, the best known solutions often require some intricate dynamic programming procedure as an essential subroutine. Interestingly enough, it is not hard to show that given two decision trees: one with good performance in the average case and one with good performance in the worst case, a simple algorithm can be used to create new decision tree with fairly good performance for both metrics [SLC14].

Costs So far, we assumed all queries to incur uniform cost. However it often might be a case that a particular query is more expensive than others. For example, determining the value of some complex comparison operation for two large objects may take a substantial amount of time. In such cases we associate with each query a cost function. To calculate the cost of finding a target x using a decision tree D , instead of measuring the amount of queries made using D , we measure the sum of their costs. The worst case and the average case criteria are then defined according to these new values.

1.3 Motivations and applications

Fast retrieval of information in graph structures is a fundamental problem in computer science and related fields. Graphs are widely used to model complex relationships and structures in various domains, including social networks, transportation systems, biological networks, and communication networks. Efficiently searching for specific information within these graphs is crucial for applications such as data mining, network analysis, and decision-making processes. When the underlying search space provides non-local information about the target, the search process can be accelerated, since each query rules out a large set of possible target locations. The goal of designing search strategies is to find ways of exploiting this property as effectively as possible.

Searching arises in various practical problems, but it can also be reformulated, to fit a wide range of real-life applications. Searching in graphs can be used to model a variety of problems, of which some may initially appear unrelated. These include:

1. Automated bug detection in computer code [Riv+75; DW22; DW24; Szy25b]. This use case involves locating the source of an error in a program’s semantic structure by performing unit tests, each verifying whether the tested code segment contains the error. The resulting graph is a tree in which each child node represents a subsection of a given code segment. The objective is to minimize the total time required to identify the erroneous code segment by strategically selecting which segments to test.
2. Version control systems and regression bug identification [CDL22]. In software development projects using Version Control Systems (VCS) like Git or Mercurial, the project’s evolution forms a Directed Acyclic Graph (DAG) where vertices represent commits (project versions) and edges model changes between versions. When a regression bug is discovered, developers must identify the specific commit that introduced the error by performing queries on historical commits. Each query involves testing a commit to determine whether it contains the bug or is clean, which can be extremely time-consuming (potentially requiring compilation, extensive testing, or manual verification). When the input DAG is a tree, this problem reduces to the tree search problem.
3. Hierarchical clustering of data [Das16; DL05; CC17]. In this application, the objective is to construct a hierarchical clustering of data points by querying pairwise similarities. The data points are represented as nodes in a graph, and edges represent similarity measures. By querying specific pairs of nodes, one can partition the data points into separate clusters, thereby building a hierarchical clustering of data.
4. Bayesian active learning problem [BH08; CLS16; LLM; SLC14; BBS12; Car+04; KPB99; CLS14; KZ13]. In this scenario, the goal is to determine a correct hypothesis among the possibly very large set of possibilities. To do so the learner can execute tests which sample the features of the hypothesis revealing a partial information about the target. Each test is a partition of the input space. By strategically selecting features to test, one can build a decision tree that minimizes the

classification time. If the partitions incurred by the tests form a tree like hierarchy this problem reduces to the graph search problem. This problem has applications in various domains, including medical diagnosis, fault detection in systems, and criminal investigations.

5. VLSI routing and layer assignment [SDG92]. In VLSI circuit design, one is required to find a partition of *nets* connecting *terminals* into separate conductor layers in order to ensure that no such nets cross each other. In the *generalized river routing* variant of this problem, placing a net on each layer may incur different costs. Therefore, it is beneficial to assign as many nets to the cheapest layers as possible. Decision trees model the sequential layer assignment choices, where each level of the decision tree corresponds to the set of nets assigned to a layer. The objective is to minimize the total cost of layer assignments while ensuring that no nets cross each other.
6. Parallel assembly of multi-part products from their components [DK97; Der06]. The objective is to assemble complex products by performing assembly operations in parallel. The components of the product are represented as nodes in a graph, and hyperedges represent groups of such components which at some point need to be joined together. We assume that no component can undergo two join operations simultaneously. Therefore, the set of join operations performed at any given time is a matching. It can be showed that by reversing the ordering of queries given by the shallowest decision tree of the hyperedge search problem on the graph representing the query plan, one can obtain an optimal schedule for the join operations. A special case of this problem includes scheduling of parallel database join operations [DK06; Der01; Der00]. Here, the objective is to schedule join operations required to process a database query in a parallel manner. Each table is represented as a node, and edges represent required join operations.
7. Parallel Cholesky factorization of sparse matrices [Der03]. In linear algebra, the Cholesky factorization of a symmetric positive-definite matrix can be parallelized by exploiting its sparsity structure. The matrix is represented as a graph, where nodes correspond to matrix rows/columns and edges represent non-zero entries. The *elimination tree* which is equivalent to a decision tree captures column dependencies: if column i is the parent of column j in this tree, then i must be computed before j . Vertices belonging to the same layer of the elimination tree can be therefore processed in parallel. The objective is to construct an elimination tree with minimum height, thereby minimizing the number of parallel steps required for factorization.
8. Fast implementation of distance and shortest path queries [Ang18]. In this application, the goal is to efficiently answer distance and shortest path queries in large graphs. The so called *hub labeling* of a graph, is a data structure computed during the preprocessing of the graph enabling fast query responses. The hub labeling is an assignment of labels to each vertex, where each label contains a subset of vertices (hubs) such that for any pair of vertices, their labels contain a common hub on the shortest path between them. When the input graph is a tree, the problem of constructing an optimal hub labeling is equivalent to the tree search problem, since it can be proven that any optimal hub labeling can be transformed to a hierarchical form, in which the labeling of each vertex v corresponds to the sequence of queries made when searching for v using the corresponding decision tree.
9. Locating sources of river pollution. In this application we are required to locate a hidden source of pollution in a river. To do so, we can probe the river in certain points. Each test reveals whether the sampled location is polluted: since the pollution is carried downstream with the current its detection certifies that the source of the pollution must be located upstream. Otherwise, if the water is clean, the source must be located in a different creek. The river's hierarchical structure naturally forms a tree where each confluence represents a branching point. This application extends to other hierarchical distribution systems such as water supply networks, natural gas pipelines, or electrical grids where fault localization follows similar principles.

Additionally, the graph search problem is also important from a theoretical point of view. In particular, for the case of uniform costs, the depth of the optimal decision tree is in fact equivalent to a well-known graph parameter called the *tree-depth* [GHT12]. This parameter, tightly connected to

other graph invariants like tree-width, is of wide interest in graph theory and parametrized complexity [Bod+95; NO06; BDO23]. The significance of the non-uniform cost model, is due to the fact that it is one of the rare graph problems which are NP-hard already on trees. This happens because the decision trees have a non local structure, which in turn yields most of the classic dynamic programming techniques inapplicable. Approximation algorithms presented hereby, are therefore of particular interest, since they provide efficient ways of obtaining solutions of guaranteed quality for problems with such characteristics. It is interesting whether similar techniques can be applied to other problem, with a similar structure. For example the conflict-free connection coloring which differs only slightly from the Tree Search Problem since it requires that each path in the tree contains a vertex/edge with a unique color [Cha+21], a relaxed condition of the ranking coloring which is equivalent to TSP.

1.4 The three field notation for the search problem

The multiplicity of variants for the problem motivates the usage of unified notation for the Graph Search Problem. Throughout this work we will use the following three field notation resembling the notation commonly used in Task Scheduling problems. It will consists of the three fields: α , β and γ . The α field is the search space environment field resembling the machine environment. The β field is the query characteristics which resembles the job characteristics. The γ field is the objective function which we are trying to optimize. In order not to confuse the two notations we will separate the fields for the graph searching notation with double lines $||$ instead of $|$. The following table showcases example variants which may be considered:

α - search space	β - query characteristics	γ - objective value
P - paths	E - edge queries	C_{max} - maximum search time
T - trees	V - vertex queries	$\sum C_i$ - average search time
$POSET$ - POSETs	Q - any queries	$\sum U_i$ - throughput
G - graphs	c - cost function on queries	F_{max} - maximum flow time
HT - hypertrees	w - weight function on vertices	$\sum F_i$ - average flow time
HG - hypergraphs	d - due dates/deadlines	L_{max} - maximum lateness
	r - release times	$\sum L_i$ - average lateness
	$prec$ - precedences	T_{max} - maximum tardiness
		$\sum T_i$ - average tardiness

Table 1.1: Sample values for the three field notation for the Graph Search Problem.

Note that due to the limited scope of this thesis most of the above variants will not be considered in this work. However, we believe that this notation may prove useful in future works regarding the Graph Search Problem. The striking resemblance between these two notations suggests that we can view the Graph Search Problem as a specific variant of Task Scheduling, in which the search strategy is the schedule and the queries are the jobs. At the start of the schedule there is only one machine processing the queries. After completing each such job, the machine that was processing it is replaced by a new set of machines, one for each possible response to the query. Additionally, from this moment on, each such machine only processes the queries to vertices belonging to response associated with it.

In contrast to the classical scheduling the Graph Search Problem is not nearly as explored and most of the variants which can be constructed using the table above are not even mentioned in the literature. It also seems, that the Graph Search Problem is usually harder than the corresponding scheduling variants. For example, the best algorithm known for the NP-hard variant $T||V, c||C_{max}$ achieves an $O(\sqrt{\log n})$ -approximation [Der+17]. A somewhat similar scheduling problem $P||C_{max}$ has a simple $\frac{4}{3}$ -approximation algorithm based on sorting the jobs according to their costs [Gra69], admits a PTAS for an unbounded number of machines [Leu89] and if the number of machines is bounded an FPTAS can be obtained [Sah76].

1.5 Many names, one problem

Throughout recent years the Graph Search Problem has been continuously rediscovered under various names and definitions. The following list consists of different formulations under which the problem have been studied in the context of graphs:

- Binary Search [Knu73; OP06; Der+17; DMS19; EKS16; DW22; DW24; DŁU25; DGW24; DŁU21; DGP23],
- Tree Search Problem [Jac+10; Cic+14; Cic+16],
- Binary Identification Problem [Cic+12; KZ13],
- Ranking Colorings [Der06; Der08; DK06; DN06; LY98],
- Ordered Colorings [KMS95],
- Elimination Trees [Pot88],
- Hub Labeling [Ang18],
- Tree-Depth [NO06; BDO23],
- Partition Trees [Høg+21; Høg24],
- Hierarchical Clustering [CC17],
- Search Trees on Trees [BK22; Ber+22],
- LIFO-Search [GHT12].

Various different problem definitions stem from the learning theory including:

- Decision Tree [LN04; LLM; GNR10; SLC14],
- Bayesian Active Learning [GKR10; Das04; BH08],
- Discrete Function evaluation [CLS14],
- Tree Split [KPB99],
- Query Selection [BBS12].

1.6 Related work

In what follows, we summarize the most important and relevant results, organized according to the query model and the objective function⁴.

Vertex queries, worst-case cost ($G||V, c||C_{max}$)

When the input graph is a tree and all costs are uniform, the problem is solvable in linear time [Sch89; OP06] and $O(\log n)$ queries always suffice. Beyond trees, the problem is known to be NP-hard even in: chordal graphs [DN06], bipartite and co-bipartite graphs [Bod+98]. By combining the results of [Bod+95] and [FHL05] one can obtain a general $O(\log^{3/2} n)$ -approximation algorithm. Additionally, the problem is solvable in polynomial time for graphs with bounded treewidth [Bod+98].

In the case of non-uniform costs, the problem is known to be NP-hard even for trees [Der06; Cic+12; Cic+16], for which there exists an $O(\sqrt{\log n})$ -approximation algorithm [Der+17]. Further improvements are possible for restricted classes of the cost function. These results include:

- A parametrized PTAS running in $O((cn/\epsilon)^{2c/\epsilon})$ time, where c is the largest cost [DN06],
- A 2-approximation algorithm for down-monotonic cost functions [DW22],
- An 8-approximation algorithm for up-monotonic cost functions [DW22; DW24]

For general graphs with non-uniform costs, the problem can be reduced to the uniform-cost case by rounding all costs to polynomial values and applying a reduction similar to that in [DN06], which yields an $O(\log^{3/2} n)$ -approximation.

⁴Note that the notion of vertex weights is relevant only in the average-case setting.

Vertex queries, average-case cost ($G||V, c|| \sum C_i$)

For trees with non-uniform weights and uniform query costs, a PTAS running in $O(n^{2/\epsilon+1})$ time exists [BK22]. This was later improved to an FPTAS with running time $O\left((1/\epsilon)^{2/\log_2 3} \cdot n^{1+4/\log_2 3} \cdot \log^2(n/\epsilon)\right)$ [Ber24]. Every optimal decision tree has height at most $O(\log w(T))$, and a simple decision tree that always queries the weighted centroid achieves a 2-approximation [Ber+22]. Beyond trees, the problem is NP-hard even for graphs with treewidth at most 15, as well as for dense graphs with uniform weights [Ber24].

For non-uniform costs and non-uniform weights, we recently obtained an $(4 + \epsilon)$ -approximation FPTAS for trees and an $O(\sqrt{\log n})$ -approximation for general graphs [Szy25a]. Additionally, we showed that the problem is NP-hard even for trees of degree at most 16 and for trees of diameter at most 8.⁵

Edge queries, worst-case cost ($G||E, c|| C_{max}$)

For trees with uniform costs, the problem is solvable in linear time [LY98; MOW08], and at most $\frac{\Delta(T)-1}{\log(\Delta(T)+1)-1} \cdot \log n$ queries always suffice [EKS16]. For general graphs, the problem is known to be NP-hard [LY98]. When query costs are non-uniform, an $O(\log n)$ -approximation was given [Der06], later improved to $O(\log n / \log \log \log n)$ [Cic+12], then to $O(\log n / \log \log n)$, and finally, by reduction from the vertex version of the problem, to $O(\sqrt{\log n})$ [Der+17].

Edge queries, average-case cost ($G||E, c|| \sum C_i$)

For trees with uniform costs, the problem is known to be (weakly) NP-hard even for trees with diameter at most 4 and for trees with degree at most 16 [Jac+10]. Every optimal decision tree has height at most $O(\Delta \log w(T))$, and there exists an FPTAS running in $\text{poly}(n^{\Delta(T)}/\epsilon)$ time. A simple greedy algorithm that always queries the edge which splits the weights most evenly achieves a 2-approximation [Jac+10], later improved to ϕ in [Cic+14] and $3/2$ in [Høg24] which is tight.

For uniform weights and non-uniform costs, there exists an 6.75-approximation algorithm for trees and an $O(\sqrt{\log n})$ -approximation algorithm for general graphs [DL05; CC17]. It can be shown that the algorithm for the vertex query variant can be adapted to achieve $4 + \epsilon$ -approximation FPTAS for trees as well [Szy25a]. On the hardness side, the problem cannot be approximated within any constant factor if the *Small Set Expansion Hypothesis* holds [CC17], even when all costs are uniform.

1.7 The aim of the thesis

Hereby, we will be mostly concerned with the situation in which the input graph is tree. A motivation for this is twofold. Firstly, trees come up most often in the practical scenarios regarding the problem. Secondly, from the algorithmic perspective, the most interesting and structural results are obtained for them. Beyond that, most of the algorithms with provable guarantees follow some simple greedy rule and the achieved approximations are far from the objective value. For example, the problem $T||V||C_{max}$ is solvable in linear time (the algorithm is non-trivial) [Sch89]. If we however allow arbitrary graphs $(G||V||C_{max})$ then it becomes NP-hard even in chordal graphs [DN06] and the best known approximation in general case is $O\left(\log^{\frac{3}{2}} n\right)$ which is trivially obtained via an almost blackbox use of the tree decomposition of the graph [Bod+98]. We will also be mostly concerned with the vertex query variant of the problem, since it is usually, the more general variant and we will only regard the worst case criterion in order not to inflate the already wide scope of this thesis.

Throughout this work we use our new three field notation. Its usage is motivated by the large amount of variants of the problem encountered in the literature. We hope that such notation will be used in future works regarding the Graph Search Problem, similarly as in the case of the Task Scheduling notation.

We present an unified theoretical analysis of the state-of-the-art algorithms for the Tree Search Problem with non-uniform query costs. Importantly we show a new formulation of the QPTAS from [Der+17], which is both more intuitive then the original result and slightly faster. This procedure then

⁵These results will appear as a part of the proceeding of WALCOM 2026.

can be used to obtain an $O(\sqrt{\log n})$ -approximation algorithm for the problem via generic reduction. Additionally, we present a new result which shows how to use the QPTAS to obtain an $O(\log \log n)$ -approximation algorithm running in time parametrized by an input parameter k denoting the k -up modularity of a the cost function. This concept generalizes the notion of monotonic cost functions introduced in [DW22; DW24] for which a constant factor approximation is known. In fact monotonicity of the cost function is equivalent to 1-up modularity. However for higher values of k the structure of the costs behaves differently and thus we present a new algorithm which exploits their structure in a novel way.

We conclude a series of experiments aimed at verifying whether the theoretical claims regarding the discussed algorithms are reflected in an experimental setup. We employ randomized techniques to generate various classes of inputs including random trees, random bounded degree trees and random bounded diameter trees along with randomly generated cost functions. We implement the algorithms presented in the theoretical chapter and test their performance both in terms of running time and the quality of the solutions obtained.

1.8 Organization of the work

The second chapter serves as a more formal and detailed introduction necessary for further considerations. In Section 2.3 we recall the basic notions of graph theory required for the analysis and in Section 2.2 we define optimization problems as well as the approximation algorithms and their classes. Then, in Section 2.3 we formally state the general Graph Search Problem we are interested in throughout this work.

In the third chapter we focus ourselves on the formal analysis of the worst-case variants of the search problem on trees, including the presentation of the most interesting algorithmic results known for this problem as well as few new contributions. We begin with the exact dynamic programming algorithm for the uniform cost query model due to [OP06; MOW08] in Section 3.1. In Section 3.2.1 we show an $O(\log n / \log \log n)$ -approximation algorithm by [Cic+16] adapted to the vertex query model which is based on an exact exponential time procedure running in $O(2^n n)$ time. Then, in Section 3.2.2 we show an $O(\sqrt{\log n})$ -approximation algorithm following a similar idea by [Der+17]. This result is based on the QPTAS presented in the later work. However, the construction of the original procedure is counter-intuitive. To alleviate this inconvenience, in Section 3.2.3 we restate the QPTAS by showing a novel dynamic program procedure based on the algorithm used for $T||V|| \sum C_i$ [Jac+10; Ang18]. However, it should be noted that this is not a straightforward adaptation and required significant strengthening of the basic procedure. As a second corollary of the aforementioned QPTAS, in Section 3.2.4 we describe how to use it to obtain an $O(\log \log n)$ -approximation algorithm for the Tree Search Problem running in time $k^{O(\log k)} \cdot n^{O(1)}$, where k is the k -up modularity of the cost function.

In the fourth chapter we present the results of the computational experiments conducted to evaluate the practical performance of the algorithms presented in previous chapter. In Section 4.1 we describe the experimental setup, including: tree generation methods and cost distributions used. For the uniform cost model, in Section 3.1 we evaluate the performance of the exact algorithm on random trees, bounded degree trees, and bounded diameter trees. For the non-uniform cost model, in Sections 4.3.1 to 4.3.9 we measure the approximation quality and runtime of the $O(\log n / \log \log n)$ -approximation, $O(\sqrt{\log n})$ -approximation, and $O(\log \log n)$ -approximation algorithms across various tree structures and cost distributions (uniform, binomial, and exponential). Additionally, we compare these approximation algorithms against the optimal solutions computed by the exact exponential time algorithm for small instances. Finally, in Section 4.3.10 we present regression analysis confirming that the runtime of the exact dynamic programming algorithm in fact follows an exponential function with base much lower than 2.

The fifth chapter serves as a summary of our considerations and points the further research directions regarding this field.

Chapter 2

Notions and Definitions

2.1 Graph theory

A *graph* is a pair $G = (V(G), E(G))$ where $V(G)$ is the set of *vertices* and $E(G)$ is the set of *edges* which are unordered pairs of vertices. We denote $n(G) = |V(G)|$ and $m(G) = |E(G)|$, which are often abbreviated as n and m when the graph is clear from context. For $u, v \in V(G)$ by uv we denote the edge which connects them. A *subgraph* of a graph G is another graph G' formed from a subset of the vertices and edges of G . For any $V' \subseteq V(G)$ by $G[V']$ we denote the *subgraph induced* by V' in G (i. e. for every $u, v \in V'$ if $uv \in E(G)$, then also $uv \in E(G')$). For graphs G and H we denote by $G|H$ the graph G restricted to vertices of H , i. e., $G|H = G[V(H)]$.

For $v \in V(G)$, we denote by $G - v$ the set of connected components of G after removing vertex v and all its incident edges. Similarly, for $e \in E(G)$, we denote by $G - e$ the set of connected components of G after removing edge e while keeping all vertices. Additionally, by $G - V'$ we denote the set of connected components occurring after deleting all vertices in V' from G . The set of *neighbors* of $v \in V(G)$ will be denoted as $N_G(v) = \{u \in V(G) | uv \in E(G)\}$ and the set of neighbors of subgraph G' of G as $N_G(G') = \bigcup_{v \in V(G')} N_G(v) - V(G')$. By $\deg_G(v) = |N_G(v)|$ we will denote the *degree* of v in G . By $\Delta(G) = \max_{v \in V(G)} \{\deg(v)\}$ we denote the degree of G .

A *cycle* is a non-empty sequence of vertices in which for every two consecutive vertices u, v : $uv \in E(G)$ and only the first and last vertices are equal. A *tree* T is a connected graph that contains no cycle. A *forest* is a (not necessarily connected) graph that contains no cycle. A *path* P is a tree such that $\Delta(P) = 2$. A *rooted tree* is a tree in which one vertex has been designated as the root. For a rooted tree T , by $r(T)$ we denote the root of T . Let $v \in V(T)$. The set of *children* of v in T will be denoted as $\mathcal{C}_T(v)$. The *outdegree* of v in T will be denoted as $\deg_T^+(v) = |\mathcal{C}_T(v)|$. By $P_T(u, v) = T \setminus \{u, v\}$ we denote a path of vertices between u and v in T (excluding u and v). Analogously, for $V_1, V_2 \in V(T)$ we define $P_T(V_1, V_2) = T \setminus (V_1 \cup V_2)$. For any we denote the minimal connected subtree of T containing all vertices from V' by $T[V']$.

2.2 Optimization problems

A *minimization problem* is one in which given an input I , the set of valid solutions S and a cost function $C : S \rightarrow \mathbb{R}^+$ we are required to find a solution $s^* \in S$ such that $C(s^*) = \min_{s \in S} \{C(s)\}$. Analogously, a *maximization problem* is one in which we are required to find a solution $s^* \in S$ such that $C(s^*) = \max_{s \in S} \{C(s)\}$. For both types of problems we define $\text{OPT}(I) = C(s^*)$. Given an instance (I, S, C) of a minimization problem such that $|I| = n$, an $\alpha(n)$ -approximation algorithm is an algorithm which always outputs a solution s such that:

$$\frac{C(s)}{\text{OPT}(I)} \leq \alpha(n).$$

Analogously, for a maximization problem such an $\alpha(n)$ -approximation algorithm is an algorithm which always outputs a solution s such that:

$$\frac{C(s)}{\text{OPT}(I)} \geq \alpha(n).$$

If $\alpha(n) = O(1)$ we say that the algorithm is a constant factor approximation algorithm for I .

If $\alpha(n) = 1$ we say that the algorithm is an exact algorithm for I . For a minimization problem if for every $0 < \epsilon \leq 1$ the algorithm provides a $(1 + \epsilon)$ -approximation (or a $(1 - \epsilon)$ -approximation in case of a maximization problem) and:

- Runs in time $\text{poly}(n/\epsilon)$, then it is called a Fully-Polynomial Time Approximation Scheme (FP-TAS).
- Runs in time $f(\epsilon) \cdot \text{poly}(n)$ for some computable function f , then it is called a Efficient-Polynomial Time Approximation Scheme (EPTAS).
- Runs in time $n^{O(1/\epsilon)}$, then it is called a Polynomial Time Approximation Scheme (PTAS).
- Runs in time $n^{\text{poly}(\log n/\epsilon)}$, then it is called a Quasi-Polynomial Time Approximation Scheme (QPTAS).

2.3 The Graph Search Problem

Below we list the definitions regarding the Graph Search Problem. Since the problem has a modular form and one can almost freely swap criteria and constraints, the number of separate variants is very large. Due to this we will present a general Graph Search Problem.

The *Graph Search Instance* consists of a pair $G = (V(G), E(G))$. Among $V(G)$ there is a unique hidden target element x which is required to be located. During the *search process* the searcher is allowed to iteratively perform *queries*, each about a chosen vertex v (or alternatively an edge e). If the answer to a query is affirmative, then v is the target, otherwise a connected component $H \in G - v$ is returned such that $x \in V(H)$ (for the edge version always $H \in G - e$ is returned). Based on this information the searcher narrows the subgraph of G which might contain x until there is only one possible option left.¹

2.3.1 Additional input parameters

As a part of the input we will also allow the cost function. Let $\mathcal{Q}$ be the space of possible queries (either vertex or edge queries). The *cost* of query $q \in \mathcal{Q}$ is then denoted as $c : \mathcal{Q} \rightarrow \mathbb{R}^+$. When all queries have unit cost, i.e., $c(q) = 1$ for all $q \in \mathcal{Q}$, we sometimes denote this by writing $C(D, 1)$ to emphasize that the cost function assigns unit costs.

Note, that it is also sometimes allowed for each vertex to have a *weight function* $c : V(G) \rightarrow \mathbb{R}^+$ on vertices. However, this is only sensible notion for the average case version of the problem which is not considered in this work.

2.3.2 Decision trees, optimization criteria and the Graph Search Problem

Let G be a graph. A decision tree is a rooted tree $D = (V(D), E(D))$, where $V(D) = V(G)$ are the vertices of D and $E(D)$ are the edges of D . It is required that each child of $q \in V(D)$ corresponds to a distinct response to the query at q , with respect to the subtree of candidate solutions that remain after performing all previous queries.

Let $Q_D(G, x)$ denote the sequence of queries made to locate a target $x \in V(G)$ using D , i.e., the sequence of vertices belonging to the unique path in D starting at $r(D)$ and ending at x . We define the worst case cost of a decision tree D in (G, c) as:

$$C_G(D, c) = \max_{x \in V(G)} \left\{ \sum_{q \in Q_D(G, x)} c(q) \right\}.$$

By a slight abuse of notation we will also sometimes use $Q_D(G, x)$ as the set consisting of queries in sequence $Q_D(G, x)$. This is done in order to not inflate the amount of symbols and will not

¹In the vertex query model we require that every vertex must be queried even when such vertex is the last among the candidate set. Note that it is sometimes assumed that in such case, this vertex does not need to be queried which may reduce the cost of the solution. Note that all of the algorithms showed in this work can be altered to take this assumption into account. For the sake of the brevity we do not include them but we encourage the reader to obtain them as an exercise.

become problematic during the analysis of the solutions. Whenever clear from the context, for the clarity of the analysis, we will occasionally drop any of the subscripts or arguments of the C function. We are now ready to define the *Graph Search Problem*:

Generalized Search Problem

Input: Graph G , the query model and the optimization criterion

Output: A valid decision tree D for G with respect to the query model, which optimizes the criterion.

Chapter 3

Theoretical Analysis

The following chapter is concerned with the presentation and theoretical analysis of the algorithms for the Tree Search Problem in its worst-case variants. We partition the analysis into 2 main sections.

Firstly, we present a classic $O(n \log n)$ time algorithm for the uniform cost variant of the problem based on the equivalence of the ranking coloring and the Tree Search Problem [Sch89; OP06; MOW08].

Secondly, for the non-uniform cost model, we present previously known results including an $O(\log n / \log \log n)$ -approximation algorithm inspired by the algorithm for the edge query variant [Cic+16]. We also rewrite the QPTAS from [Der+17] using a somewhat different dynamic program inspired by the work on the average-case variant of the problem [Jac+10; Ang18]. As a result we also obtain an incremental improvement on the running of the QPTAS since the original construction yields an algorithm with running time roughly $n^{112898 \cdot \log n / \epsilon^2}$ while we managed to reduce this to approximately $n^{6965 \cdot \log n / \epsilon^2}$.

We then use this QPTAS to derive a polynomial time $O(\sqrt{\log n})$ -approximation algorithm and a $O(\log \log n)$ approximation algorithm running in $O(k^{O(\log n)} \cdot \text{poly}(n))$ time, where k is the k -up modularity of the cost function.

Table 3.1: Summary of analyzed algorithms with theoretical guarantees

Uniform Costs: $T V C_{max}$	Non-uniform Costs: $T V, c C_{max}$
OPT in $O(n \log n)$ time, Section 3.1	OPT in $O(2^n \cdot n)$ time, Section 3.2.1 $O(\log n / \log \log n)$ -approx. in $\text{poly}(n)$ time, Section 3.2.1 $(1 + \epsilon)$ -approx. in $n^{O(\log n / \epsilon^2)}$ time, Section 3.2.3 $O(\sqrt{\log n})$ -approx. in $\text{poly}(n)$ time, Section 3.2.2 $O(\log \log n)$ -approx. in $k^{O(\log k)}$ time, Section 3.2.4

3.1 Trees, Worst Case, Uniform Costs

The *vertex ranking* of T is a labeling of vertices $l : V \rightarrow \{1, 2, \dots, \lceil \log n \rceil + 1\}$, which satisfies the following condition: for each pair of vertices $u, v \in V(T)$, whenever $l(u) = l(v)$, there exists $z \in \mathcal{P}_T(u, v)$ for which $l(z) > l(v)$. Such a labeling always exists and can be computed in linear time by means of dynamic programming [Sch89; OP06; MOW08]. For a visual example, see Figure 3.4

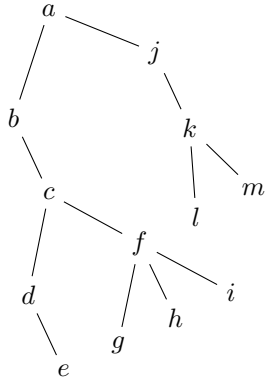


Figure 3.1

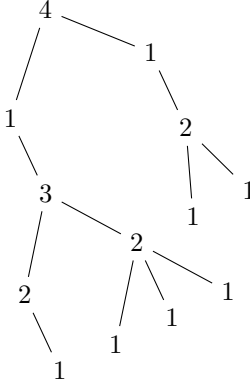


Figure 3.2

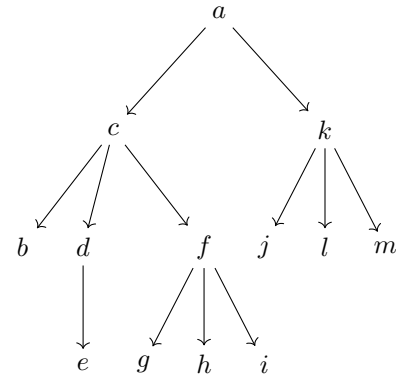


Figure 3.3

Figure 3.4: Sample input tree T (Figure 3.1), vertex ranking labeling l of T (Figure 3.2) and a decision tree D for T built using l (Figure 3.3).

Having a vertex ranking of T , one can easily obtain a decision tree for T using the following procedure:

Algorithm 1: Ranking-based decision tree construction

Procedure RankingBasedDT (T, l)
 Let $z \leftarrow \arg \max_{v \in V(T)} l(v)$.
 $D \leftarrow (\{z\}, \emptyset)$.
 foreach $T' \in T - z$ **do**
 $D_{T'} \leftarrow \text{RankingBasedDT}(T', l)$.
 Attach $D_{T'}$ as a child of z in D .
 return D .

When the input tree has uniform costs and the ranking uses the minimal number of labels, the decision tree built in this way is optimal and never uses more than $\lceil \log n \rceil + 1$ queries [OP06]. Let $\text{rank}(T)$ be the number of colors used by the optimal ranking of T . It is not hard to see that the running time of the above procedure is $O(n \cdot \text{rank}(T))$. We have the following corollary:

Corollary 3.1.0.1. *There exists an $O(n \log n)$ time procedure **RankingBasedDT** that finds the optimal decision tree for the Tree Search Problem when all costs are uniform. Moreover, the depth of such a decision tree, i.e., the worst-case number of queries, is at most $\lceil \log n \rceil + 1$.*

Note that it is in fact possible to implement the above procedure in $O(n)$ time by using more intricate data structures [OP06; MOW08]. However we decided to keep the implementation simple since the running time of this procedure is negligible compared to the other algorithms evaluated in this thesis.

We assume that the input tree is rooted at an arbitrary vertex. Below we show how to calculate the optimal vertex ranking of a given tree T . For any vertex $v \in V(T)$, and any coloring l we define the *visibility sequence* $S(v)$ as following: If there exists a vertex $u \in V(T)$, such that for every $z \in \mathcal{P}_T(u, v)$, $l(u) > l(z)$, then $l(u) \in S(v)$. We also demand that $S(v)$ is sorted decreasingly. For a given label k , $k \in S(v)$, we say that k is *visible* from v . In order to find a coloring using the minimal amount of colors, we will make use of the standard lexicographic order on visibility sequences. Notice, that for any vertex $v \in V(T)$, we have that $\max_{u \in V(T)} \{l(u)\} \in S(v)$. Therefore, a labeling with the

lexicographically lowest $S(r(T))$ is also an optimal. We focus ourselves on finding such labeling. We devise the following dynamic programming procedure which calculates a labeling of T with a minimal visibility sequence of $r(T)$.

Algorithm 2: The CalculateRanking procedure.

Procedure CalculateRanking(T):
 for $1 \leq j \leq \deg_{r(T)}^+$ **do**
 $S(c_j) \leftarrow \text{CalculateRanking}(T_{c_j})$.
 $m \leftarrow$ maximal value belonging to at least two distinct $S(c_j)$.
 $S(v) \leftarrow \bigcup_{j=1}^{\deg_{r(T)}^+} S(c_j)$.
 $l(r(T)) \leftarrow \arg \min_{m < k \leq \log n + 1, k \notin S(v)} \{k\}$.
 Append $l(v)$ to $S(v)$.
 Remove any $l < l(v)$ from $S(v)$.
 return $S(v)$.

The following theorem is by [OP06]:

Theorem 3.1.0.2. *Let T be a tree. Algorithm 2 can be implemented in $O(n)$ time and returns a correct ranking labeling such that $S(r(T))$ is lexicographically optimal.*

3.2 Trees, worst case, non-uniform costs

For non-uniform costs the problem is NP-hard even when restricted to spiders of diameter 6 and binary trees [Cic+12; Cic+16]. A simple greedy heuristic which always queries the middle vertex of the graph achieves a $O(\log n)$ -approximation [Der06]. However one can obtain better results. We begin with the following simple lemma, which will become useful in few arguments:

Lemma 3.2.0.1. *Let T' be a connected subtree of T . Then, $OPT(T') \leq OPT(T)$.*

3.2.1 A warm up: $O(\log n / \log \log n)$ -approximation algorithm for $T||V, c||C_{max}$

This first algorithm is an adapted and simplified version of the algorithm due to [Cic+16] for the edge query model.

Theorem 3.2.1.1. *There exists a polynomial time, $O(\log n / \log \log n)$ -approximation algorithm for the $T||V, c||C_{max}$ problem.*

Proof. To construct a decision tree we will use the procedure from the lemma below:

Lemma 3.2.1.2. *There exists an $O(2^n n)$ algorithm for $T||V, c||C_{max}$*

Proof. The algorithm is a general version of the dynamic programming procedure for paths. We have that:

$$OPT(T, c) = \min_{v \in V(T)} \left\{ c(v) + \max_{H \in T-v} \{OPT(H, c)\} \right\}.$$

Notice, that there are at most $O(2^n)$ different subtrees of T to be checked. Additionally, for each $v \in V(T)$, there are at most $\deg_T(v)$ possible responses to check in the inner max function. Therefore, for each subproblem, there are at most $\sum_{v \in V(T)} \deg_T(v) = 2m = 2n - 2$ comparison operations to be performed. As at each level of the recursion the algorithm considers all possible choices of the next queried vertex v , it necessarily returns the optimal decision tree for T and the claim follows. $\square$

We will denote the above procedure as **Exact**. Let $k = 2^{\lfloor \log \log n \rfloor + 2}$. The basic idea of the Algorithm 3 is as follows: The algorithm is recursive. Let $\mathcal{T}$ be the tree currently processed by the algorithm. If $n(\mathcal{T}) \leq k$ then we call **Exact**($\mathcal{T}, c$) to find the optimal solution in time $2^k k = \text{poly}(n)$.

If otherwise, to build a solution we will firstly define a set $\mathcal{X} \subseteq V(\mathcal{T})$ which will be of size at most k . We build $\mathcal{X}$ iteratively. Starting with an empty set we pick the centroid x_1 of $\mathcal{T}$ which we add to $\mathcal{X}$. Then we take the forest $\mathcal{F} = \mathcal{T} - x_1$, find the largest $H \in \mathcal{F}$, pick its centroid x_2 and append it to $\mathcal{X}$. We continue this in $(\mathcal{F} - H) \cup (H - x_2)$ until $|\mathcal{X}| = k$.

Lemma 3.2.1.3. *For every $H \in \mathcal{T} - \mathcal{X}$ we have that $n(H) \leq n(\mathcal{T}) / \log(n)$.*

Proof. We prove by induction on t that deleting first 2^t centroids from $\mathcal{T}$ each connected components H_t has size at most $n(H_t) \leq n(\mathcal{T}) / 2^{t-1}$. For the case when $t = 0$ we have that after 1 iteration every H_1 has size at most $n(\mathcal{T}) / 2 \leq 2n$ so the base of induction is complete.

Fix $t > 0$ and assume by the induction hypothesis that after 2^{t-1} iterations all components of $\mathcal{T} - \mathcal{X}$ have size at most $n(\mathcal{T}) / 2^{t-2}$. Let p denote the number of such components that have size at least $n(\mathcal{T}) / 2^{t-1}$ so that $p \leq 2^{t-1}$. By the property of centroids, after deleting one centroid from each of these components, each resulting component has size at most $n(\mathcal{T}) / 2^{t-1}$. Therefore, after deleting $n(\mathcal{T}) / 2^{t-1} + p \leq n(\mathcal{T}) / 2^t$ centroids, we have that all resulting components have size at most $n(\mathcal{T}) / 2^t$. This completes the induction step and the claim follows by the choice of k . $\square$

We also define set $\mathcal{Y} \subseteq V(\mathcal{T})$ which consists of vertices in $\mathcal{X}$ and all vertices in $v \in \mathcal{T} \setminus \mathcal{X}$ such that $\deg_{\mathcal{T} \setminus \mathcal{X}}(v) \geq 3$. Furthermore, we define set $\mathcal{Z} \subseteq V(\mathcal{T})$ as a set consisting of vertices in $\mathcal{Y}$ and for every $u, v \in \mathcal{Y}$ such that $\mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset$ and $\mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$ we add to $\mathcal{Z}$ the vertex $\arg \min_{z \in \mathcal{P}_{\mathcal{T}}(u, v)} \{c(z)\}$ (for example see Figure 3.21). We then create an auxiliary tree $\mathcal{T}_{\mathcal{Z}} = (\mathcal{Z}, \{uv | \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Z} = \emptyset\})$ (for example see Figure 3.22). The algorithm builds an optimal decision

tree D_Z for $\mathcal{T}_Z$ by applying the **Exact** procedure for $(\mathcal{T}_Z, c)$. Observe, that D_Z is a partial decision tree for $\mathcal{T}$, so we get that:

Observation 3.2.1.4. $C_{\mathcal{T}_Z}(D_Z) = C_{\mathcal{T}}(D_Z)$.

Then for each $H \in \mathcal{T} - \mathcal{Z}$ we recursively apply the same algorithm to obtain the decision tree D_H and we hang it in D_Z below the unique last query to vertex in $N_{\mathcal{T}'}(H)$ (By Observation 3.2.4.5).

Algorithm 3: Main recursive procedure (k is a global parameter)

```

Procedure DecisionTree ( $\mathcal{T}, c$ ):
  if  $n(\mathcal{T}) \leq k$  then
     $D \leftarrow \text{Exact}(\mathcal{T}, c)$ .
    return  $D$ 
   $\mathcal{X} \leftarrow \emptyset$ .
   $\mathcal{F} \leftarrow \{\mathcal{T}\}$ .
  for  $1 \leq i \leq k$  do
    if  $\mathcal{F} = \emptyset$  then
      break
     $H \leftarrow \arg \max_{H \in \mathcal{F}} \{n(H)\}$ .
     $x \leftarrow$  the centroid of  $H$ .
     $\mathcal{X} \leftarrow \mathcal{X} \cup \{x\}$ .
     $\mathcal{F} \leftarrow \mathcal{F} \cup (H - x)$ .
   $\mathcal{Z} \leftarrow \mathcal{Y} \leftarrow \mathcal{X} \cup \{v \in \mathcal{T}(\mathcal{X}) \mid \deg_{\mathcal{T}(\mathcal{X})}(v) \geq 3\}$ . // Branching vertices in  $\mathcal{T}(\mathcal{X})$ .
  foreach  $u, v \in \mathcal{Y}, \mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset, \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$  do
     $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{\arg \min_{z \in \mathcal{P}_{\mathcal{T}}(u, v)} \{c(z)\}\}$ . // Lightest vertex on path  $\mathcal{P}_{\mathcal{T}}(u, v)$ .
   $\mathcal{T}_Z = (\mathcal{Z}, \{uv \mid \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Z} = \emptyset\})$ .
   $D \leftarrow D_Z \leftarrow \text{Exact}(\mathcal{T}_Z, c)$ .
  foreach  $H \in \mathcal{T} - \mathcal{Z}$  do
     $D_H \leftarrow \text{DecisionTree}(H, c)$ .
    Hang  $D_H$  in  $D$  below the last query to a vertex  $v \in N_{\mathcal{T}}(H)$ .
  return  $D$ 

```

Lemma 3.2.1.5. Let $\mathcal{T}_Z$ be the auxiliary tree. Then, $|V(\mathcal{T}_Z)| \leq 4k - 3$.

Proof. We firstly show that $|\mathcal{Y}| \leq 2k - 1$. We use induction on the elements of set $\mathcal{X}$. For $1 \leq i \leq k$, let x_i denote the i -th centroid added to $\mathcal{X}$. We will construct a family of sets $\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_{|\mathcal{H}|}$, such that for every integer $1 \leq t \leq |\mathcal{X}|$: $|\mathcal{X}_t| = t$ and $\mathcal{X}_{|\mathcal{X}|} = \mathcal{X}$. For each $\mathcal{X}_t$, we will also construct a corresponding set $\mathcal{Y}_t$, eventually ensuring that $\mathcal{Y}_{|\mathcal{X}|} = \mathcal{Y}$. We will build the sets $\mathcal{Y}_t$ to ensure that $|\mathcal{Y}_t| \leq 2t - 1$.

Let $\mathcal{X}_1 = \{x_1\}$, $\mathcal{Y}_1 = \{x_1\}$. This establishes the base case. Assume by induction on $t \geq 1$ that $|\mathcal{Y}_t| \leq 2t - 1$ for some $t > 1$. Let $\mathcal{X}_{t+1} = \mathcal{X}_t \cup \{x_{t+1}\}$ and let $\mathcal{T}_t = \mathcal{T}(\mathcal{X}_t)$. If $x_t \in V(\mathcal{T}_t)$, then $\mathcal{Y}_{t+1} = \mathcal{Y}_t \cup \{x_t\}$. If otherwise, let $y_t \in V(\mathcal{T}_t)$ be the unique vertex, such that $P(x_t, y_t) \cap V(\mathcal{T}_t) = \emptyset$. Then, $\mathcal{Y}_{t+1} = \mathcal{Y}_t \cup \{x_t, y_t\}$. As by induction, $|\mathcal{Y}_t| \leq 2t - 1$ and we add at most two vertices to it to obtain $\mathcal{Y}_{t+1}$, the induction step is complete.

By construction, $\mathcal{X}_{|\mathcal{X}|} = \mathcal{X}$ and $\mathcal{Y}_{|\mathcal{H}|} = \mathcal{Y}$, so $|\mathcal{Y}| \leq 2 \cdot |\mathcal{H}| - 1 \leq 2k - 1$. As paths between vertices in $\mathcal{Y}$ form a tree when contracted, at most $2k - 2$ additional vertices are added while constructing $\mathcal{Z}$ (at most one per path). The lemma follows. $\square$

Lemma 3.2.1.6. Let $\mathcal{T}_Z$ be the auxiliary tree. Then, $\text{OPT}(\mathcal{T}_Z) \leq \text{OPT}(\mathcal{T})$.

Proof. Let D^* be the optimal strategy for $\mathcal{T}(\mathcal{Z})$. We build a new decision tree D'_Z for $\mathcal{T}_Z$ by transforming D^* : Let $u, v \in \mathcal{Y}$ such that $\mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset$ and $\mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$. Let $q \in V(D^*)$ such that $q \in \mathcal{P}_{\mathcal{T}}(u, v)$ is the first query among vertices of $\mathcal{P}_{\mathcal{T}}(u, v)$. We replace q in D^* by the query to the

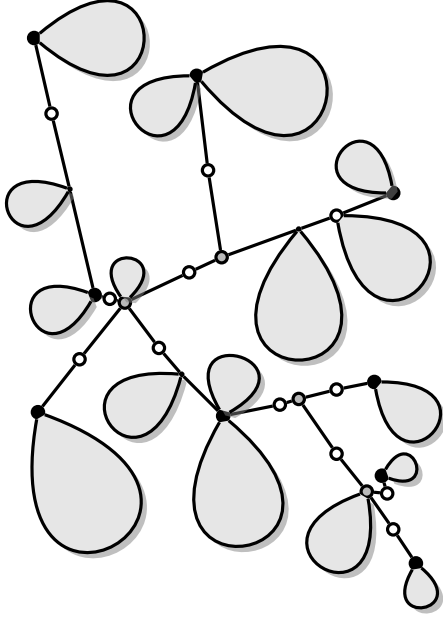


Figure 3.5: Example tree $\mathcal{T}$. Light grey regions represent light subtrees. Black vertices represent $\mathcal{X}$. Gray and black vertices represent $\mathcal{Y}$. White, gray and black vertices represent $\mathcal{Z}$. Lines represent paths of vertices between vertices of $\mathcal{Z}$.

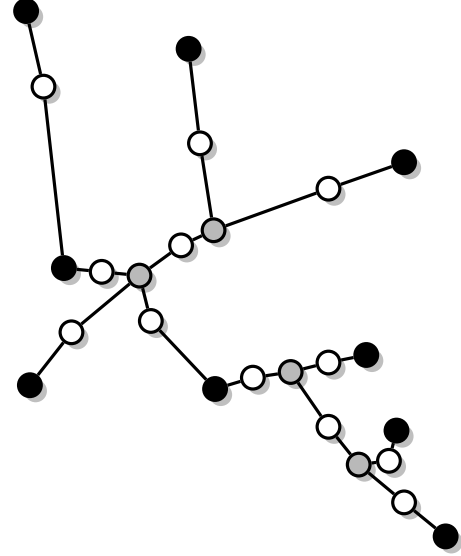


Figure 3.6: Auxiliary tree $\mathcal{T}_Z$ built from vertices of set $\mathcal{Z}$.

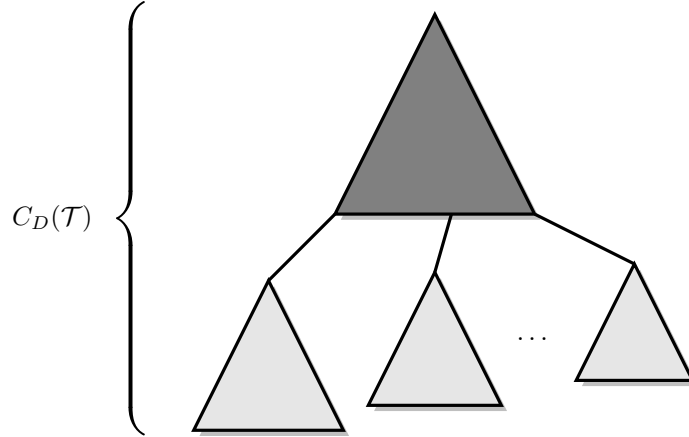


Figure 3.7: The structure of the decision tree D , built by the Algorithm 3. The dark gray subtree represents the decision tree D_Z , obtained by calling the **Exact** procedure for $\mathcal{T}_Z$ and c . Light gray subtrees represent decision trees D_L , built for each $H \in \mathcal{T} - \mathcal{Z}$, by recursively calling **DECISIONTREE** with H and c .

distinct vertex $v_{u,v} \in \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Z}$ and delete all queries to vertices $\mathcal{P}_{\mathcal{T}}(u, v) - v_{u,v}$ from D^* . By construction, $D'_{\mathcal{Z}}$ is a valid decision tree for $\mathcal{T}_{\mathcal{Z}}$ and as for every $z \in \mathcal{P}_{\mathcal{T}}(u, v)$: $c(v_{u,v}) \leq c(z)$ such strategy has cost at most $C_{\mathcal{T}_{\mathcal{Z}}}(D'_{\mathcal{Z}}) \leq \text{OPT}(\mathcal{T}(\mathcal{Z}))$. We get:

$$\text{OPT}(\mathcal{T}_{\mathcal{Z}}) \leq C_{\mathcal{T}_{\mathcal{Z}}}(D'_{\mathcal{Z}}) \leq \text{OPT}(\mathcal{T}(\mathcal{Z})) \leq \text{OPT}(\mathcal{T})$$

where the first inequality is due to the optimality and the last inequality is due to the fact that $\mathcal{T}(\mathcal{Z})$ is a subtree of $\mathcal{T}$ (by Lemma 3.2.0.1). The lemma follows. $\square$

Lemma 3.2.1.7. *Let $D_{\mathcal{T}}$ be the solution returned by the algorithm. Then the approximation factor of such solution is bounded by $\text{APP}_{\mathcal{T}}(D_{\mathcal{T}}) \leq \log n / \log \log n$.*

Proof. Let $\mathcal{T}$ be the tree processed at some level of the recursion and let $D_{\mathcal{T}}$ be the decision tree returned by the algorithm. The proof is by induction on the size of $\mathcal{T}$. We claim that $\text{APP}_{\mathcal{T}}(D_{\mathcal{T}}) \leq \max\{1, \log n(\mathcal{T}) / \log \log n\}$. If $n(\mathcal{T}) \leq k$ then $D_{\mathcal{T}}$ is the optimal decision tree for $\mathcal{T}$ which establishes the base case. Let $n(\mathcal{T}) > k$ and assume that claim holds for every $t < n(\mathcal{T})$. By construction, we have that:

$$\begin{aligned} \text{APP}_{D_{\mathcal{T}}}(\mathcal{T}) &= \frac{C_{\mathcal{T}}(D_{\mathcal{T}})}{\text{OPT}(\mathcal{T})} \\ &\leq \frac{C_{\mathcal{T}}(D_{\mathcal{Z}}) + \max_{H \in \mathcal{T} - \mathcal{Z}} \{C_{\mathcal{T}}(D_H)\}}{\text{OPT}(\mathcal{T})} \\ &\leq \frac{C_{\mathcal{T}_{\mathcal{Z}}}(D_{\mathcal{Z}})}{\text{OPT}(\mathcal{T}_{\mathcal{Z}})} + \max_{H \in \mathcal{T} - \mathcal{Z}} \left\{ \frac{C_{\mathcal{T}}(D_H)}{\text{OPT}(H)} \right\} \\ &\leq 1 + \frac{\log \left(\frac{n(\mathcal{T})}{\log n(\mathcal{T})} \right)}{\log \log n} = \frac{\log n(\mathcal{T})}{\log \log n} \end{aligned}$$

where the first inequality is by construction, the second is by usage of Observation 3.2.1.4, Lemma 3.2.1.6 and Lemma 3.2.0.1 and the last inequality is due to the Lemma 3.2.1.3 and the induction hypothesis. $\square$

Using the fact that the call to the exponential time procedure requires $O(2^{4k-3}(4k-3)) = \text{poly}(n)$ time (Due to Lemma 3.2.1.5), all other computations require polynomial time, and each $v \in V(T)$ belongs to $\mathcal{Z}$ at most once during the execution we get that the overall running time is polynomial in n . $\square$

In the above analysis we lose one factor of OPT per each level of recursion of which there are at most $O(\log n / \log \log n)$. Notice however, that we can allow some more loss (i. e. $c \cdot \text{OPT}$) without affecting the asymptotical approximation factor. As it turns out it is possible to obtain a constant factor approximation for this problem in quasipolynomial time. This is the main idea behind the improvement of the approximation factor for this problem as in such case the size of the set $\mathcal{Z}$ may be greater and less recursion levels are needed which directly improves the approximation.

3.2.2 An $O(\sqrt{\log n})$ -approximation algorithm for $T||V, c||C_{\max}$

We begin with the following proposition [Der+17] about the existence of QPTAS for $T||V, c||C_{\max}$:

Proposition 3.2.2.1. *For any $0 < \epsilon \leq 59$ there exists a $(1 + \epsilon)$ -approximation algorithm for the Tree Search Problem running in $2^{O(\frac{\log^2 n}{\epsilon^2})}$ time.*

The algorithm is very intricate and is deferred to Section 3.2.3.

Theorem 3.2.2.2. *There exists a polynomial time, $O(\sqrt{\log n})$ -approximation algorithm for the $T||V, c||C_{\max}$ problem.*

Proof. We use the same procedure as in the $O(\log n / \log \log n)$ -approximation algorithm, however we set $k = 2\lfloor \sqrt{\log n} \rfloor + 2$ and we swap the exact procedure to the QPTAS with $\epsilon = 1$. The analysis of the algorithm is largely the same, except while evaluating the cost of the resulting decision tree.

Lemma 3.2.2.3. *Let D_T be the solution returned by the algorithm. Then the approximation factor of such solution is bounded by $\text{APP}_T(D_T) \leq 2\sqrt{\log n}$.*

Proof. Let $\mathcal{T}$ be the tree processed at some level of the recursion and let $D_{\mathcal{T}}$ be the decision tree returned by the algorithm. The proof is by induction on the size of $\mathcal{T}$. We claim that $\text{APP}_{\mathcal{T}}(D_{\mathcal{T}}) \leq \max\{1, 2\log n(\mathcal{T}) / \sqrt{\log n}\}$. If $n(\mathcal{T}) \leq k$ then $D_{\mathcal{T}}$ is the optimal decision tree for $\mathcal{T}$ which establishes the base case. Let $n(\mathcal{T}) > k$ and assume that claim holds for every $t < n(\mathcal{T})$. By construction, we have that:

$$\begin{aligned} \text{APP}_{D_{\mathcal{T}}}(\mathcal{T}) &= \frac{C_{D_{\mathcal{T}}}(\mathcal{T})}{\text{OPT}(\mathcal{T})} \\ &\leq \frac{C_{D_{\mathcal{Z}}}(\mathcal{T}) + \max_{H \in \mathcal{T}-\mathcal{Z}} \{C_{D_H}(H)\}}{\text{OPT}(\mathcal{T})} \\ &\leq \frac{C_{D_{\mathcal{Z}}}(\mathcal{T}_{\mathcal{Z}})}{\text{OPT}(\mathcal{T}_{\mathcal{Z}})} + \max_{H \in \mathcal{T}-\mathcal{Z}} \left\{ \frac{C_{D_H}(H)}{\text{OPT}(H)} \right\} \\ &\leq 2 + \frac{2\log\left(\frac{n(\mathcal{T})}{\sqrt{\log n}}\right)}{\sqrt{\log n}} = \frac{2\log n(\mathcal{T})}{\sqrt{\log n}} \end{aligned}$$

where the first inequality is by construction, the second is by usage of Observation 3.2.1.4, Lemma 3.2.1.6 and Lemma 3.2.0.1 and the last inequality is due to the Lemma 3.2.1.3 and the induction hypothesis. $\square$

$\square$

3.2.3 QPTAS for $T||V, c||C_{\max}$

The following algorithm is a simplified version of the QPTAS provided in [Der+17]. The core idea of the algorithm is the same, however, our solution uses the language of decision trees instead of the language of sequence assignments, which makes the algorithm more intuitive.

For the rest of the analysis, without loss of the generality, we will assume that T is rooted in a vertex v minimizing $c(v)$. We will also assume that all costs are normalized, so that $\max_{v \in V(T)} \{c(v)\} = 1$. If not, the costs are scaled by dividing them by $\max_{v \in V(T)} \{c(v)\}$. Note that this operation does not affect the optimality of a strategy or the quality of the approximation.

Observation 3.2.3.1. *Let T be a tree such that $|V(T)| > 1$ and $c : V \rightarrow \mathbb{R}^+$ be a normalized cost function. Then, $1 \leq \text{OPT}(T) \leq \lfloor \log n \rfloor + 1$.*

Proof. The first inequality is due to the fact that there exists $v \in V(T)$, such that $c(v) = 1$ and for any decision tree D we have $v \in Q_D(T, v)$. The second inequality is due to the fact that we can always locate the target using $\lfloor \log n \rfloor + 1$ queries [OP06]. $\square$

Rounding

We will use the following rounding scheme which will allow us to discretize the space of possible solutions in order to process it efficiently. Let $p \in \mathbb{N}$, and $k = a/pn$ for some $a \in \mathbb{N}$. Define:

$$c'(v) = \begin{cases} \lceil c(v) \rceil_k, & \text{if } c(v) > pk, \text{ in which case the vertex will be called } \textit{heavy}, \\ \lceil c(v) \rceil_{\frac{1}{pn}}, & \text{otherwise, in which case the vertex will be called } \textit{light}. \end{cases}$$

Lemma 3.2.3.2.

$$\text{OPT}(T, c') \leq \left(1 + \frac{2}{p}\right) \cdot \text{OPT}(T, c).$$

Proof. Let D^* be an optimal strategy for (T, c) . By definition, we have that for every vertex $v \in V(T)$, $c'(v) \leq \left(1 + \frac{1}{p}\right) \cdot c(v) + \frac{n}{pn}$ and therefore:

$$\begin{aligned} \text{OPT}(T, c') &\leq C_T(D^*, c') = \max_{v \in V(T)} \left\{ \sum_{q \in Q_{D^*}(T, v)} c'(q) \right\} \\ &\leq \max_{v \in V(T)} \left\{ \sum_{q \in Q_{D^*}(T, v)} \left(\left(1 + \frac{1}{p}\right) \cdot c(v) + \frac{1}{pn} \right) \right\} \\ &\leq \frac{1}{p} + \left(1 + \frac{1}{p}\right) \cdot \max_{v \in V(T)} \left\{ \sum_{q \in Q_{D^*}(T, v)} c(v) \right\} \leq \left(1 + \frac{2}{p}\right) \text{OPT}(T, c) \end{aligned}$$

where in the third inequality we used the fact that for every $v \in V(T)$, $|Q_{D^*}(T, v)| \leq n$ and in the last inequality we used Observation 3.2.3.1. $\square$

While calculating the decision tree, we will divide the time into boxes of duration k , which will be further subdivided into a identical slots of length $\frac{1}{pn}$. Let t_q denote the start of some query in a decision tree D . Note that the numbers t_v provide a complete information about any decision tree, and are an equivalent representation of any strategy. We will assume that for any heavy vertex $v \in V(T)$, t_v is an integer multiple of c and for any light vertex $v \in V(T)$, t_v is an integer multiple of $\frac{1}{pn}$.¹ We have the following lemma:

Lemma 3.2.3.3. *There exists a decision tree D for (T, c') , such that $C_T(D, c') \leq \left(1 + \frac{3}{p}\right) \cdot \text{OPT}(T, c')$ and for every vertex $v \in V(T)$ we have:*

1. if $c(v) > pk$, then $t_v/k \in \mathbb{N}$ (every heavy query is aligned to a multiple of c),
2. if $c(v) \leq pk$, then $t_v/pn \in \mathbb{N}$ (every light query is aligned to a multiple of $\frac{1}{pn}$).

Proof. Let D^* be any optimal decision tree for (T, c') . For any $v \in V(T)$, let t_v^* be the start of query to v in D^* and let $t'_v = \left(1 + \frac{2}{p}\right) t_v^*$, yielding a new decision tree D' . Since in D' , the ordering of vertices is exactly the same as in D^* , for any two consecutive queries v, u in D' we have:

$$t'_u - t'_v = \left(1 + \frac{2}{p}\right) \cdot (t_u - t_v) \geq \left(1 + \frac{2}{p}\right) \cdot c(v).$$

We now construct D as follows: If $v \in V(T)$ is heavy, we assign $t_v = \lceil t'_v \rceil_k$ and $t_v = t'_v$ otherwise. For any two consecutive queries v, u in D , such that v is heavy we have:

$$t_u - \lceil t'_v \rceil_k > t_u - t_v - k \geq \left(1 + \frac{2}{p}\right) \cdot c(v) - k > w(v) + k > c'(v).$$

So we conclude that no two queries overlap. To obtain the second part of the claim, we round up the starting time of each query to a light vertex in D to an integer multiple of $\frac{1}{pn}$. We have:

$$\begin{aligned} C_T(D, c') &\leq \max_{v \in V(T)} \left\{ \sum_{q \in Q_{D^*}(T, v)} \left(\left(1 + \frac{2}{p}\right) \cdot c'(v) + \frac{1}{pn} \right) \right\} \\ &\leq \frac{1}{p} + \left(1 + \frac{2}{p}\right) \cdot \max_{v \in V(T)} \left\{ \sum_{q \in Q_{D^*}(T, v)} c'(v) \right\} \leq \left(1 + \frac{3}{p}\right) \text{OPT}(T, c') \end{aligned}$$

¹Note that by doing so, we allow decision trees to contain idle time intervals, in which no queries are scheduled. However, if this occurs, after obtaining such decision tree, we simply delete the idle times, which results in a valid decision tree

where in the third inequality we used the fact that for every $v \in V(T)$, $|Q_D(T, v)| \leq n$ and in the last inequality we used Observation 3.2.3.1. $\square$

We will call a decision tree fulfilling above conditions *aligned*. In subsequent considerations, we will limit ourselves to finding only aligned decision trees, whose properties will allow us to devise an efficient dynamic programming procedure finding an optimal, aligned decision tree.

Up and down responses, heavy module contraction and aligned cost

Since our decision tree is rooted, we can reasonably talk about up and down responses to a query. An *up* response to a query to v in T occurs when the connected component $\in T - v$, which is the reply, happens to contain $r(T)$. If this is not the case, then such response is called a *down* response. As it will turn out, a repeating occurrence of light queries with down responses will become problematic for our algorithm. To account for this issue we will use the following notions:

We define a new measure of cost for aligned decision trees called the *aligned cost*. Let D be any aligned strategy for (T, c') . For any vertex $v \in V(T)$ and query $q \in Q_D(T, v)$ the contribution $\kappa_{T,c,k}(q, v)$ of u is defined as:

$$\kappa_{T,c,k}(q, v) = \begin{cases} \lfloor t_v + c(q) \rfloor_k - t_v, & \text{if } c(v) \leq pk \text{ and the response to query } q \text{ in } T, \text{ towards } v \text{ is down,} \\ c(q), & \text{otherwise.} \end{cases}$$

Then, the *aligned cost* of D is defined as:

$$C'_T(D, c', k) = \max_{v \in V(T)} \left\{ \sum_{q \in Q_D(T, v)} \kappa_{T,c',k}(q, v) \right\}.$$

Let $\text{OPT}'(T, c', k)$ denote the optimal aligned cost among all aligned decision trees for (T, c', k) . Notice that in the above cost, we lose up to k units of time per each light query with a down response. Since of course, the amount of such queries may be of order $O(n)$, the difference between $C'_T(D, c', k)$ and $C_T(D, c', k)$ may grow almost arbitrarily large. However, we will make sure that this does not happen to often, which will give us the desired bound on the cost of the solution. We have the following simple observation:

Observation 3.2.3.4. *Let T' be a subtree of T . Then, $\text{OPT}'(T') \leq \text{OPT}'(T)$.*

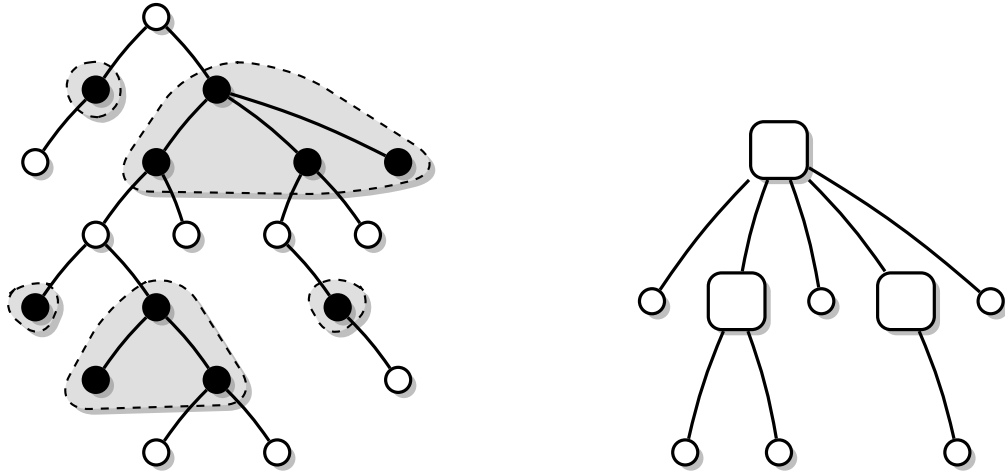


Figure 3.8: Example of contracting 5 heavy modules. Black vertices represent heavy vertices, white vertices represent light vertices and square vertices represent vertices which were a parent of at least one heavy module before contraction.

We define a *heavy module* as $H \subseteq V(T)$ such that: $T[H]$ is connected, every $v \in H$ is heavy, i. e., $c(v) \geq pk$ and H is maximal - no vertex can be added to it without violating one of its properties. A *contraction* of a heavy module H is an operation which consists of deleting all of the vertices in H from T and connecting every vertex u which was a child of some vertex in H to the parent of $r(T \setminus H)$ if it exists. For example see Figure 3.8.

The main procedure

We will use the following propositions:

Proposition 3.2.3.5. *Let T be a tree, c' an aligned cost function, $p \in \mathbb{N}$, k the box size and $d \in \mathbb{N}$ be the depth. There exists a **DPTimelinesCosts** procedure, which (if it exists) calculates an aligned decision tree D for (T, c') of cost at most $C'_T(D, c', k) \leq kd$, running in $(pn)^{O(d)}$ time.*

Proposition 3.2.3.6. *Let T be a tree, c' be an aligned cost function, $p \in \mathbb{N}$, $k \in \mathbb{R}_{>0}$ be the box size, D_A be a decision tree for T and F_C be forest of decision trees for T with all heavy modules contracted. There exists a polynomial time **MergeDTs** procedure which returns a decision tree of cost at most:*

$$C_T(D, c', k) \leq C'_T(D_A, c', k) + 2pk \cdot C_T(F_C, 1).$$

The proofs will be provided in the further sections. We will now prove the Proposition 3.2.2.1.

Algorithm 4: The QPTAS for $T||V, c, w||C$.

```

Procedure QPTAS( $T, c, \epsilon$ ):
   $p \leftarrow \lceil 59/\epsilon \rceil$ .
   $d \leftarrow p^2 \cdot (\lfloor \log(n) \rfloor + 1)$ .
   $k \leftarrow 0$ .
  while true do
     $k \leftarrow k + \frac{1}{pn}$ .
    foreach  $v \in V(T)$  do
      if  $c(v) > pk$  then
         $c'(v) \leftarrow \lceil c(v) \rceil_k$ .
      else
         $c'(v) \leftarrow \lceil c(v) \rceil_{\frac{1}{pn}}$ .
     $D_A \leftarrow \text{DPTimelinesCosts}(T, c', p, k, d)$ 
    if  $D_A \neq \emptyset$  then
       $T_C \leftarrow T$  with all heavy modules contracted.
       $D_C \leftarrow \text{RankingBasedDT}(T_C)$ .
       $D \leftarrow \text{MergeDTs}(T, D_A, D_C)$ .
    return  $D$ .

```

Algorithm 4 starts by picking $p = \lceil \frac{59}{\epsilon} \rceil$, $d = p^2 \cdot (\lfloor \log n \rfloor + 1)$ and $k = 0$ and. At each iteration of the repeat loop, the algorithm picks k to be the next integer multiple of $\frac{1}{pn}$ and performs the rounding operation. After that, the Proposition 3.2.3.5 is applied for (T, c', p, k, d) . If the returned decision tree $D_A \neq \emptyset$, then a new tree T_C is constructed by contracting all heavy modules of T . By calling the **RankingBasedDT** for T_C , the algorithm builds a second decision tree D_C , and then merges it with D_A by applying Proposition 3.2.3.6. Then, the algorithm returns the resulting decision D .

Let k' be the value of k for which D was built. Let $k'' = k - \frac{1}{pn}$ and c'' be the values of k and c' of the previous iteration of the while loop. Since we know that for k'' and c'' we had $D_A = \emptyset$, by Proposition 3.2.3.5 we have that $k''d \leq \text{OPT}'(T, c'')$. Hence, $k' \leq \frac{\text{OPT}'(T, c'', k')}{d} + \frac{1}{pn} \leq \frac{2 \cdot \text{OPT}'(T, c'', k'')}{p^2 \cdot (\lfloor \log n \rfloor + 1)}$, so

we have that:

$$\begin{aligned}
C_T(D, c') &\leq \text{OPT}'(T, c', k') + 2pk' \cdot (\lfloor \log n \rfloor + 1) \\
&\leq \text{OPT}'(T, c'', k'') + 2p \cdot (\lfloor \log n \rfloor + 1) \cdot \frac{2 \cdot \text{OPT}'(T, c'', k'')}{p^2 \cdot (\lfloor \log n \rfloor + 1)} \\
&\leq \left(1 + \frac{4}{p}\right) \cdot \text{OPT}'(T, c'', k'') \leq \left(1 + \frac{2}{p}\right) \cdot \left(1 + \frac{3}{p}\right) \cdot \left(1 + \frac{4}{p}\right) \cdot \text{OPT}(T, c) \\
&\leq \left(1 + \frac{59}{p}\right) \cdot \text{OPT}(T, c) = \left(1 + \frac{59}{\lceil \frac{59}{\epsilon} \rceil}\right) \cdot \text{OPT}(T, c) \leq (1 + \epsilon) \cdot \text{OPT}(T, c)
\end{aligned}$$

where the first inequality is by Proposition 3.2.3.6, the second inequality is by the fact that $\text{OPT}(T, c', k') \leq \text{OPT}(T, c', k'')$ and by applying Corollary 3.1.0.1 and the fourth inequality is by the fact that $\text{OPT}(T, c', k'') \leq \text{OPT}(T, c)$, Lemma 3.2.3.2 and Lemma 3.2.3.3.

We can assume that $c = \text{poly}(n)$, since beyond that the problem can be solved to optimality in $O(2^n n)$ time. Therefore the running time of the procedure is bounded by:

$$n^{O(d)} = n^{O(p^2 \log n)} = n^{O(\log n / \epsilon^2)}.$$

Proof of Proposition 3.2.3.6

Algorithm 5: The MergeDTs procedure.

Procedure MergeDTs(T, D_A, F_C):

- if** F_C is connected **then**
 - $r \leftarrow r(F_C)$.
- else**
 - $r \leftarrow r(D_A)$.
- $D \leftarrow (\{r\}, \emptyset)$. **foreach** $T' \in T - r$ **do**
 - $D' \leftarrow \text{MergeDTs}(T, D_A|T', F_C|T')$.
 - Hang D' below r in D .
- return** D .

Algorithm 5 takes as arguments a tree T , a decision tree D_A and a forest of decision trees F_C for T with heavy groups contracted. Since F_C may not be a valid partial decision tree for T , it may happen that it is disconnected. This is because, after performing a light query to v according to F_C , if the response was down and v has heavy child modules, the children of v in T_C , may not be separated yet. Based on this, we derive two cases for our procedure:

1. F_C is connected. In such case we pick $r = r(F_C)$.
2. F_C is disconnected. In such case we pick $r = r(D_A)$.

We set $D = (\{r\}, \emptyset)$ and then, for each $T' \in T - r$, we build a decision tree D' for T' recursively by calling MergeDTs with arguments $(T, D_A|T', F_C|T')$ and hang D' below r in D .

Lemma 3.2.3.7. *Let D be the decision tree returned by the MergeDTs procedure. Then, $C_T(D, c', k) \leq C'_T(D_A, c', k) + 2pk \cdot C_T(F_C, 1)$.*

Proof. Let $x \in V(T)$. We have two cases:

1. r is heavy or r is light and the response is up. In this case the cost of query to r is incorporated into $C'_T(D_A, c', k)$.
2. r is light and the response is down. Let r_1 and r_2 be two consecutive light queries in $Q_D(T, x)$ with a down response, and let $T'' \in T - \{r_1, r_2\}$, such that $x \in T''$. We will show that $C_T(F_C|T, 1) \leq C_T(F_C, 1) - 1$, so the additional cost of such queries is at most $2pk \cdot C_T(F_C, 1)$.

- (a) F_C is connected. In such case $r_1 = r(F_C)$ so after query to r_1 , by definition of the decision tree $C_T(F_C|T', 1) \leq C_T(F_C, 1) - 1$.
- (b) F_C is disconnected. Let T' be any down response to query to r . If $r_1 = r(F_C|T')$, then $C_T(F_C|T', 1) \leq C_T(F_C, 1) - 1$. Otherwise, we argue that $F_C|T'$ is connected. Assume contrary. Firstly, we observe that $F_C|T_{r_1}$ is connected. This is because, r_1 is light, so the tree $T_{r_1, C}$ denoting T_{r_1} with all heavy modules contracted is connected. By assumption, there are at least two connected components $D_1, D_2, \dots, D_j$ of $F_C|T'$, with roots $d_1, d_2, \dots, d_j$ respectively. Since every other query $q \in V(F_C|T')$ is a descendant of some $d_1, \dots, d_j$, the only query which could be the parent of $d_1, \dots, d_j$ in $F_C|T'$ is r_1 , which is a contradiction. Therefore, the next query r_2 of D in T' is $r(F_C|T')$. We get that by definition of the decision tree $C_T(F_C|T'') \leq C_T(F_C, 1) - 1$.

□

Dynamic programming procedure for a fixed box size

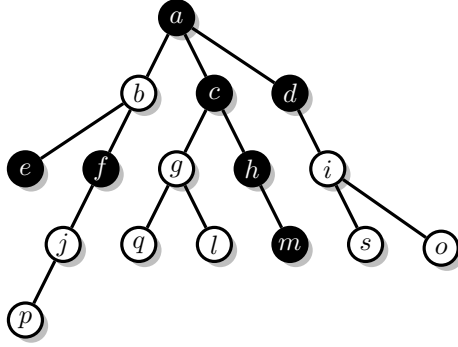


Figure 3.9

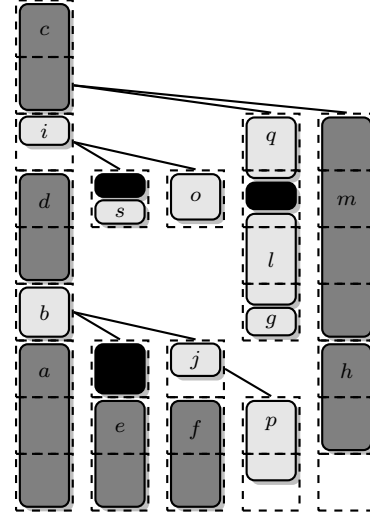


Figure 3.10

Figure 3.11: Structure of a boxed decision tree. Figure 3.9 shows example input tree T , black vertices are heavy and white vertices are white. Figure 3.10 shows example boxed decision tree D for T , dark gray queries are heavy, light gray queries are light, and black spaces represent additional load of boxes.

To devise our dynamic programming procedure, we will need the following generalization of the aligned decision tree. A *boxed decision tree* $D = (V(D), E(D), u, l)$ for the tree T is a tuple, in which $V(D)$ are the nodes of the decision trees, which we will call *boxes*, $E(D)$ are edges of the decision tree, $u : V(T) \times V(D) \rightarrow \{0, 1/pn, 2/pn, \dots, k\}$ is the *usage* function and $l : V(D) \rightarrow \{0, 1/pn, 2/pn, \dots, k\}$ is the *load* function. Based on u , for every $b \in V(D)$ we will also define the *query assignment* as following: $Q(b) = \{v \in V(T) \mid u(v, b) > 0\}$, these are the vertices of T , such that queries to them overlap with the box b .

The boxed decision tree is defined analogously as ordinary decision tree, however, to each query in $b \in V(D)$ instead of one vertex of $V(T)$, we assign an arbitrary subset of $V(T)$, via the query assignment function Q . Let $b \in V(D)$, and let T' be a candidate subtree of T right before any vertex in $Q(b)$ was queried. A box p is a left child of b , if either b corresponds to a subtree $T'' \in T - Q(b)$ such that $r(T') \in V(T'')$ or $Q(b) \cup Q(p) \neq \emptyset$. If otherwise, then p is a right child of b .

We demand that for every $v \in V(T)$, all boxes $b \in V(D)$, such that $v \in Q(b)$ form a connected path in D , in which every child is a left child, for any interior box b of this path, $l(b) = 0$ and $u(v, b) = k$. We will also require that: for every $q \in V(D)$, $l(b) + \sum_{v \in V(T)} u(v, b) \leq k$ and for every $v \in V(T)$ either $\sum_{b \in V(D)} u(v, b) = 0$, in which case such query is called *unassigned* or $\sum_{b \in V(D)} u(v, b) =$

$c(v)$ and the query is called *assigned*. Since we want the boxed decision tree to also be aligned, we will demand that if for any vertex $v \in V(T)$, $c(v) > pk$, then for every $b \in V(D)$, either $u(v, b) = 0$ or $u(v, b) = k$.

We now define how to search in T using a boxed decision tree D . Firstly, the query process stops for time $l(u)$. If $Q(r(D)) \cup V(T) = \emptyset$, then we recurse on the appropriate child of $r(T)$ (which corresponds to our current candidate subtree). Otherwise, we pick the least costly vertex v in $Q(r(D))$, we remove v from $Q(q)$ for any $q \in V(D)$ and we recurse on the last box which contained a query to v (note that this can also be $r(v)$). This is done in order to ensure that the start of the query t_v of each vertex v happens in the box containing it. The aligned cost of seaching using a boxed decision tree is defined analogously as the aligned cost of searching using ordinary decision tree. Note that any boxed decision tree, can also be transformed to an equivalent aligned decision tree with the same aligned cost of searching. Conversely, any aligned decision tree can be transformed into a boxed decision tree, by subdividing the queries into boxes according to the their starting points. Note that since we do not count the cost of light down responses, by sorting all queries starting in a given box according to their cost, we cannot increase the aligned cost.

We define a *boxline* $B\langle(b_1, \tau_1), (b_2, \tau_2), \dots, (b_d, \tau_d)\rangle$ to be a sequence of pairs, each consisting of box and additional boolean flag, such that for every box b in B , $Q(b) = \emptyset$. We will build our decision trees around boxlines. Define the *left box-path* $B_D = \langle q_1, f_1, (q_2, f_2), \dots, (q_h, f_h) \rangle$ of D as a sequence of pairs. Each such pair consists of the overall loads of consecutive boxes obtained by traversing D starting from root $r(D)$, and stepping to the left child until there is none (for each such box b with index j , $q_j = l(b) + \sum_{v \in Q(b)} q(v, b)$), and boolean values denoting whether there exists a query transcending the current box unto the next one (f_h is always false, however we include it for convenience). We will say that a decision tree D with a left box-path B_D is *box-compatible* with a boxline B , such that $h \leq d$, if for every integer $1 \leq j \leq h$, $l(q_j) \geq l(b_j)$ and if τ_j implies f_j . To build a decision D tree using B , we simply create a boxed decision tree D consisting of path of vertices $\langle q_1, \dots, q_h \rangle$, such that $l(q_j) = l(b_j)$ and $Q(q_j) = \emptyset$.

We will also use the following operations:

- Putting a query to vertex v at s -th slot of a box b :

1. $\sigma(v) \leftarrow c(v)$;
2. **while** $\sigma(v) > 0$:
 - (a) $u(v, b) \leftarrow \min\{k - s/pn, \sigma(v)\}$.
 - (b) $\sigma(v) \leftarrow \sigma(v) - u(v, b)$.
 - (c) $b \leftarrow$ left child of b .

If such operation violates the definition of D or query to v transcends any box b_j , such that τ_j we mark D as *conflicted*.

- Building a decision tree D based on boxline B :

1. $D \leftarrow \emptyset$.
2. **for** $1 \leq j \leq |B|$:
 - (a) Create box q_j in D .
 - (b) $l(j) \leftarrow b_j$.
 - (c) $Q(q_j) \leftarrow \emptyset$.
 - (d) **if** $j > 1$ **then**: hang q_j as the left child of q_{j-1} .
3. **return** D .

- Rotating a decision tree D around vertex $v \in V(D)$:

1. $q_h \leftarrow$ the box containing the end of query to v .
2. Sort queries starting in $Q(q_h)$ according to c' .
3. Create box q .
4. Move queries from $Q(q_h)$ to $Q(q)$, so that all queries after v are in $Q(q)$.
5. Hang q'_h as a right child of q_h .

Algorithm 6: The dynamic programming procedure finding $\text{OPT}'(T_{v,i}, B)$ (k, p, c, n and d are global parameters).

```

Procedure DPTimelinesCosts( $T_{v,i}, B$ ):
  if  $i = 0$  then
    for  $1 \leq b \leq d$  and  $0 \leq s \leq (k/pn \text{ if } c(v) > pk \text{ else } 0)$  do
       $D \leftarrow$  a decision tree based on  $B$ .
      Put query to  $v$  at the  $s$ -th slot of  $q_b$ .
      if  $D$  is not conflicted. then
        if  $C'_{T_{v,i}}(D, c', k) \leq dk$  then: return  $D$ , else: return  $\emptyset$ .
    return  $\emptyset$ .
   $\mathcal{D} \leftarrow \emptyset$ .
  if  $i = 1$  then
    for  $1 \leq b \leq d$  and  $0 \leq s \leq (k/pn \text{ if } c(v) > pk \text{ else } 0)$  do
       $D \leftarrow$  a decision tree based on  $B$ .
      Put query to  $v$  at the  $s$ -th slot of  $q_b$ .
      if  $D$  is not conflicted and  $C'_{T_{v,i}}(D, c', k) \leq dk$  then
        if  $C'_{T_{v,i}}(D, c, k) \leq dk$  then
           $B' \leftarrow$  left box-path of  $D$ .
           $h \leftarrow$  index of the last box  $q_h$  occupied by the query to  $v$ .
          for  $h < j \leq d$  do
             $b'_j \leftarrow 0$ .
             $t'_j \leftarrow \text{false}$ .
           $D' \leftarrow \text{DPTimelinesCosts}(T_{c_1}, c', B')$ .
          Put query to  $v$  at the  $s$ -th slot of  $q'_b$ .
          Rotate  $D'$  around  $v$ .
           $\mathcal{D} \leftarrow \mathcal{D} \cup \{D \text{ and } D' \text{ with their left paths aligned}\}$ .
        else
          foreach bipartition  $(B_1, B_2)$  of  $B$  do
             $D_1 \leftarrow \text{DPTimelinesCosts}(T_{v,i-1}, c', B_1)$ .
             $h \leftarrow$  index the last box  $q_{1,h}$  occupied by the query to  $v$ .
            for  $h \leq j \leq d$  do
               $b_{2,j} \leftarrow 0$ .
               $t_{2,j} \leftarrow \text{false}$ 
             $D_2 \leftarrow \text{DPTimelinesCosts}(T_{c_i}, c', B_2)$ .
            Put query to  $v$  at the  $s$ -th slot of  $q_{2,b}$ .
            Rotate  $D_2$  around  $v$ .
             $\mathcal{D} \leftarrow \mathcal{D} \cup \{D_1 \text{ and } D_2 \text{ with their left paths aligned}\}$ .
  return  $\arg \min_{D \in \mathcal{D}} \{C'_{T_{v,i}}(D, c, k)\}$ .

```

6. Rehang left child of q_h as the left child of q .
- Bipartitioning of B . A bipartition of a boxline B consists of a pair of boxlines (B_1, B_2) such that:
 - $|B| = |B_1| = |B_2|$.
 - $l(b_{1,j}) + l(b_{2,j}) - k = l(b_j)$.
 - $(\tau_{1,j} \wedge \tau_{2,j} \iff \tau_j)$.
 - $(\tau_{1,j} \vee \tau_{2,j})$.
- Aligning D_1 and D_2 by their left paths to create a new decision tree D :
 1. $D \leftarrow \emptyset$.
 2. **for** $1 \leq j \leq |B|$:
 - (a) Create box q_j in D .
 - (b) $l(j) \leftarrow l(q_{1,j}) + l(q_{2,j}) - k$.
 - (c) **for** $v \in V(T)$: $u(v, q_j) \leftarrow \max\{u(v, q_{1,j}), u(v, q_{2,j})\}$.
 - (d) Hang all right children of $q_{1,j}$ and $q_{2,j}$ below q_j .
 - (e) **if** $j > 1$ **then**: hang q_j as the left child of q_{j-1} .
 3. **return** D .

We now introduce the subproblems which our dynamic programming solves. A problem $\text{OPT}'(T_{v,i}, B)$ consists of finding an optimal boxed decision tree for the tree $T_{v,i}$, which is box-compatible with B . If no B is given, we assume that $B = \langle (b_1, \tau_1), \dots, (b_d, \tau_d) \rangle$, where for every $1 \leq j \leq d$, $b_j = 0$ and τ_j is false. The algorithm computes the solutions in a bottom-up, left-to-right manner. If at any point there is no way to create an extended decision tree with given parameters we simply declare such instance *unfeasible*. We will now show how to compute $\text{OPT}(T_{v,i}, B)$ efficiently. The Algorithm 6 consists of 3 cases:

1. $T_{v,0}$. We start by building a decision tree D based on B . Then, we greedily pick the box with smallest index $1 \leq b \leq d$, such that there exists slot s for which it is possible to place query to v at the s -th slot of q_b , without introducing conflicts. If there is no such index, we declare the subproblem unfeasible. In other case, the solution obtained by taking timeline P and setting $p_k = v$.
2. $T_{v,1}$. Assume that we have already solved all the subproblems of T_u . We again start by building a decision tree D based on B . Then, for any $1 \leq b \leq d$, such that there exists slot s for which it is possible to place query to v at the s -th slot of q_b , without introducing conflicts we put such query. Let h be the index of last box q_h occupied by v . We set the load of each next consecutive box in B' to be 0 and we set all further boolean flags to false. We then retrieve the optimal decision tree D' for (T_{c_1}, B') , put query to v in D' at the s -th slot of box q'_b , rotate D' around v and align D' with D . Then, among all of such obtained decision trees we pick the one which minimizes the aligned cost.
3. $T_{v,i}$ for $i > 1$, we assume that we have already solved all the subproblems of $T_{v,i-1}$ and T_{c_i} . We consider all bipartitions (B_1, B_2) of B . We retrieve the optimal decision tree D_1 for $(T_{v,i-1}, B_1)$. Let h be the index of last box q_h occupied by v in D_1 . We set the load of each next consecutive box in B_2 to be 0 and we set all further boolean flags to false. Then, we retrieve the optimal decision tree D_2 for (T_{c_i}, B_2) , put query to v in D_2 at the s -th slot of box q'_b , rotate D_2 around v and align D_1 with D_2 . Then, among all of such obtained decision trees we pick the one which minimizes the aligned cost.

Lemma 3.2.3.8. *The running time of the `DPTimelinesCosts` procedure is bounded by $(pn)^{O(d)}$.*

Proof. The number of subtrees $T_{v,i}$ is $O(n)$. Each box in the boxline can have one of at most $O(n)$ values of load and at most 2 values of the boolean flag. Since each boxline is of length d , number of different boxlines for each $T_{v,i}$ is bounded by $(pn)^{O(d)}$. Additionally, the running time of each

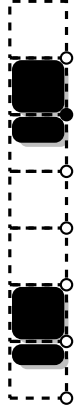


Figure 3.12

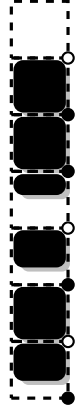


Figure 3.13



Figure 3.14



Figure 3.15

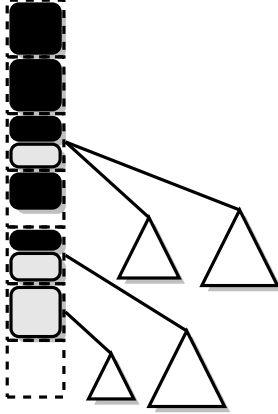


Figure 3.16

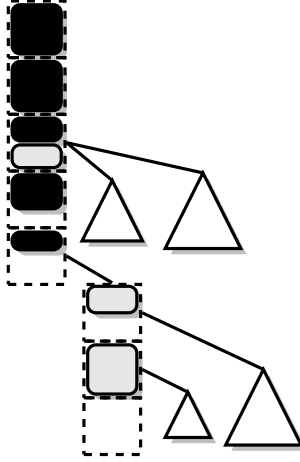


Figure 3.17

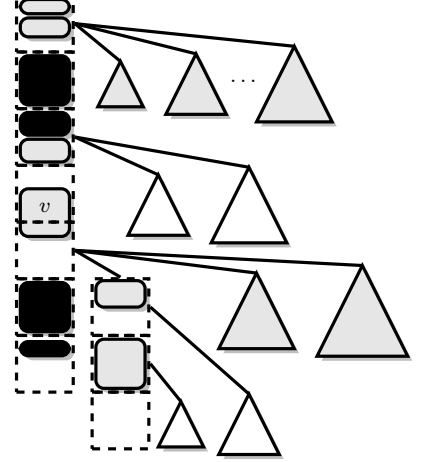


Figure 3.18

Figure 3.19: Basic steps of the case when $i > 1$. The black regions represent the load of a box. Fig. 3.12: An example of a boxline B . Fig. 3.13: boxlines B_1 and B_2 induced by bipartitioning B . Fig 3.14: decision tree D_1 box-compatible with timeline B_1 . Fig 3.15: timeline B_2 after deleting loads of nodes with indices below k . Fig 3.16: a decision tree D_2 compatible with B_2 . Fig 3.17: D_2 with left child of q_k rehanged to right. Fig 3.18: D_1 and D_2 aligned by their left box-paths.

subproblem is bounded by $(pn)^{O(d)}$, since there at most this many bipartitions of available load and boolean flags among boxes. Therefore, the overall running time is bounded by $(pn)^{O(d)}$. $\square$

3.2.4 An $O(\log \log n)$ -approximation algorithm parametrized by the k -up-modularity of the cost function

k -up-modularity

The main algorithmic difficulty in dealing with the problem arises when the values of the cost function vary drastically. We would like to measure this "irregularity" in a quantifiable way. To do so, we introduce the notion of k -up-modularity.

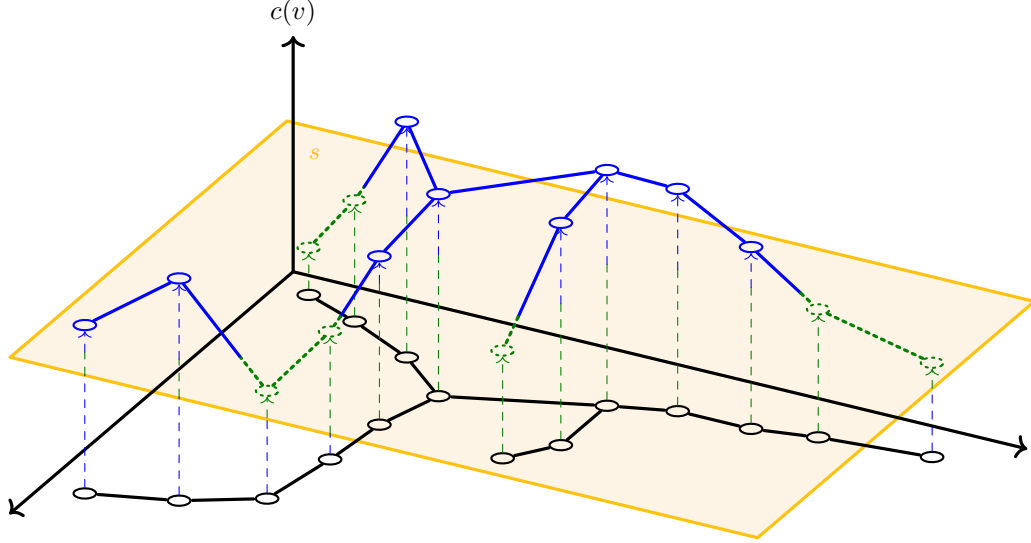


Figure 3.20: A visual depiction of a tree T with a 3-up-modular cost function c . Each vertex of a tree is mapped onto some value of c . The yellow plane represents some threshold value $t \in \mathbb{R}_{\geq 0}$ (in this particular example $k(T, c, t) = 2$). The (two) blue subtrees represent members of $\mathcal{H}_{T,c}(t)$.

Let $t \in \mathbb{R}_{\geq 0}$. We define a *heavy module* with respect to t as $H \subseteq V(T)$ such that, $T[H]$ is connected, for every $v \in H$, $c(v) > t$, and H is maximal - no vertex can be added to it without violating one of its properties. We then define the *heavy module set* with respect to t in (T, c) as:

$$\mathcal{H}_{T,c}(t) = \{H \subseteq V(T) \mid H \text{ is a heavy module w.r.t. } t\},$$

Let $k(T, c, t) = |\mathcal{H}_{T,c}(t)|$ be the size of the heavy module set, and finally let:

$$k(T, c) = \max_{t \in \mathbb{R}_{\geq 0}} \{k(T, c, t)\}$$

We say that a function c is k -up-modular in T when $k \geq k(T, c)$. Whenever clear from the context, we will use $k(T, c)$, $k(T)$, or k to denote the lowest value such that c is k -up-modular in T . To illustrate the notion of k -up-modularity, see Figure 3.20.

The concept of k -up-modularity is a direct generalization of the notion of up-monotonicity of the cost function introduced in [DW22] (as monotonicity) and in [DW24] (as up-monotonicity). Let $z = \arg \max_{v \in V(T)} \{c(v)\}$. A function c is *up-monotonic* in T if for every $v, u \in V(T)$, whenever v lies on the path between z and u , we have $c(v) \geq c(u)$.

It is easy to see that 1-up-modularity is equivalent to up-monotonicity. Observe that if c is up-monotonic in T , then for every $t \in \mathbb{R}_{\geq 0}$, $T[V(T) - \{v \in V(T) \mid c(v) \leq t\}]$ is connected and forms a single heavy module. Conversely, let $r = \arg \max_{v \in V(T)} \{c(v)\}$ and u be any other vertex. If c is 1-up-modular in T , then there is no vertex v on the path between r and u such that $c(v) < c(u)$.

Otherwise, for any $t \in (c(v), c(u))$, v does not belong to any heavy module, but u and r do. Since v lies between them, $|\mathcal{H}_{T,c}(t)| > 1$, a contradiction.

The parametrized $O(\log \log n)$ -approx. solution

Cost levels

The main idea of the algorithm is to partition vertices into intervals called *cost levels* and process them in a top-down manner. At each level of the recursion, the algorithm schedules all necessary queries to vertices belonging to the given cost level. The rest of the decision tree is then built recursively. We consider the following intervals²:

1. Firstly, an interval $(0, 1/\log n]$.
2. Then, each subsequent interval $\mathcal{I}' = (a', b']$ starts at the left endpoint of the previous interval $\mathcal{I} = (a, b]$, that is, $a' = b$, and ends with $b' = \min\{2b, 1\}$.

This results in the following sequence of intervals, which partitions the interval $(0, 1]$:

$$(0, 1/\log n], (1/\log n, 2/\log n], (2/\log n, 4/\log n], \dots, (2^{\lceil \log \log n \rceil - 1}/\log n, 1].$$

We will ensure that when we call our procedure with parameters $(T, c, (2^{\lceil \log \log n \rceil - 1}/\log n, 1])$, the returned decision tree will be a valid decision tree for T .

We are now ready to introduce the notions of heavy and light vertices (and queries to them). We say that a vertex v (or a query to it) is *heavy* with respect to the interval $\mathcal{I} = (a, b]$ when $c(v) > a$. Otherwise, i.e., if $c(v) \leq a$, the vertex (and the query to it) is *light* with respect to $\mathcal{I}$. Note that each heavy vertex belongs to some heavy module. Whenever clear from the context, we will omit the phrase "with respect to" and simply call the vertices and queries heavy and light.

The main recursive procedure

We are ready to present the main recursive procedure. To avoid ambiguity, let $\mathcal{T}$ be the subtree of T processed at some level of the recursion. Alongside $\mathcal{T}$ and a cost function c , the algorithm takes as input an interval $(a, b]$, such that for every $v \in V(\mathcal{T})$, $c(v) \leq b$ and $2a \geq b$. The basic steps of Algorithm 7 are as follows:

1. If every vertex is heavy, return a decision tree built by calling the **RankingBasedDT** procedure for $\mathcal{T}$.
2. Otherwise, find a set $\mathcal{Z}$, such that each connected component of $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$ contains at most one heavy module.
3. Create an auxiliary tree $T_{\mathcal{Z}}$ using the vertices of $\mathcal{Z}$ and create a new decision tree $D_{\mathcal{Z}}$ for $T_{\mathcal{Z}}$, using the QPTAS from [Der+17].
4. For each $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$, build a decision tree D_H , by calling the **RankingBasedDT** procedure for $\mathcal{T}' \setminus H$. Then, hang D_H below the last query to $v \in N_{\mathcal{T}}(\mathcal{T}')$ in $D_{\mathcal{Z}}$.
5. For each $L \in \mathcal{T}' - H$, build a decision tree recursively. Then, hang D_L below the last query to a vertex $v \in N_{\mathcal{T}'}(L)$ in $D_{\mathcal{Z}}$.
6. Return the resulting decision tree D .

Before providing a detailed description and analysis of the above procedure, we first present some basic properties necessary for the subsequent considerations. In particular, we will make use of the following well-known lemma [CLS16]:

Lemma 3.2.4.1. *Let T' be a subtree of T . Then, $\text{OPT}(T') \leq \text{OPT}(T)$.*

Algorithm 7: The main recursive procedure

```

Procedure CreateDecisionTree( $\mathcal{T}, c, (a, b]$ ):
  if  $b \leq 1/\log n$  or for every  $v \in V(\mathcal{T}), c(v) > a$  ;           // Every  $v \in \mathcal{T}$  is heavy
  then
    return RankingBasedDT( $\mathcal{T}$ ) ;                                     // Apply Corollary 3.1.0.1
  else
     $\mathcal{X} \leftarrow \emptyset$ .
    foreach  $H \in \mathcal{H}_{\mathcal{T},c}(a)$  do
      Pick arbitrary  $v \in H$ .
       $\mathcal{X} \leftarrow \mathcal{X} \cup \{v\}$ .
     $\mathcal{Z} \leftarrow \mathcal{Y} \leftarrow \mathcal{X} \cup \{v \in V(\mathcal{T}(\mathcal{X})) \mid \deg_{\mathcal{T}(\mathcal{X})}(v) \geq 3\}$ .
    foreach  $u, v \in \mathcal{Y}$  with  $\mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset$  and  $\mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$  do
       $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{\arg \min_{z \in \mathcal{P}_{\mathcal{T}}(u, v)} \{c(z)\}\}$  ;           // Lightest vertex on path
     $\mathcal{T}_{\mathcal{Z}} \leftarrow (\mathcal{Z}, \{uv \mid \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Z} = \emptyset\})$  ;           // Build auxiliary tree
     $D \leftarrow D_{\mathcal{Z}} \leftarrow \text{QPTAS}(\mathcal{T}_{\mathcal{Z}}, c, \epsilon = 1)$  ;           // Apply Theorem 3.2.2.1
    foreach  $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$  do
       $H \leftarrow$  the unique heavy module in  $\mathcal{T}'$ .
       $D_H \leftarrow \text{RankingBasedDT}(\mathcal{T}'(H))$  ;           // Apply Corollary 3.1.0.1
      Hang  $D_H$  in  $D$  below the last query to  $v \in N_{\mathcal{T}}(\mathcal{T}')$  ;           // By Obs. 3.2.4.5
      foreach  $L \in \mathcal{T}' - H$  do
         $D_L \leftarrow \text{CreateDecisionTree}(L, c, (a/2, a])$ .
        Hang  $D_L$  in  $D$  below the last query to  $v \in N_{\mathcal{T}'}(L)$  ;           // By Obs. 3.2.4.5
    return  $D$ .

```

For the rest of the analysis, fix $\mathcal{H} = \mathcal{H}_{\mathcal{T},c}(a)$ to be the set of heavy modules in $\mathcal{T}$. We have the following observations, which will be useful in the description and analysis of the algorithm:

Observation 3.2.4.2. Let $\mathcal{H}$ be the set of heavy modules in T . Then, $|\mathcal{H}| \leq k(T)$.

Proof. Since $\mathcal{H} = \mathcal{H}_{\mathcal{T},c}(a)$, we have $|\mathcal{H}| = k(\mathcal{T}, c, a) \leq \max_{t \in \mathbb{R}_{\geq 0}} k(\mathcal{T}, c, t) = k(\mathcal{T}, c)$. $\square$

Observation 3.2.4.3. Let T' be a subtree of T . Then, $k(T') \leq k(T)$.

Proof. Fix any $t \in \mathbb{R}_{\geq 0}$ and let $H \in \mathcal{H}_{T,c}(t)$. We show that each such H contributes at most 1 to $k(T', c, t)$. If $H \cap V(T') = \emptyset$, then H contributes 0. Otherwise, $H \cap V(T')$ forms a connected subtree of T' , and thus contributes at most 1. The lemma follows. $\square$

Observation 3.2.4.4. Let T' be a subtree of a tree T and let D' be a decision tree for T' . Then, D' is a partial decision tree for T .

Observation 3.2.4.5. Let T' be a subtree of a tree T and let D be a partial decision tree for T having no queries to vertices of T' , but containing at least one query to the vertices of $N_T(V(T'))$. Let Q denote the set of all such queries to vertices of $N_T(V(T'))$ in D . Then, $D(Q)$ forms a path in D .

Proof. Let q be any query in D . There are two cases:

1. $q \in V(T - V(T') - N_T(V(T')))$. Then, for every $x \in N_T(V(T'))$ being the target, x belongs to the same connected component of $T - q$. Thus, no matter which vertex is the target, the answer is always the same. Therefore, q has at most one child u in D , such that $V(D_u) \cap Q \neq \emptyset$.

²We present the intervals in the ascending order in which a complete solution for each of them is obtained. However, since the procedure is recursive, the order in which the recursive calls are made is reverse.

2. $q \in N_T(V(T'))$. After a query to q , the situation is as in the first case, except when $x = q$. Then, the response is x itself, so no further queries are needed, and again q has at most one child u in D , such that $V(D_u) \cap Q \neq \emptyset$.

Since each $q \in Q$ has at most one child u in D , with $D_u \cup Q \neq \emptyset$, $D \setminus Q$ forms a path and the claim follows. $\square$

Base of the recursion

We begin the description of our algorithm with the recursion base, which occurs whenever $b \leq 1/\log n$ or for every $v \in V(\mathcal{T})$, $c(v) > a$, i.e., every vertex is heavy. In such a situation, a solution is built by disregarding the costs of vertices and constructing a decision tree using the vertex ranking of $\mathcal{T}$.

Lemma 3.2.4.6. *Let D be a decision tree built, by calling `RankingBasedDT` ($\mathcal{T}$) in line 7 of the `CreateDecisionTree` procedure. Then,*

$$C_{\mathcal{T}}(D) \leq 2 \cdot \text{OPT}(\mathcal{T}).$$

Proof. There are two cases:

1. If $b \leq \frac{1}{\log n}$, then:

$$C_{\mathcal{T}}(D) \leq \frac{\lfloor \log n \rfloor + 1}{\log n} \leq \frac{\log n + 1}{\log n} \leq 2 \leq 2 \cdot \text{OPT}(\mathcal{T}) \leq 2 \cdot \text{OPT}(T),$$

where the first inequality is due to Corollary 3.1.0.1, the fourth inequality follows from Observation 3.2.3.1, and the last inequality is due to Observation 3.2.4.1.

2. If for every $v \in V(\mathcal{T})$, we have $c(v) > a$, then, define $c'(v) = a$ for all $v \in V(\mathcal{T})$ (note that any value could be chosen here, since we treat each query as unitary). As $2c'(v) = 2a \geq b \geq c(v)$, we obtain $2 \cdot C_{\mathcal{T}}(D, c') \geq C_{\mathcal{T}}(D, c)$. Additionally, using the fact that $c'(v) \leq c(v)$, we have $\text{OPT}(\mathcal{T}, c') \leq \text{OPT}(\mathcal{T}, c)$. Therefore:

$$C_{\mathcal{T}}(D, c) \leq 2 \cdot C_{\mathcal{T}}(D, c') = 2 \cdot \text{OPT}(\mathcal{T}, c') \leq 2 \cdot \text{OPT}(\mathcal{T}, c) \leq 2 \cdot \text{OPT}(T, c),$$

where the equality is due to Corollary 3.1.0.1 and the last inequality is due to Observation 3.2.4.1. The lemma follows. $\square$

Construction of the Auxiliary Tree

To obtain the solution for the non-base case of our algorithm, we first construct the so-called *auxiliary tree*. To do so, we begin by defining a set $\mathcal{X} \subseteq V(\mathcal{T})$. For every heavy module $H \in \mathcal{H}$, we pick an arbitrary $v \in H$ and add it to $\mathcal{X}$. We also define a set $\mathcal{Y} = \mathcal{X} \cup \left\{ v \in V(\mathcal{T} \setminus \mathcal{X}) \mid \deg_{\mathcal{T} \setminus \mathcal{X}}(v) \geq 3 \right\}$, by extending $\mathcal{X}$ to contain all vertices with degree at least 3 in $\mathcal{T} \setminus \mathcal{X}$. Furthermore, we define a set $\mathcal{Z} \subseteq V(\mathcal{T})$ consisting of the vertices in $\mathcal{Y}$ and, for every $u, v \in \mathcal{Y}$, such that $\mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset$ and $\mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$, we add to $\mathcal{Z}$ the lightest vertex between them, i. e., $v_{u,v} = \arg \min_{z \in \mathcal{P}_{\mathcal{T}}(u,v)} \{c(z)\}$. To see an example of construction of the sets $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$, see Figure 3.21.

We then create the auxiliary tree $\mathcal{T}_{\mathcal{Z}} = (\mathcal{Z}, \{uv \mid \mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Z} = \emptyset\})$ (for an example, see Figure 3.22). Our algorithm starts by building a decision tree $D_{\mathcal{Z}}$ for $\mathcal{T}_{\mathcal{Z}}$, by taking $\epsilon = 1$ and applying the QPTAS from Theorem 3.2.2.1. Observe that, since $D_{\mathcal{Z}}$ is a partial decision tree for $\mathcal{T}$ and corresponding vertices in $\mathcal{T}$ and $\mathcal{T}_{\mathcal{Z}}$ have the same costs, we have that:

Observation 3.2.4.7. $C_{\mathcal{T}_{\mathcal{Z}}}(D_{\mathcal{Z}}) = C_{\mathcal{T}}(D_{\mathcal{Z}})$.

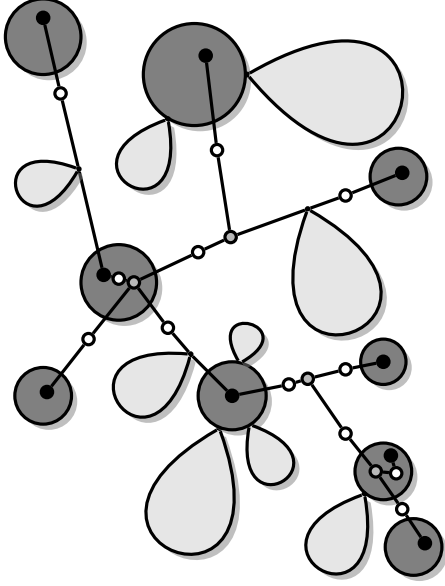


Figure 3.21: Example tree $\mathcal{T}$. Dark grey circles represent heavy modules. Light grey regions represent light subtrees. Black vertices represent $\mathcal{X}$. Gray and black vertices represent $\mathcal{Y}$. White, gray and black vertices represent $\mathcal{Z}$. Lines represent paths of vertices between vertices of $\mathcal{Z}$.

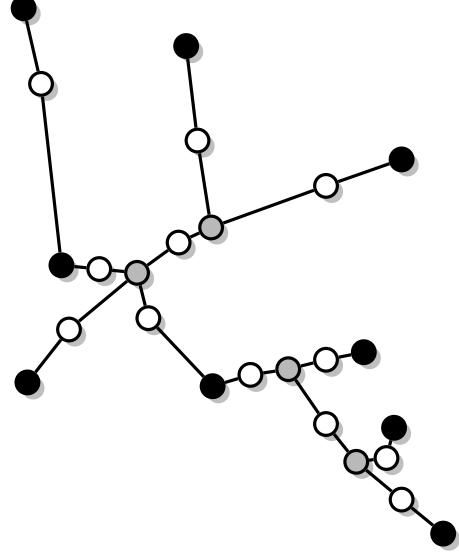


Figure 3.22: Auxiliary tree $\mathcal{T}_Z$ built from vertices of set $\mathcal{Z}$.

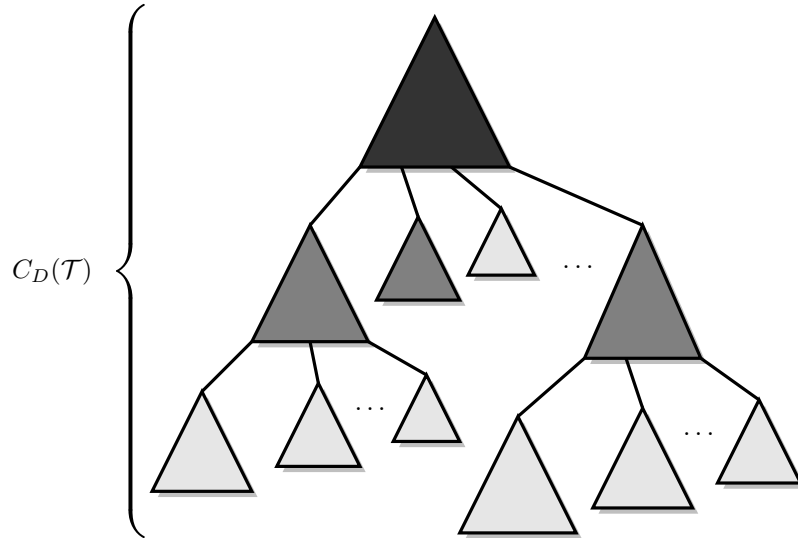


Figure 3.23: The structure of the decision tree D , built by the Algorithm 7. The dark gray subtree represents the decision tree D_Z , obtained by calling the QPTAS for $\mathcal{T}_Z$, c and $\epsilon = 1$. Gray subtrees represent decision trees D_H , each built for a unique heavy module $H \subseteq V(\mathcal{T}')$ of every $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$, by calling the RANKINGBASEDDT procedure for $\mathcal{T}' \setminus H$. Light gray subtrees represent decision trees D_L , built for each $L \in \mathcal{T}' - H$, by recursively calling CREATEDECISIONTREE with L , c and $(a/2, a]$.

Let $D = D_{\mathcal{Z}}$. For each connected component $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$, we build a new decision tree as follows: By the construction of $\mathcal{Z}$, all heavy vertices in $V(\mathcal{T}')$ form a single heavy module $H \subseteq V(\mathcal{T}')$. We create a new decision tree D_H for $\mathcal{T}' \setminus H$, by calling the **RankingBasedDT** procedure with argument $\mathcal{T}' \setminus H$ and we hang D_H in D below the unique last query to a vertex in $N_{\mathcal{T}'}(\mathcal{T}')$ (which is possible due to Observation 3.2.4.5). As, by Observation 3.2.4.4, D_H is a partial decision tree for $\mathcal{T}'$, it follows that D is also a partial decision tree for $\mathcal{T}$.

Now notice that for each $L \in \mathcal{T}' - H$, there is no $v \in V(L)$, such that $c(v) > a$. This allows us to create a decision tree D_L recursively, by calling the **CreateDecisionTree** procedure with arguments L , c and $(a/2, a]$. Next, we hang D_L in D below the unique last query to a vertex in $N_{\mathcal{T}'}(L)$ (again, using Observation 3.2.4.5). Since after all such operations, every vertex $v \in V(\mathcal{T})$ also belongs to D , we obtain a valid decision tree D for $\mathcal{T}$. To see example structure of such solution, see Figure 3.23.

Analysis of the algorithm

Lemma 3.2.4.8. *Let $\mathcal{T}_{\mathcal{Z}}$ be the auxiliary tree. Then, $|V(\mathcal{T}_{\mathcal{Z}})| \leq 4k - 3$.*

Proof. First, we show that $|\mathcal{Y}| \leq 2k - 1$. We use induction on the elements of $\mathcal{H}$. We construct a family of sets $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_{|\mathcal{H}|}$, such that for every integer $1 \leq h \leq |\mathcal{H}|$, $|\mathcal{H}_h| = h$ and $\mathcal{H}_{|\mathcal{H}|} = \mathcal{H}$. For each $\mathcal{H}_h$, we also construct a corresponding set $\mathcal{Y}_h$, eventually ensuring that $\mathcal{Y}_{|\mathcal{H}|} = \mathcal{Y}$.

Let $\mathcal{H}_1 = \emptyset$, $\mathcal{Y}_1 = \emptyset$. Pick any heavy module $H \subseteq V(\mathcal{T})$ and add it to $\mathcal{H}_1$. Add the unique vertex v , such that $v \in H \cap \mathcal{X}$ to $\mathcal{Y}_1$, so that $|\mathcal{Y}_1| = 1$. Assume by induction that for some $h \geq 1$, $|\mathcal{Y}_h| \leq 2h - 1$. Two heavy modules $H_1, H_2 \subseteq V(\mathcal{T})$ will be called *neighbors* if for every $H_3 \subseteq V(\mathcal{T})$ with $H_3 \neq H_1, H_2$, we have $\mathcal{P}_{\mathcal{T}}(H_1, H_2) \cap H_3 = \emptyset$. Pick $H \in \mathcal{H}$, such that $H \notin \mathcal{H}_h$ to be a heavy module that is a neighbor of some member of $\mathcal{H}_h$. We define $\mathcal{H}_{h+1} = \mathcal{H}_h \cup \{H\}$. Let z be the unique vertex, such that $v \in H \cap \mathcal{X}$, and let $\mathcal{Y}_{h+1} = \mathcal{Y}_h \cup \{z\}$. Define $\mathcal{T}_{h+1} = \mathcal{T} \setminus \{v \in \mathcal{Y}_{h+1} \mid \mathcal{P}_{\mathcal{T}}(v, z) \cap \mathcal{Y}_{h+1} = \emptyset\}$. Note that $\mathcal{T}_{h+1}$ is a spider (a tree with at most one vertex of degree above 2). Add to $\mathcal{Y}_{h+1}$ the unique vertex $v \in V(\mathcal{T}_{h+1})$, such that $\deg_{\mathcal{T}_{h+1}}(v) \geq 3$, if it exists. Clearly, $|\mathcal{Y}_{h+1}| \leq 2h + 1$, completing the induction.

By construction, $\mathcal{H}_{|\mathcal{H}|} = \mathcal{H}$ and $\mathcal{Y}_{|\mathcal{H}|} = \mathcal{Y}$, so $|\mathcal{Y}| \leq 2 \cdot |\mathcal{H}| - 1 \leq 2k - 1$ where the last inequality is by Observation 3.2.4.2. As paths between vertices in $\mathcal{Y}$ form a tree when contracted, at most $2k - 2$ additional vertices are added while constructing $\mathcal{Z}$ (at most one per path). The lemma follows. $\square$

Lemma 3.2.4.9. *Let $\mathcal{T}_{\mathcal{Z}}$ be the auxiliary tree. Then, $\text{OPT}(\mathcal{T}_{\mathcal{Z}}) \leq \text{OPT}(\mathcal{T})$.*

Proof. Let D^* be the decision tree for $\mathcal{T} \setminus \mathcal{Z}$. We build a new decision tree $D'_{\mathcal{Z}}$ for $\mathcal{T}_{\mathcal{Z}}$ by transforming D^* as follows:

Let $u, v \in \mathcal{Y}$, such that $\mathcal{P}_{\mathcal{T}}(u, v) \neq \emptyset$ and $\mathcal{P}_{\mathcal{T}}(u, v) \cap \mathcal{Y} = \emptyset$. Let $q \in V(D^*)$ be the first query to a vertex among $\mathcal{P}_{\mathcal{T}}(u, v)$. Recall that we picked $v_{u,v} = \arg \min_{z \in \mathcal{P}_{\mathcal{T}}(u, v)} \{c(z)\}$, so $c(v_{u,v}) \leq c(q)$. We replace q in D^* with the query to $v_{u,v}$ and delete all queries to vertices in $\mathcal{P}_{\mathcal{T}}(u, v) - v_{u,v}$. By construction, $D'_{\mathcal{Z}}$ is a valid decision tree for $\mathcal{T}_{\mathcal{Z}}$, and by choosing $v_{u,v}$ to minimize c , we did not increase the cost, so we have that:

$$C_{\mathcal{T}_{\mathcal{Z}}}(D'_{\mathcal{Z}}) \leq \text{OPT}(\mathcal{T} \setminus \mathcal{Z}).$$

Therefore, we have:

$$\text{OPT}(\mathcal{T}_{\mathcal{Z}}) \leq C_{\mathcal{T}_{\mathcal{Z}}}(D'_{\mathcal{Z}}) \leq \text{OPT}(\mathcal{T} \setminus \mathcal{Z}) \leq \text{OPT}(\mathcal{T}),$$

where the first inequality is due to the definition of optimality and the last inequality follows by Lemma 3.2.4.1. $\square$

Lemma 3.2.4.10. *Let H be the unique heavy module of $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$. Then, the decision tree D_H is of cost at most:*

$$C_{\mathcal{T}' \setminus H}(D_H) \leq 2 \cdot \text{OPT}(\mathcal{T}).$$

Proof. For every $v \in H$ let $c'(v) = a$. We have $2c'(v) \geq bc'(v)/a = b \geq c(v)$ so we get that $2 \cdot C_{\mathcal{T}' \setminus H}(D_H, c') \geq C_{\mathcal{T}' \setminus H}(D_H, c)$. Additionally, using the fact that $c'(v) \leq c(v)$ we have that $\text{OPT}(\mathcal{T}' \setminus H, c') \leq \text{OPT}(\mathcal{T}' \setminus H, c)$. Hence:

$$\begin{aligned}
C_{\mathcal{T}'\langle H \rangle}(D_H, c) &\leq 2 \cdot C_{\mathcal{T}'\langle H \rangle}(D_H, c') = 2 \cdot \text{OPT}(\mathcal{T}'\langle H \rangle, c') \\
&\leq 2 \cdot \text{OPT}(\mathcal{T}'\langle H \rangle, c) \leq 2 \cdot \text{OPT}(\mathcal{T}, c)
\end{aligned}$$

where the equality is by the Corollary 3.1.0.1 and the last inequality is due to the fact that $\mathcal{T}'\langle H \rangle$ is a subtree of $\mathcal{T}'$, which is a subtree of $\mathcal{T}$ (Lemma 3.2.4.1). $\square$

The main result

Let d be the remaining depth of the recursion call performed in Line 7 of the algorithm, i.e., the number of recursive steps from the current call to the base case (for the base case, this value is equal to $d = 0$). We show that at each level of the recursion we pay $O(\text{OPT}(T))$, so the approximation ratio of the algorithm is bounded by $O(d)$:

Lemma 3.2.4.11. $C_{\mathcal{T}}(D) \leq (4d + 2) \cdot \text{OPT}(T)$.

Proof. Let $Q_D(\mathcal{T}, x)$ be the sequence of queries performed in order to find $x \in V(\mathcal{T})$. By construction of Algorithm 7, $Q_D(\mathcal{T}, x)$ consists of at most three distinct subsequences of queries (see Figure 3.23):

1. Firstly, there is a sequence of queries belonging to $Q_{D_{\mathcal{Z}}}(\mathcal{T}_{\mathcal{Z}}, x)$.
2. If $x \notin \mathcal{Z}$, then, there is a sequence of queries belonging to $Q_{D_H}(\mathcal{T}'\langle H \rangle, x)$ for a unique heavy group $H \subseteq V(\mathcal{T}')$ of $\mathcal{T}' \in \mathcal{T} - \mathcal{Z}$, such that $x \in \mathcal{T}'$.
3. At last, if $x \notin H$, there is a sequence of queries belonging to $Q_{D_L}(L, x)$ for $L \in \mathcal{T}' - H$, such that $x \in V(L)$.

Note that it sometimes may happen that some of the above sequences are empty.

We prove by induction that $C_{\mathcal{T}}(D) \leq (4d + 2) \cdot \text{OPT}(T)$. When $d = 0$ (the base case), the induction hypothesis is true, due to the Lemma 3.2.4.6. For $d > 0$, assume by induction that the cost of the decision tree built for each L , is at most $C_{\mathcal{T}}(D_L) \leq (4(d - 1) + 2) \cdot \text{OPT}(T)$. We have:

$$\begin{aligned}
C_{\mathcal{T}}(D) &\leq C_{\mathcal{T}}(D_{\mathcal{Z}}) + \max_{\mathcal{T}' \in \mathcal{T} - \mathcal{Z}} \left\{ C_{\mathcal{T}'\langle H \rangle}(D_H) + \max_{L \in \mathcal{T}' - H} \{ C_L(D_L) \} \right\} \\
&\leq C_{\mathcal{T}_{\mathcal{Z}}}(D_{\mathcal{Z}}) + \max_{\mathcal{T}' \in \mathcal{T} - \mathcal{Z}} \{ 2 \cdot \text{OPT}(\mathcal{T}) + (4(d - 1) + 2) \cdot \text{OPT}(T) \} \\
&\leq 2 \cdot \text{OPT}(\mathcal{T}_{\mathcal{Z}}) + 2 \cdot \text{OPT}(T) + (4(d - 1) + 2) \cdot \text{OPT}(T) \\
&\leq 2 \cdot \text{OPT}(\mathcal{T}) + 4d \cdot \text{OPT}(T) = (4d + 2) \cdot \text{OPT}(T)
\end{aligned}$$

where the first inequality is due to the construction of the decision tree returned by the Algorithm 7, the second inequality is by Observation 3.2.4.7, Observation 3.2.4.10 and by the induction hypothesis, the third inequality is due to Theorem 3.2.2.1 and using the fact that $\mathcal{T}$ is a subtree of T (Lemma 3.2.4.1) and the last inequality is due to Lemma 3.2.4.9. $\square$

We are now ready to prove our main theorem:

Theorem 3.2.4.12. *There exists an $O(\log \log n)$ -approximation algorithm for the Tree Search Problem running in $k^{O(\log k)} \cdot \text{poly}(n)$ time.*

Proof. Let $D = \text{CreateDecisionTree}(T, (2^{\lceil \log \log n \rceil - 1} / \log n, 1])$. Since there are at most $\lceil \log \log n \rceil + 1$ intervals processed, the depth of the recursion is bounded by $d \leq \lceil \log \log n \rceil \leq \log \log n + 1$. Hence, using Lemma 3.2.4.11 we get that:

$$C_{\mathcal{T}}(D) \leq (4 \cdot \log \log n + 6) \cdot \text{OPT}(T) = O(\log \log n \cdot \text{OPT}(T)).$$

By Observation 3.2.4.3, for every subtree $\mathcal{T}$ of T , processed at some level of the recursion, we have $k(\mathcal{T}) \leq k(T)$. Using Lemma 3.2.4.8, at each such level the call to the QPTAS from Theorem 3.2.2.1 (line 7 of the CREATEDECISIONTREE) runs in time bounded by:

$$k(\mathcal{T})^{O(\log(4 \cdot k(\mathcal{T})))} = k(T)^{O(\log k(T))}.$$

Since $d = O(\text{poly}(n))$ and all other computation can be performed in polynomial time, the overall running time is bounded by $k^{O(\log k)} \cdot \text{poly}(n)$, as required. $\square$

Chapter 4

Experimental Results

The following chapter is concerned with a series of experiments conducted in order to evaluate the performance of the algorithms for the Tree Search Problem presented in the previous chapter.

Our main goal was to compare the quality of the solutions produced by the different algorithms on various classes of trees and cost functions and the computation time required to obtain such results. We measured the performance of our algorithms on randomly generated trees of three main classes: random trees, random trees with bounded degree and random trees with a bounded diameter. For the two latter, we have chosen trees of degree at most 3 and diameter at most 6, since these are minimal values of these parameters for which the Tree Search Problem with non uniform costs is known to be NP-hard [Cic+12]. We also used three different ways of generating the cost functions: random uniform distribution, binomial distribution and exponential distribution.

We implemented the previously described algorithms using python programming language. The experiments were conducted on a machine with 32 x AMD Ryzen 9 7950X 16-Core Processor and 128 GB of RAM. Each process was limited to use at most 32 GB of RAM and was single-threaded.

4.1 Overview of the experiments

The experimental analysis is divided into two sections concerning uniform and non-uniform query costs. In the first section we measured the running time of the **RankingBasedDT** algorithm from Section 3.1 and the depth of the decision trees produced by it. The second section was concerned with the non-uniform cost model and included the evaluation of the $O(\log n / \log \log n)$ -approximation algorithm from Section 3.2.1, the $O(\sqrt{\log n})$ -approximation algorithm from Section 3.2.2 based on the QPTAS and the $O(\log \log n)$ -approximation algorithm from Section 3.2.4. We compared the quality of the solutions produced by these algorithms on various classes of trees and cost functions and the computation time required to obtain these results. Additionally, for small input sizes we managed to calculate the optimal solutions using the exact exponential time algorithm. This allowed us to compare the approximation ratios of the algorithms with respect to the optimal solutions.

We have also employed a few heuristic speed-ups, particularly for the QPTAS. Firstly, instead of using $\epsilon = 1$ we set it to $\epsilon = 59$, which reduces most of the computational complexity of each QPTAS call. For both the $O(\sqrt{\log n})$ -approximation algorithm and the $O(\log \log n)$ -approximation algorithm this does not impact the asymptotic guarantee of the algorithms. Secondly, a careful reading of the analysis of the QPTAS reveals that the depth of the considered boxed decision trees can be limited to $\text{rank}(T)$. This reduces the running time of the QPTAS to $n^{O(\log \text{rank}(T)/\epsilon^2)}$, which in practice is usually slightly better than the upper bound $\text{rank}(T) = O(\log n)$.

Lastly we decided to simplify the implementation of the linear time algorithm for $T||V||C_{max}$ based on calculating the ranking coloring. We implemented this algorithm naively with $O(n \log n)$ time complexity. This is done in order to simplify the already complex implementation. Additionally, this running time is negligible compared to the other algorithms and we are mainly focused on the quality of the solutions produced by them. Also note that although we performed experiments for the uniform costs variant of the problem using this algorithm, the main focus of this thesis is the non-uniform costs model and this algorithm serves as an essential subroutine for both QPTAS and the $O(\log \log n)$ -approximation algorithm, rather than a standalone procedure.

Tree Generation Methods

We used three different methods of generating random trees for our experiments: random trees, random trees with bounded degree and random trees with bounded diameter. Table 4.1 summarizes all tree generation methods used in our experiments. For the uniform costs model, we decided to measure the performance of our algorithms on instances of sizes 10 to 1000 with 10 instances per size. For non-uniform costs model, we focused on relatively small instances, 10 to 100 vertices per instance, since the complex dynamic programming subroutines of our algorithm require significant computation time even for such sizes.

Table 4.1: Tree generation methods and their properties

Method	Parameters	Description
Random	n	Uniformly random labeled tree generated using Prüfer sequence
Random Bounded Degree	n, Δ	Random tree with maximum degree constraint Δ (each vertex has degree $\leq \Delta$)
Random Bounded Diameter	n, D	Random tree with maximum diameter constraint D (maximum distance between any two vertices $\leq D$)

Detailed Generation Algorithms

Random Trees. Generated using NetworkX’s `random_labeled_tree` function, which employs Prüfer sequences to sample uniformly from all labeled trees on n vertices. A random vertex is chosen as root, and the tree is oriented via BFS traversal from this root.

Random Bounded Degree Trees. Constructed incrementally by maintaining a list of available parent nodes with fewer than $\Delta - 1$ children. Each new vertex is attached to a uniformly random

available parent. When a parent reaches its degree limit, it is removed from the available list. This ensures every vertex has total degree at most Δ .

Random Bounded Diameter Trees. For diameter bound D , vertices are distributed across $h = \lfloor D/2 \rfloor + 1$ levels. The root occupies level 0, and remaining vertices are randomly assigned to levels $1, \dots, h - 1$. Each non-root vertex connects to a random parent in the previous level, ensuring the diameter does not exceed D .

Cost Function Generation

Table 4.2 lists all probability distributions used for generating the vertex costs. Note that after generation, all costs are normalized so that the maximum cost equals 1. This is done due to the fact that it is convenient for the design of the QPTAS which is used as an essential subroutine for both $O(\sqrt{\log n})$ and the $O(\log \log n)$ -approximation algorithm.

Table 4.2: Probability distributions for vertex costs

Distribution	Formula	Properties
Uniform $\mathcal{U}$	$f(x) = \begin{cases} 1/b & \text{if } 0 \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$	All values in $[0, 1]$ equally likely, $\mathbb{E}[X] = 0.5$
Binomial $\mathcal{B}$	$\mathbb{P}[X = k] = \binom{n}{k} p^k (1-p)^{n-k}$	Discrete distribution with parameters $n = 10, p = 0.5, \mathbb{E}[X] = np = 5$
Exponential $\mathcal{E}$	$f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$	Continuous distribution with rate $\lambda = 1, \mathbb{E}[X] = 1/\lambda = 1$

Extended Three-Field Notation for Experiments

To systematically describe our experimental configurations, we extend the theoretical three-field notation $\alpha||\beta||\gamma$ to include specific tree generation methods and distribution types:

Field α (Search Space): In experiments, we specify the specific tree generation method using subscripts:

- T_R - Random trees
- T_Δ - Random bounded degree trees
- T_D - Random bounded diameter trees

Field β (Query Characteristics): We consider four cost scenarios:

- V - vertex queries with uniform unit costs ($c(v) = 1$ for all v)
- $V, c = \mathcal{U}$ - vertex queries with costs from uniform distribution $\mathcal{U}(0, 1)$
- $V, c = \mathcal{B}$ - vertex queries with costs from binomial distribution $\text{Binomial}(n = 10, p = 0.5)$
- $V, c = \mathcal{E}$ - vertex queries with costs from exponential distribution $\mathcal{E}(\lambda = 1)$

Performance Metrics

Throughout the experimental analysis, we report the following metrics:

- **Avg Time (s)** - Average CPU time in seconds over all instances of a given size.
- **Max Time (s)** - Maximum CPU time in seconds among all instances of a given size.
- **Avg Depth** - Average depth of the decision tree produced by the algorithm, averaged over all instances of a given size.
- **Max Depth** - Maximum depth among all decision trees among all instances of a given size.
- **Avg Ratio** - Average approximation ratio over all instances of a given size.
- **Max Ratio** - Maximum approximation ratio among all instances of a given size.

Experiment Coverage Matrix

Table 4.3 contains references to the corresponding subsections containing particular results.

Table 4.3: Experiment coverage: tree generation methods vs. problem variants and cost distributions

Tree Type/Cost function	$T V C_{max}$ (uniform costs)	$T V, c C_{max}$		
		$c \sim \mathcal{U}$	$c \sim \mathcal{B}$	$c \sim \mathcal{E}$
Random trees T_R	4.2.1	4.3.1	4.3.2	4.3.3
Random bounded degree trees T_Δ	4.2.2	4.3.4	4.3.5	4.3.6
Random bounded diameter trees T_D	4.2.3	4.3.7	4.3.8	4.3.9

4.2 Trees with Uniform Cost ($T||V||C_{max}$)

This section presents experimental results for the Tree Search Problem with uniform query costs.

4.2.1 Random Trees ($T_R||V||C_{max}$)

Table 4.4: Performance on random trees with uniform query costs

n	Avg Depth	Max Depth	Avg Time (s)	Max Time (s)
10	3.80	4	0.001514	0.001936
20	4.60	5	0.003452	0.003635
30	5.00	5	0.006719	0.007023
40	5.10	6	0.009460	0.009888
50	5.70	6	0.012409	0.012886
60	6.00	6	0.015446	0.016239
100	6.50	7	0.021648	0.046397
150	6.90	7	0.034571	0.036309
200	7.10	8	0.051168	0.077717
250	7.90	8	0.088678	0.106789
300	8.00	8	0.110067	0.140505
350	8.00	8	0.122534	0.150200
400	8.10	9	0.139465	0.174724
450	8.20	9	0.110219	0.125244
500	8.80	9	0.129039	0.147685
550	8.40	9	0.212270	0.234442
600	8.90	9	0.168215	0.177733
650	8.70	9	0.182155	0.190271
700	9.00	9	0.270885	0.309166
750	9.00	9	0.299953	0.355814
800	9.00	9	0.275221	0.295997
850	9.10	10	0.241985	0.250178
900	9.10	10	0.359294	0.398942
950	9.20	10	0.395425	0.455887
1000	9.30	10	0.430645	0.459383

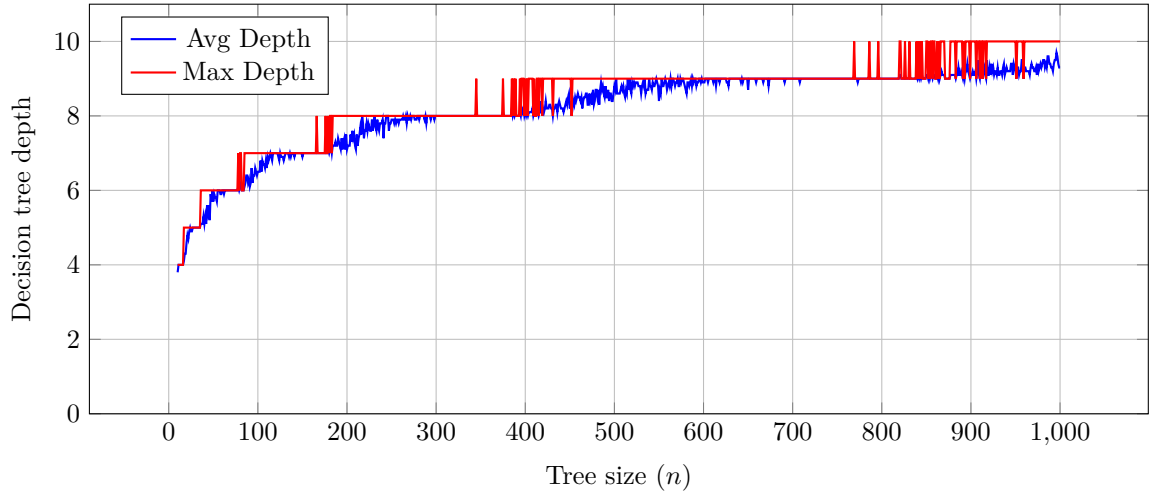


Figure 4.1: Decision tree depth for random trees (uniform query costs)

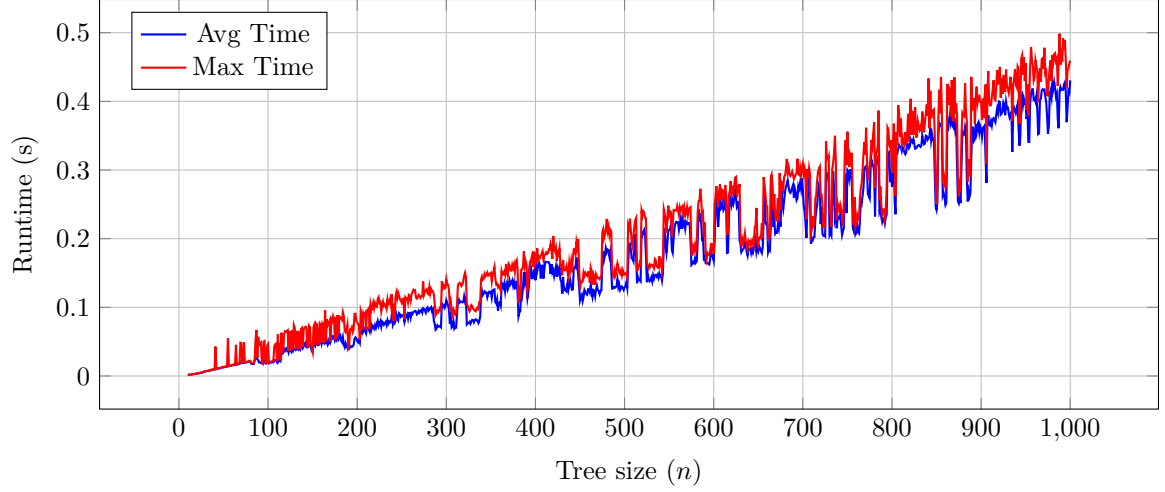


Figure 4.2: Runtime for random trees (uniform query costs)

It is clearly visible that the running time of the algorithm on randomly generated instances is almost linear. It behaves stably and does not vary much among instances of the same size. As expected, the average and worst case depths of the decision trees built grows logarithmically in n . This suggest that the typical instances for the problem achieve the asymptotical upper bound of at most $O(\log n)$ queries in the worst case.

4.2.2 Random Bounded Degree Trees ($T_\Delta || V || C_{max}$)

Table 4.5: Performance on bounded degree trees with uniform query costs

n	Avg Depth	Max Depth	Avg Time (s)	Max Time (s)
10	3.80	4	0.001883	0.002315
20	4.40	5	0.004077	0.004300
30	5.00	5	0.006379	0.006953
40	5.60	6	0.009117	0.009902
50	5.60	6	0.008258	0.008587
60	6.00	6	0.014716	0.015088
100	6.30	7	0.028718	0.054380
150	7.00	7	0.040778	0.041550
200	7.50	8	0.057506	0.058477
250	8.00	8	0.081548	0.106034
300	8.00	8	0.105412	0.130591
350	8.00	8	0.121891	0.140366
400	8.00	8	0.087627	0.090278
450	8.10	9	0.107712	0.123486
500	8.40	9	0.175684	0.199389
550	8.80	9	0.167661	0.215635
600	9.00	9	0.191693	0.206806
650	9.00	9	0.233231	0.263469
700	9.00	9	0.230719	0.241494
750	9.00	9	0.279499	0.292872
800	9.00	9	0.291323	0.305180
850	9.00	9	0.316835	0.327101
900	9.10	10	0.341218	0.350593
950	9.20	10	0.315355	0.353623
1000	9.00	9	0.262918	0.273126

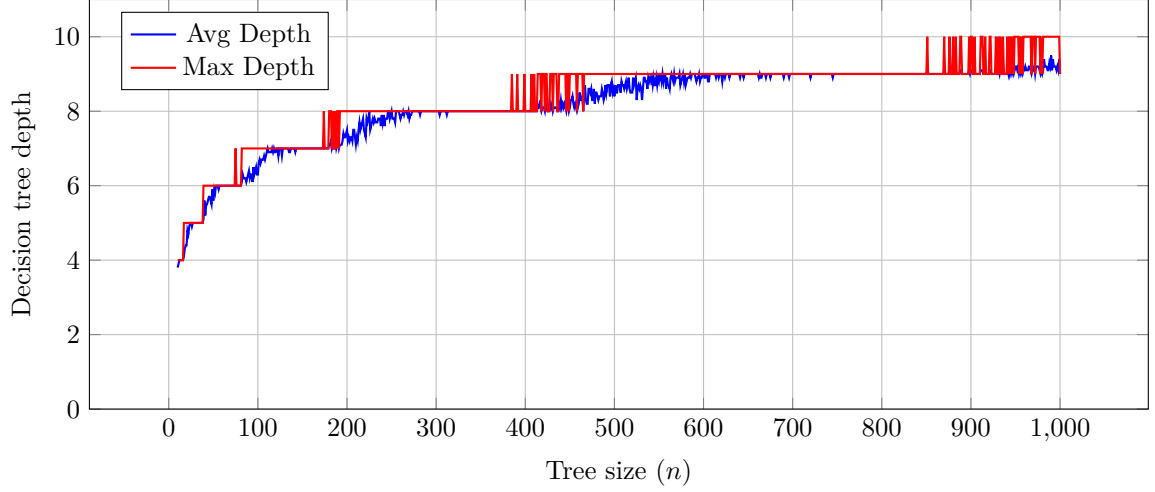


Figure 4.3: Decision tree depth for bounded degree trees (uniform query costs)

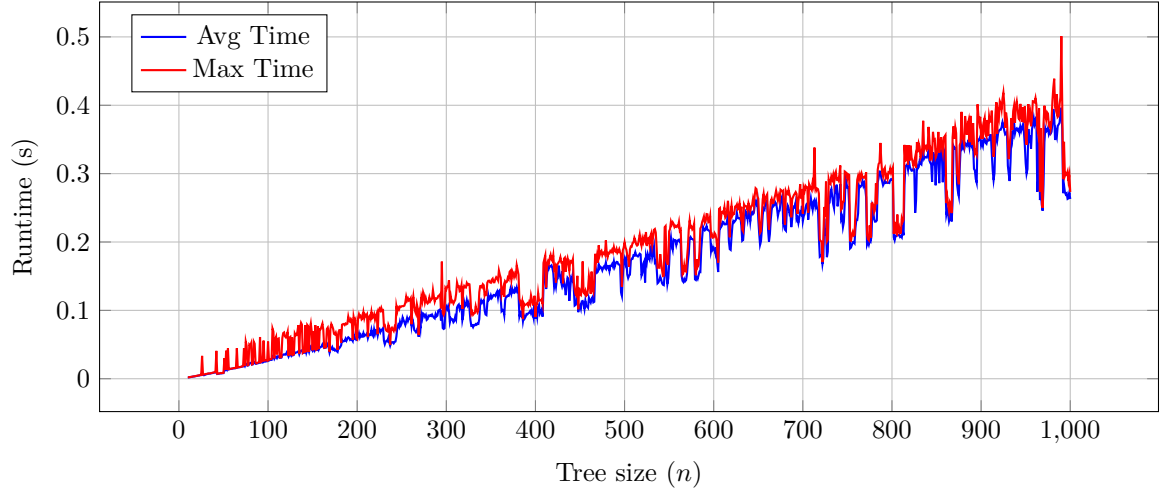


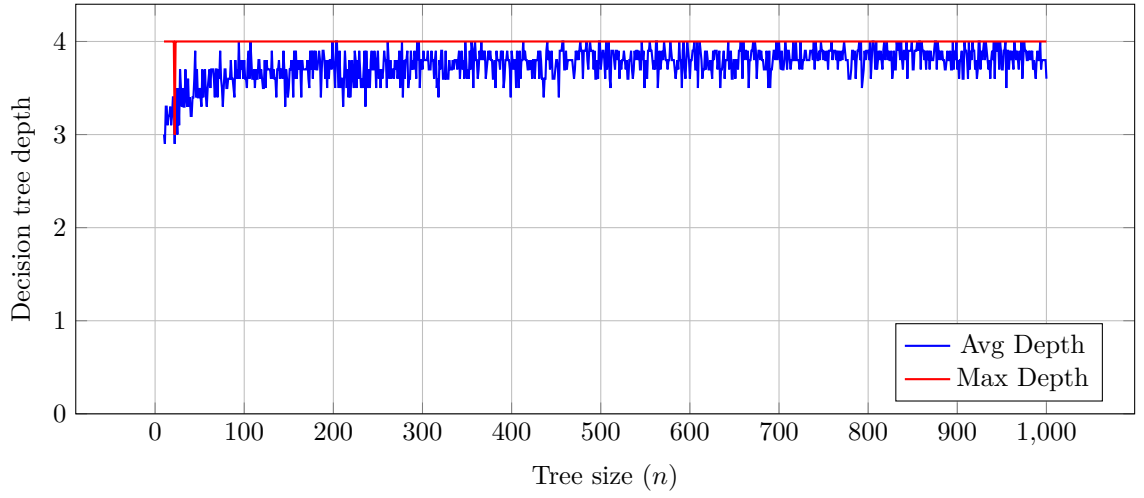
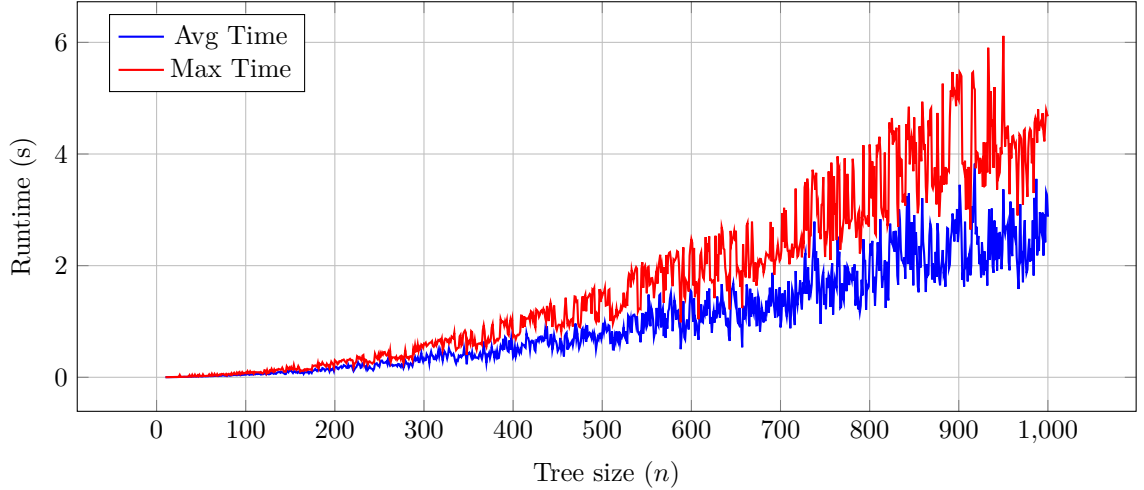
Figure 4.4: Runtime for bounded degree trees (uniform query costs)

For bounded degree trees, the algorithm behaves similarly as for the random trees. The running time of the algorithm grows linearly, a little bit slower than for the random trees. The stability is also preserved. Again, as expected, the average and worst case depths of the decision trees built grows logarithmically in n . This is easily explainable since it is not hard to see that $\text{rank}(T) = \Omega(\log_{\Delta} n)$ because each query results in at least one response of size at least $\frac{n-1}{\Delta}$.

4.2.3 Random Bounded Diameter Trees ($T_D || V || C_{max}$, $D = 6$)

Table 4.6: Performance on bounded diameter trees with uniform query costs

n	Avg Depth	Max Depth	Avg Time (s)	Max Time (s)
10	3.00	4	0.001891	0.002284
20	3.40	4	0.004504	0.004940
30	3.30	4	0.007768	0.009012
40	3.20	4	0.014947	0.045265
50	3.50	4	0.015760	0.020563
60	3.60	4	0.021442	0.027050
100	3.80	4	0.047224	0.085641
150	3.50	4	0.118843	0.148323
200	3.80	4	0.183898	0.304099
250	3.90	4	0.195326	0.339757
300	3.70	4	0.337928	0.602597
350	3.60	4	0.421854	0.613242
400	3.90	4	0.357429	0.727468
450	3.80	4	0.468534	0.910761
500	3.70	4	0.773654	1.458311
550	3.80	4	0.908509	1.504898
600	3.60	4	1.529075	2.472266
650	3.80	4	1.540004	2.624646
700	4.00	4	0.893710	2.124919
750	3.80	4	1.477481	2.154393
800	3.90	4	2.113809	4.169857
850	3.70	4	2.906338	4.655112
900	3.70	4	2.625933	4.967109
950	3.70	4	3.366969	6.116271
1000	3.60	4	2.875865	4.676517

**Figure 4.5:** Decision tree depth for bounded diameter trees (uniform query costs)**Figure 4.6:** Runtime for bounded diameter trees (uniform query costs)

For bounded degree trees the situation is quite different. The running time of our procedure achieves the worst case upper bound of $O(n \log n)$. Also the running time is more varying and unstable.

Conversely, the depth of the decision trees built is strictly upper bounded by 4. This is due to the fact that the depth of the decision tree is trivially upper bounded by $\left\lceil \frac{\text{diam}(T)+1}{2} \right\rceil$.

4.3 Trees with Non-Uniform Costs ($T||V, c||C_{max}$)

This section evaluates algorithms for the worst-case search problem with non-uniform query costs. Due to the limited computational resources and the increased complexity of handling non-uniform costs, for each algorithm, we include results up to the largest tree size that could be processed within the available time. Since we are interested in the approximation ratios achieved by the algorithms, for small instances for which we were able to compute the optimal solution, we include comparisons against the optimum as well. If such results are not available for a particular tree size, it is indicated in the respective tables by the N/A entry.

As the running times of the algorithms vary significantly, all plots of the running times use a logarithmic scale on the y-axis to better visualize these differences. Also, since the running times are often polynomial or high degree it would be hard to distinguish their running times from the exponential time procedures on the linear scale.

4.3.1 Random Trees with Uniform Cost Distribution ($T_R||V, c \sim \mathcal{U}||C_{max}$)

Exact Algorithm

Table 4.7: Random trees with uniform cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.515344	0.996765
11	0.688948	1.871738
12	1.118181	1.820036
13	3.752255	5.853787
14	3.924208	5.459482
15	6.742663	12.371523
16	8.338989	17.136759
17	13.161961	19.730425
18	49.783415	217.751785
19	53.941062	108.616709
20	72.510728	279.192467
21	126.797035	323.067169
22	146.939623	328.078981
23	159.220462	248.346399
24	444.063066	1208.271180
25	948.392856	3652.773897
26	919.064031	2638.620480
27	2219.712304	4494.591545
28	4474.570226	15483.063044
29	6210.535691	22857.351522
30	11405.857347	51794.339840
31	8092.897929	24723.579952
32	19858.948001	50624.244158
33	17082.196660	22448.280074

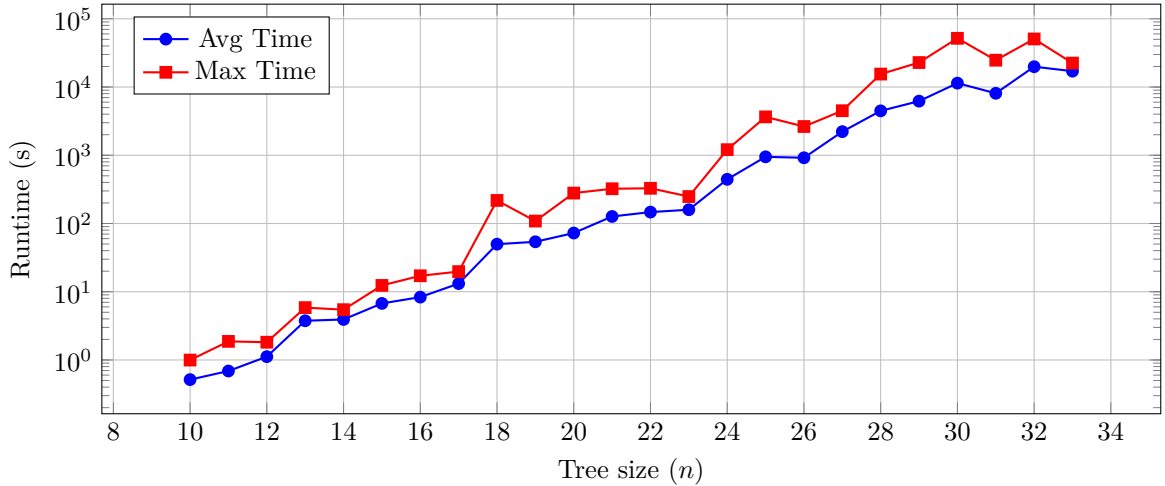


Figure 4.7: Random trees with uniform costs - exact algorithm: runtime

$O(\log n / \log \log n)$ -Approximation Algorithm

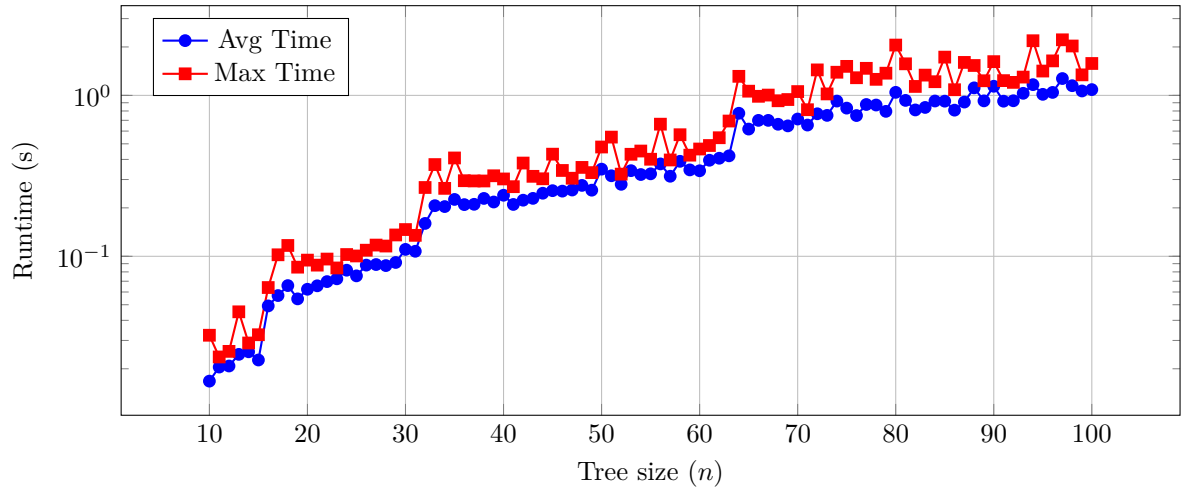


Figure 4.8: Random trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

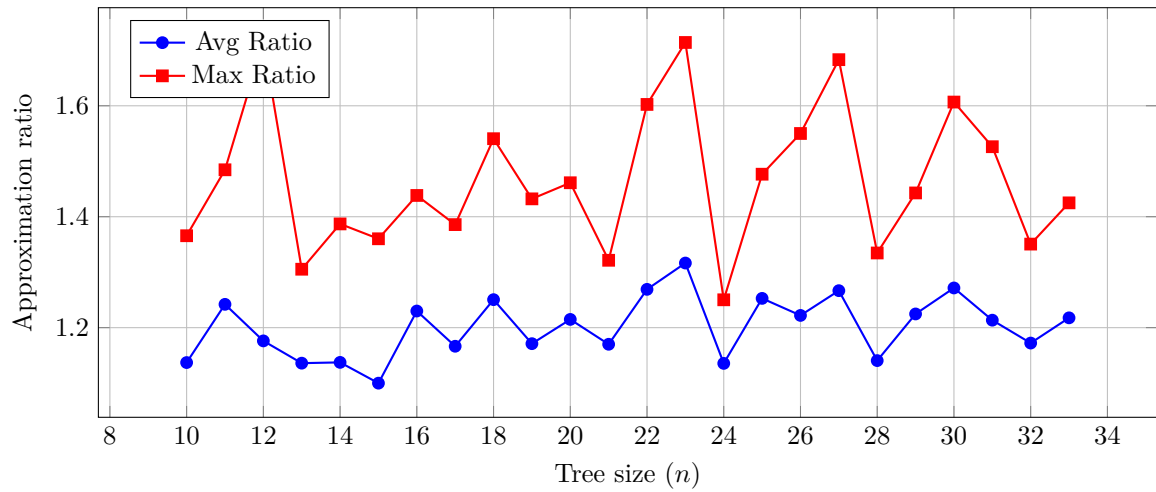


Figure 4.9: Random trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.8: Random trees with uniform cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.016762	0.032318	1.137075	1.365714
11	0.020501	0.023684	1.242042	1.484599
12	0.020812	0.025657	1.176145	1.715289
13	0.024581	0.045187	1.135985	1.305341
14	0.025502	0.028926	1.137403	1.387021
15	0.022680	0.032554	1.099938	1.360228
16	0.049179	0.063968	1.229984	1.438320
17	0.057024	0.102183	1.166470	1.385779
18	0.065770	0.116763	1.250493	1.540655
19	0.054388	0.085611	1.171139	1.432209
20	0.062404	0.094678	1.214856	1.461230
21	0.065561	0.088133	1.170052	1.321474
22	0.069631	0.096130	1.269055	1.602414
23	0.072472	0.084640	1.316487	1.714126
24	0.081970	0.102568	1.135594	1.250227
25	0.075609	0.100487	1.252776	1.476635
26	0.088151	0.109135	1.222057	1.550057
27	0.088868	0.117435	1.266685	1.683139
28	0.087563	0.115817	1.140615	1.334636
29	0.091547	0.135715	1.224709	1.442870
30	0.110397	0.146635	1.271571	1.606712
31	0.107470	0.134955	1.213483	1.526203
32	0.160086	0.267672	1.172232	1.350648
33	0.206370	0.371636	1.217703	1.424988
34	0.203727	0.264368	N/A	N/A
35	0.225650	0.408465	N/A	N/A
36	0.209532	0.294949	N/A	N/A
37	0.210439	0.294024	N/A	N/A
38	0.228890	0.293844	N/A	N/A
39	0.217514	0.316559	N/A	N/A
40	0.239622	0.302497	N/A	N/A
41	0.210024	0.270788	N/A	N/A
42	0.223183	0.379812	N/A	N/A
43	0.229074	0.314054	N/A	N/A
44	0.246577	0.303195	N/A	N/A
45	0.255775	0.431221	N/A	N/A
46	0.254112	0.341533	N/A	N/A
47	0.258181	0.305618	N/A	N/A
48	0.275816	0.356788	N/A	N/A
49	0.257603	0.331242	N/A	N/A
50	0.348037	0.477731	N/A	N/A
51	0.316554	0.550524	N/A	N/A
52	0.280130	0.324085	N/A	N/A
53	0.340562	0.430524	N/A	N/A
54	0.322247	0.451032	N/A	N/A
55	0.325494	0.400998	N/A	N/A
56	0.375180	0.662389	N/A	N/A
57	0.314435	0.396946	N/A	N/A
58	0.389405	0.569548	N/A	N/A
59	0.344359	0.425114	N/A	N/A
60	0.340081	0.464291	N/A	N/A
61	0.395388	0.488097	N/A	N/A
62	0.406840	0.545518	N/A	N/A
63	0.421006	0.692861	N/A	N/A
64	0.774373	1.313631	N/A	N/A
65	0.617531	1.061992	N/A	N/A
66	0.699391	0.986953	N/A	N/A
67	0.699400	1.004483	N/A	N/A
68	0.661586	0.925920	N/A	N/A
69	0.646546	0.942351	N/A	N/A
70	0.713039	1.056106	N/A	N/A
71	0.654112	0.814631	N/A	N/A
72	0.768729	1.440199	N/A	N/A
73	0.751237	1.021116	N/A	N/A
74	0.921879	1.392781	N/A	N/A
75	0.833485	1.514807	N/A	N/A
76	0.749446	1.287268	N/A	N/A
77	0.877686	1.472915	N/A	N/A
78	0.871158	1.258194	N/A	N/A
79	0.795813	1.373866	N/A	N/A
80	1.044580	2.054690	N/A	N/A
81	0.930796	1.571128	N/A	N/A
82	0.811977	1.136639	N/A	N/A
83	0.841508	1.336485	N/A	N/A
84	0.922589	1.216132	N/A	N/A
85	0.920700	1.729080	N/A	N/A
86	0.809507	1.084260	N/A	N/A
87	0.908831	1.602874	N/A	N/A
88	1.113665	1.533967	N/A	N/A
89	0.924536	1.232939	N/A	N/A
90	1.139403	1.620814	N/A	N/A
91	0.920372	1.235162	N/A	N/A
92	0.925961	1.204636	N/A	N/A
93	1.030268	1.299180	N/A	N/A
94	1.166125	2.185943	N/A	N/A
95	1.015151	1.416649	N/A	N/A
96	1.043433	1.638646	N/A	N/A
97	1.272435	2.215122	N/A	N/A
98	1.148774	2.027226	N/A	N/A
99	1.065212	1.341485	N/A	N/A
100	1.086270	1.578486	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.9: Random trees with uniform cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	3.667458	12.756751	1.175826	1.386895
11	6.128544	29.399862	1.305199	1.484599
12	4.272992	12.674806	1.305226	1.715289
13	2.433804	17.536711	1.169089	1.552337
14	0.718830	5.269130	1.207746	1.476948
15	0.869210	6.602548	1.136520	1.334345
16	27.517307	156.347909	1.242290	1.527108
17	128.062403	792.909551	1.175246	1.410590
18	24.850276	106.738707	1.234498	1.395855
19	11.442593	36.674916	1.224265	1.494932
20	27.091810	142.684030	1.288483	1.470591
21	117.702028	889.725199	1.272082	1.782426
22	61.510348	238.084481	1.382578	1.602414
23	42.097233	106.463449	1.326100	1.569625
24	39.732011	149.680999	1.289401	1.560927
25	27.877790	88.513900	1.246003	1.426526
26	129.002198	827.922871	1.383815	1.731408
27	21.298450	45.592222	1.337913	1.612733
28	32.335598	89.950481	1.166253	1.414333
29	138.223743	1199.429222	1.345866	1.661554
30	38.511334	65.983931	1.270631	1.479690
31	45.612598	124.638447	1.262941	1.665712
32	41.170141	83.916606	1.213553	1.440375
33	59.477436	271.004911	1.132198	1.201444
34	10.454132	28.285260	N/A	N/A
35	56.178653	189.509749	N/A	N/A
36	23.174181	55.223033	N/A	N/A
37	420.367692	3115.145578	N/A	N/A
38	77.981897	523.390589	N/A	N/A
39	20.521917	82.864569	N/A	N/A
40	80.364828	258.686817	N/A	N/A
41	29.889535	66.581582	N/A	N/A
42	1408.819309	7446.854561	N/A	N/A
43	762.760202	2743.887134	N/A	N/A
44	1170.757902	2836.208023	N/A	N/A
45	3497.560223	14352.968259	N/A	N/A
46	395.566427	910.341421	N/A	N/A
47	3811.398716	16410.480209	N/A	N/A
48	3495.496766	27287.483563	N/A	N/A
49	2659.659927	15202.043831	N/A	N/A
50	8457.697716	42358.960755	N/A	N/A
51	3506.404116	22816.730345	N/A	N/A
52	561.841314	1359.119127	N/A	N/A
53	1328.843090	2958.886347	N/A	N/A
54	321.809560	1099.148587	N/A	N/A
55	16207.413238	70117.505497	N/A	N/A
56	1235.476850	4147.176084	N/A	N/A

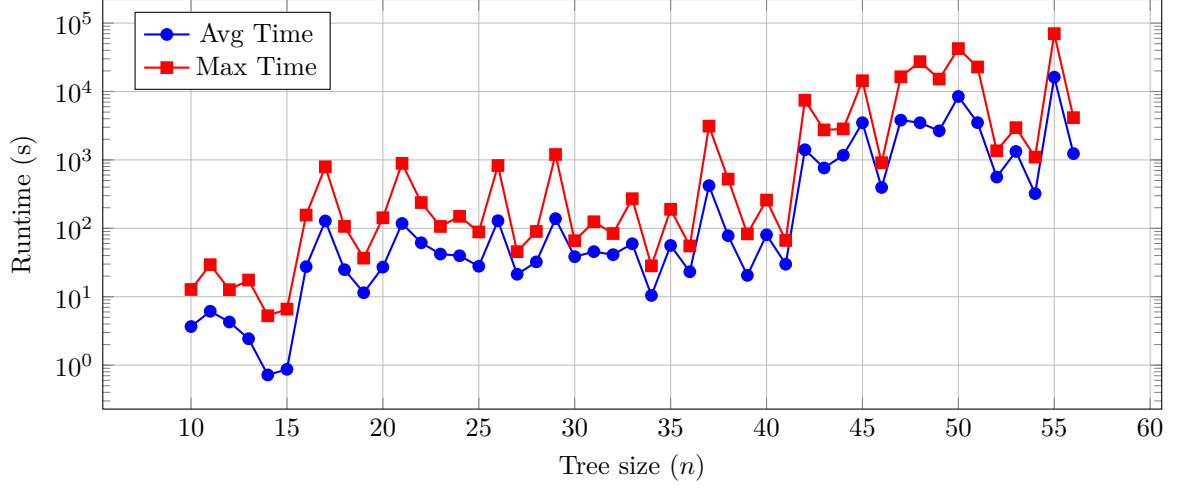


Figure 4.10: Random trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

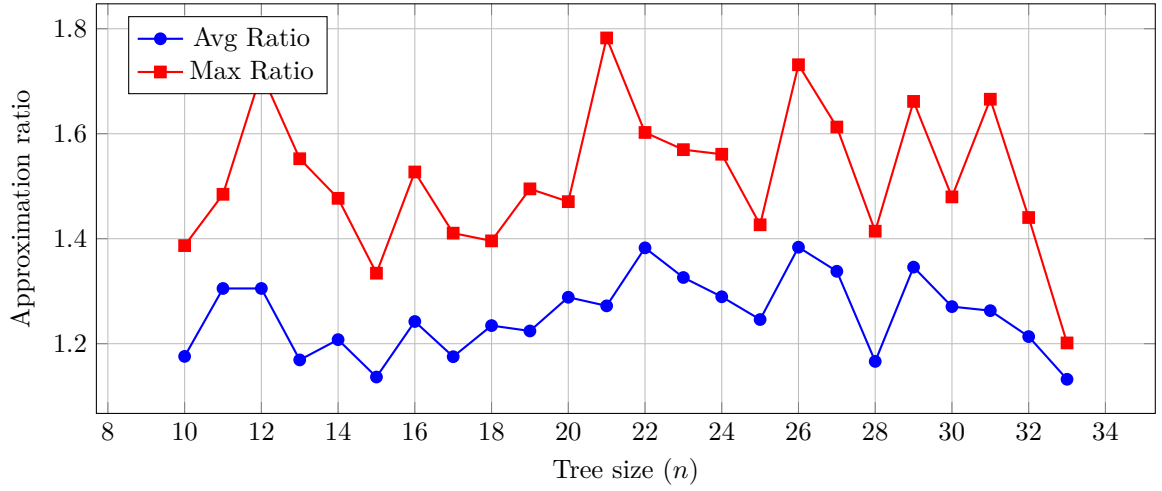


Figure 4.11: Random trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.10: Random trees with uniform cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	121.404046	756.719610	1.127822	1.319998
11	260.592758	2163.498792	1.228647	1.484599
12	5.358513	28.820250	1.243215	1.385444
13	11611.709343	116056.363491	1.195636	1.479760
14	3892.703673	34297.438517	1.256194	1.519283
15	4188.206773	19999.133534	1.365824	1.875606

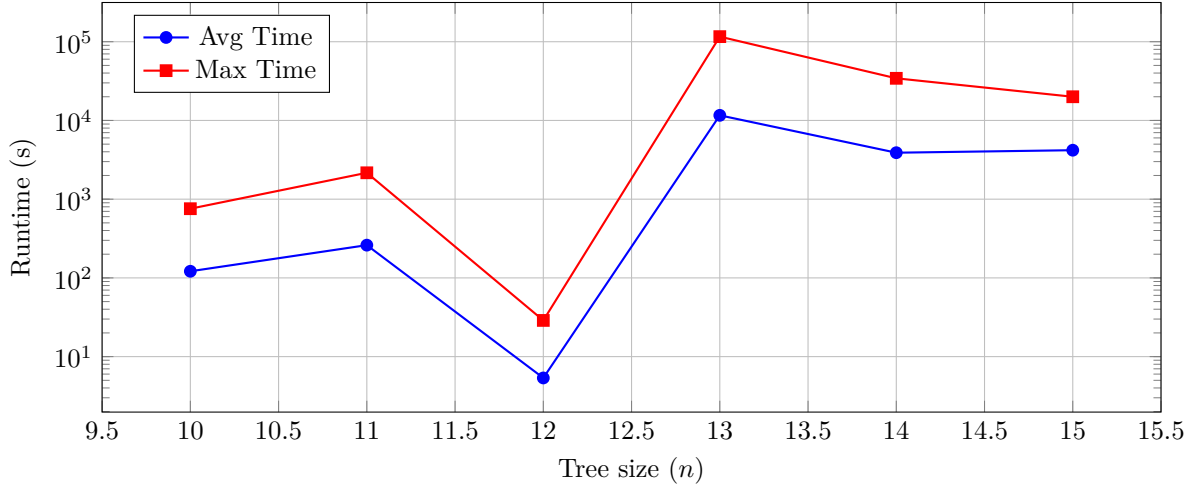


Figure 4.12: Random trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime

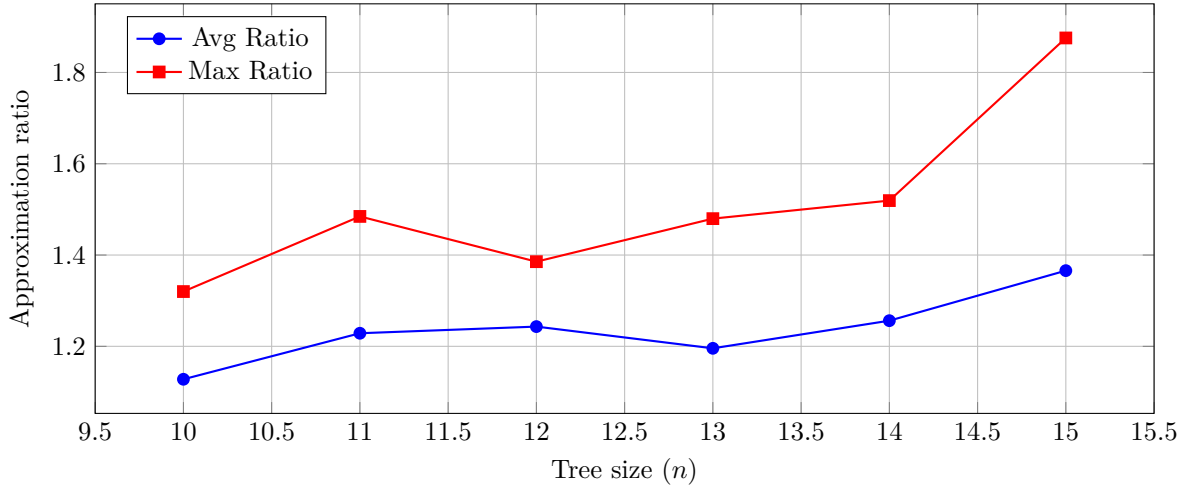


Figure 4.13: Random trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality

We have the following observations:

- We see that the running time of the exact procedure plotted on the logarithmic scale behaves as a linear function. This matches our expectation since it means that the running time is exponential in n . Notably, this running time is lower than the worst case running time of $O(2^n n)$, suggesting that for our instances the base of the exponential function representing the running time of the algorithm is smaller than 2.
- The approximation algorithms show a polynomial growth in running time, with the $O(\log n / \log \log n)$ -approximation algorithm being the fastest among them, followed by the $O(\sqrt{\log n})$ -approximation algorithm, and finally the $O(\log \log n)$ -approximation algorithm being the slowest.
- The approximation ratios achieved by all approximation algorithms are significantly better than their theoretical worst-case guarantees. All of the approximation ratios were below 2. We also see that no algorithm consistently outperformed the others.
- All of the above observations will also be prevalent among further experiments except when noted.

4.3.2 Random Trees with Binomial Cost Distribution ($T_R || V, c \sim \mathcal{B} || C_{max}$)

Exact Algorithm

Table 4.11: Random trees with binomial cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.556375	0.927114
11	0.988541	1.709204
12	2.284303	4.845210
13	4.457306	9.871454
14	6.074104	11.887673
15	10.764245	24.311487
16	10.557692	15.240707
17	28.881488	79.292225
18	30.493508	56.675099
19	56.074162	124.334870
20	107.637201	284.935946
21	101.401123	182.020454
22	225.660437	481.720823
23	327.566712	770.705944
24	697.875948	1585.505406
25	1073.534832	2295.687021
26	1525.110602	6012.207639
27	3166.287653	12571.961075
28	3275.602415	13551.198494
29	6842.866619	14913.855759
30	5809.227155	9002.505390

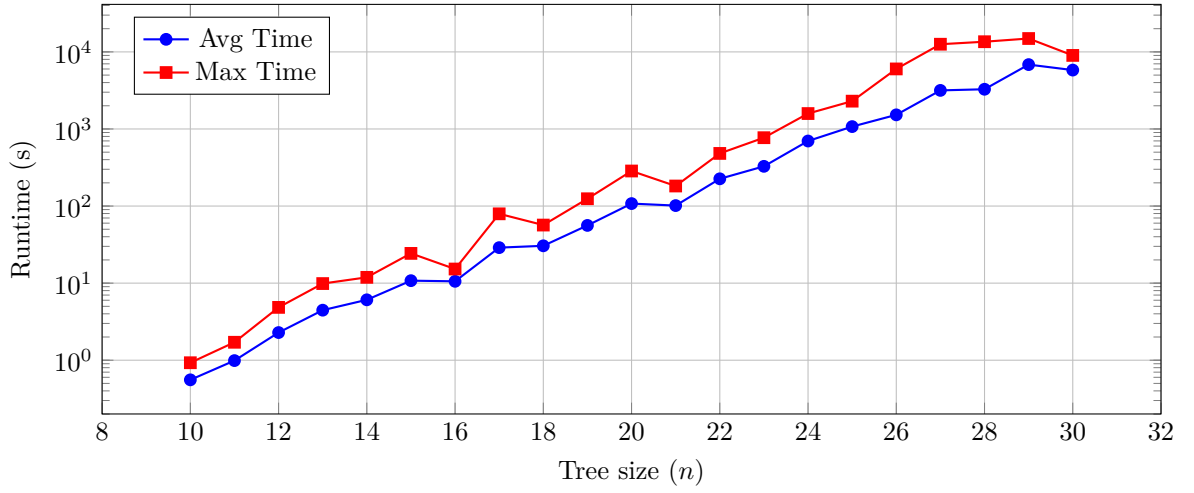


Figure 4.14: Random trees with binomial costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

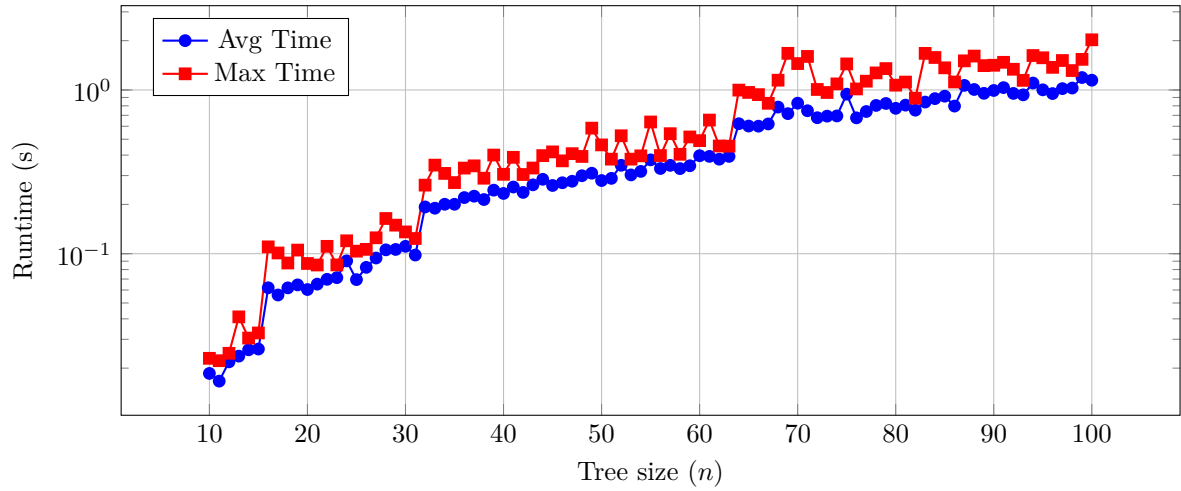


Figure 4.15: Random trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

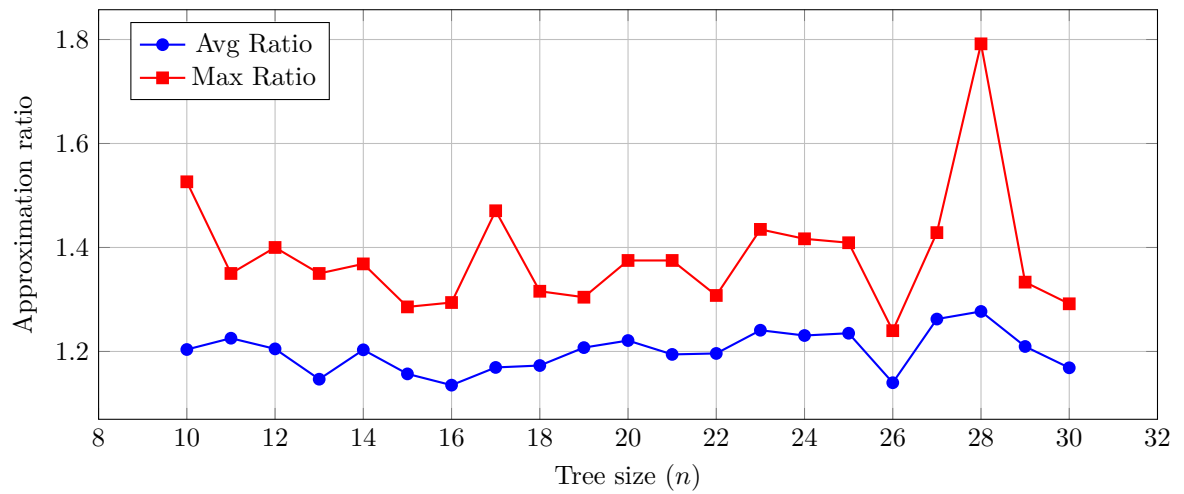


Figure 4.16: Random trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.12: Random trees with binomial cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.018569	0.022960	1.203734	1.526316
11	0.016663	0.022207	1.225465	1.350000
12	0.021850	0.024641	1.204960	1.400000
13	0.023680	0.041130	1.146632	1.350000
14	0.025879	0.030532	1.203148	1.368421
15	0.026162	0.032810	1.156865	1.285714
16	0.061919	0.110077	1.135182	1.294118
17	0.055949	0.101016	1.169257	1.470588
18	0.061837	0.087722	1.172924	1.315789
19	0.064491	0.105249	1.207410	1.304348
20	0.060418	0.087146	1.221015	1.375000
21	0.065168	0.085291	1.194317	1.375000
22	0.069730	0.110900	1.196165	1.307692
23	0.071404	0.085374	1.240822	1.434783
24	0.090381	0.119865	1.230664	1.416667
25	0.069459	0.103622	1.235042	1.409091
26	0.082531	0.106300	1.139867	1.240000
27	0.094121	0.125063	1.262302	1.428571
28	0.105554	0.164045	1.276963	1.791667
29	0.106147	0.149406	1.209574	1.333333
30	0.110902	0.135836	1.168583	1.291667
31	0.098121	0.123841	N/A	N/A
32	0.193028	0.262267	N/A	N/A
33	0.189587	0.347502	N/A	N/A
34	0.200383	0.309303	N/A	N/A
35	0.200387	0.271846	N/A	N/A
36	0.220161	0.333407	N/A	N/A
37	0.224506	0.344000	N/A	N/A
38	0.214219	0.289047	N/A	N/A
39	0.244387	0.400370	N/A	N/A
40	0.233333	0.305361	N/A	N/A
41	0.255417	0.387852	N/A	N/A
42	0.236747	0.304515	N/A	N/A
43	0.263362	0.333614	N/A	N/A
44	0.284715	0.397314	N/A	N/A
45	0.261028	0.418599	N/A	N/A
46	0.271188	0.369017	N/A	N/A
47	0.277001	0.408348	N/A	N/A
48	0.299343	0.392631	N/A	N/A
49	0.310086	0.584580	N/A	N/A
50	0.279730	0.461822	N/A	N/A
51	0.288490	0.377989	N/A	N/A
52	0.346704	0.525220	N/A	N/A
53	0.302892	0.378068	N/A	N/A
54	0.318150	0.395956	N/A	N/A
55	0.374280	0.637381	N/A	N/A
56	0.331411	0.397564	N/A	N/A
57	0.346383	0.540732	N/A	N/A
58	0.330891	0.404741	N/A	N/A
59	0.344452	0.516313	N/A	N/A
60	0.396750	0.490501	N/A	N/A
61	0.393049	0.654955	N/A	N/A
62	0.377914	0.454795	N/A	N/A
63	0.392999	0.453872	N/A	N/A
64	0.622009	0.998268	N/A	N/A
65	0.602131	0.965777	N/A	N/A
66	0.601055	0.936481	N/A	N/A
67	0.619979	0.829050	N/A	N/A
68	0.786046	1.147479	N/A	N/A
69	0.716865	1.673666	N/A	N/A
70	0.830285	1.449599	N/A	N/A
71	0.746811	1.602311	N/A	N/A
72	0.677003	1.006876	N/A	N/A
73	0.692125	0.965051	N/A	N/A
74	0.694072	1.089183	N/A	N/A
75	0.942688	1.442461	N/A	N/A
76	0.675616	1.014416	N/A	N/A
77	0.738686	1.131518	N/A	N/A
78	0.804489	1.271243	N/A	N/A
79	0.827115	1.350617	N/A	N/A
80	0.772503	1.069667	N/A	N/A
81	0.807888	1.117379	N/A	N/A
82	0.752628	0.891018	N/A	N/A
83	0.843558	1.672940	N/A	N/A
84	0.885294	1.578962	N/A	N/A
85	0.916309	1.365215	N/A	N/A
86	0.797230	1.118827	N/A	N/A
87	1.066447	1.505697	N/A	N/A
88	1.006799	1.610752	N/A	N/A
89	0.953852	1.408214	N/A	N/A
90	0.992794	1.415388	N/A	N/A
91	1.033525	1.475431	N/A	N/A
92	0.952505	1.338109	N/A	N/A
93	0.933517	1.144395	N/A	N/A
94	1.102392	1.622867	N/A	N/A
95	1.001937	1.572406	N/A	N/A
96	0.950718	1.374386	N/A	N/A
97	1.018995	1.512993	N/A	N/A
98	1.027694	1.313775	N/A	N/A
99	1.191126	1.537766	N/A	N/A
100	1.146860	2.024471	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.13: Random trees with binomial cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	2.766419	12.594700	1.292396	1.571429
11	2.086002	14.903139	1.265143	1.444444
12	2.153725	10.921436	1.287693	1.400000
13	2.613087	7.186258	1.213860	1.454545
14	5.149202	28.974054	1.207946	1.368421
15	1.701935	7.544428	1.254578	1.450000
16	31.955564	215.138099	1.178475	1.476190
17	173.457108	1603.917324	1.282322	1.666667
18	36.867875	74.429148	1.190714	1.315789
19	159.390315	980.178797	1.210639	1.304348
20	21.763832	51.343457	1.212980	1.450000
21	20.482378	69.922542	1.176545	1.375000
22	29.476656	73.267557	1.282130	1.416667
23	25.331472	54.049392	1.322905	1.523810
24	45.758897	226.747744	1.257332	1.375000
25	27.503702	90.087763	1.273956	1.409091
26	24.759023	70.813797	1.202938	1.407407
27	19.324728	75.035363	1.328577	1.500000
28	344.503286	2423.304454	1.343051	1.791667
29	28.559347	60.115264	1.293771	1.571429
30	42.816815	189.415031	1.198378	1.461538
31	24.652245	91.068090	N/A	N/A
32	42.166849	131.750438	N/A	N/A
33	182.292682	1195.174706	N/A	N/A
34	27.921171	61.951006	N/A	N/A
35	29.155326	148.524253	N/A	N/A
36	472.628700	3956.030065	N/A	N/A
37	49.684235	247.921100	N/A	N/A
38	39.706982	130.295335	N/A	N/A
39	4106.072874	40689.805987	N/A	N/A
40	78.306776	521.714127	N/A	N/A
41	441.544937	2842.636941	N/A	N/A
42	1985.783616	10832.178223	N/A	N/A
43	2032.209634	15993.999531	N/A	N/A
44	49843.050655	392703.720958	N/A	N/A
45	1756.696046	4066.276252	N/A	N/A
46	1637.604034	8195.422064	N/A	N/A
47	4518.566281	24557.124074	N/A	N/A
48	730.966484	2768.719063	N/A	N/A
49	1398.716203	7138.296084	N/A	N/A
50	1469.494227	3253.316360	N/A	N/A
51	6672.336573	44175.934440	N/A	N/A
52	2005.430994	5185.360214	N/A	N/A

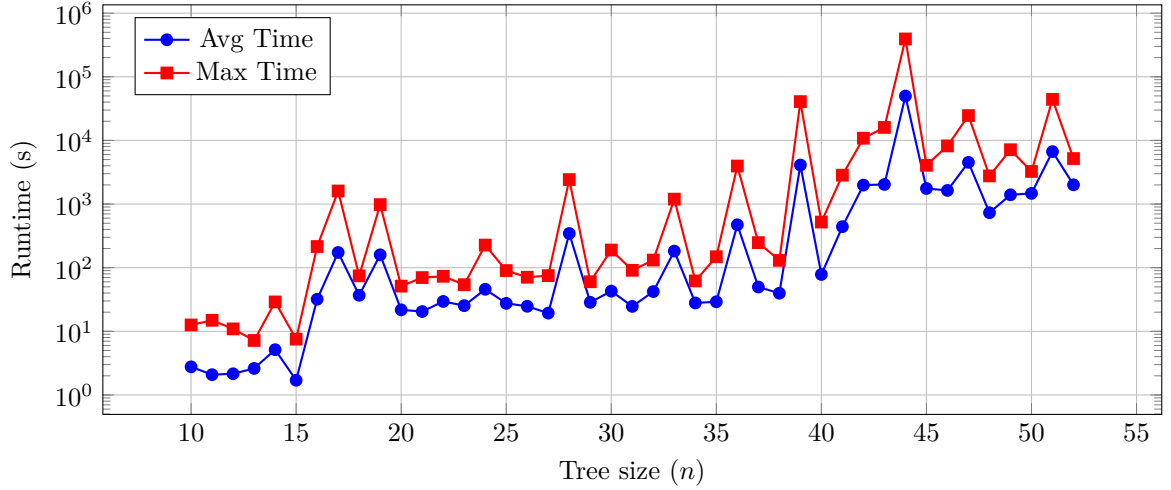


Figure 4.17: Random trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

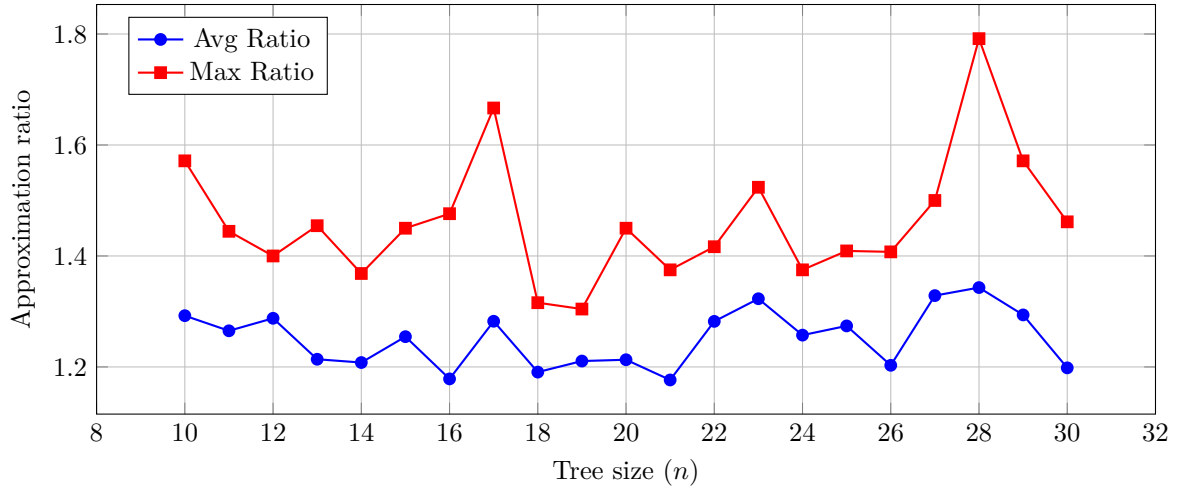


Figure 4.18: Random trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.14: Random trees with binomial cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	270.171032	1845.239705	1.242521	1.647059
11	273.610037	2390.518776	1.325272	1.888889
12	162.207568	958.610187	1.195159	1.400000
13	108.180016	731.949716	1.231742	1.380952
14	8520.517828	56561.592049	1.281385	1.736842
15	493.213468	1619.288856	1.297056	1.526316
16	369.386095	1452.884733	1.293589	1.428571

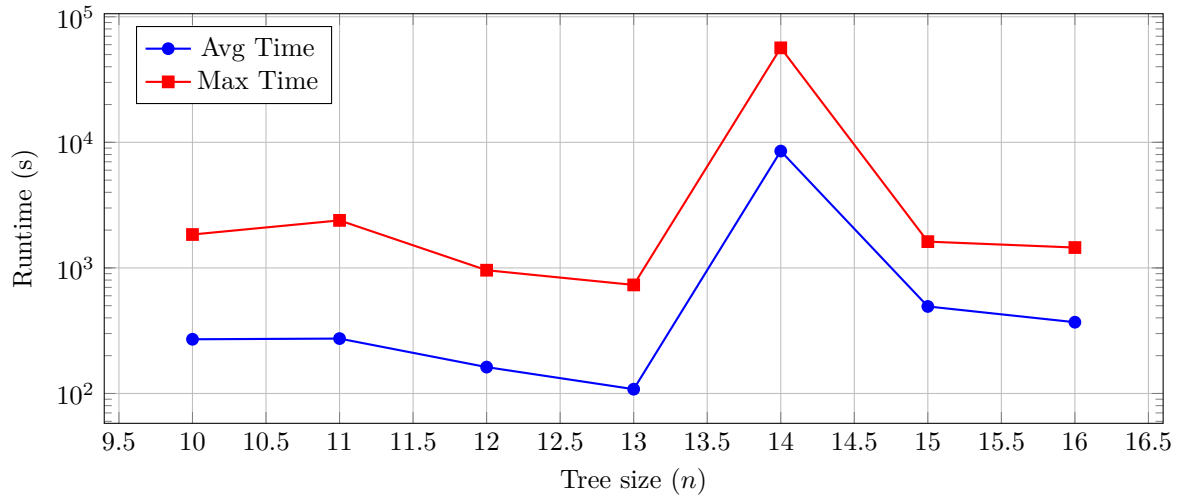


Figure 4.19: Random trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime

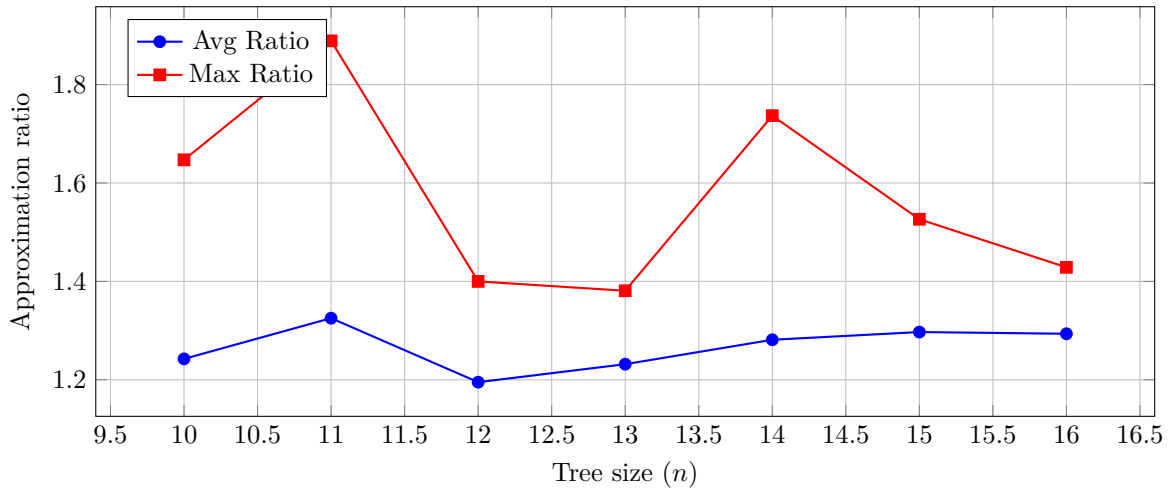


Figure 4.20: Random trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality

4.3.3 Random Trees with Exponential Cost Distribution ($T_R || V, c \sim \mathcal{E} || C_{max}$)

Exact Algorithm

Table 4.15: Random trees with exponential cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.397258	0.549761
11	0.881495	1.185577
12	1.160748	1.885142
13	1.826028	3.421310
14	4.676321	11.680625
15	7.653875	15.009777
16	12.748250	44.517421
17	23.283714	40.561355
18	21.641541	51.801028
19	62.381975	140.589124
20	101.497002	175.680065
21	131.955652	233.756469
22	163.721968	432.129969
23	180.511257	460.316967
24	734.623643	2883.680292
25	948.515655	2682.398239
26	1253.883607	2169.719410
27	2585.928873	7403.012652
28	1985.006110	3254.015142
29	5615.846933	11391.047865
30	13563.202799	29000.578610
31	16919.006757	38392.842757
32	25802.145074	69527.490872
33	6209.514245	6209.514245

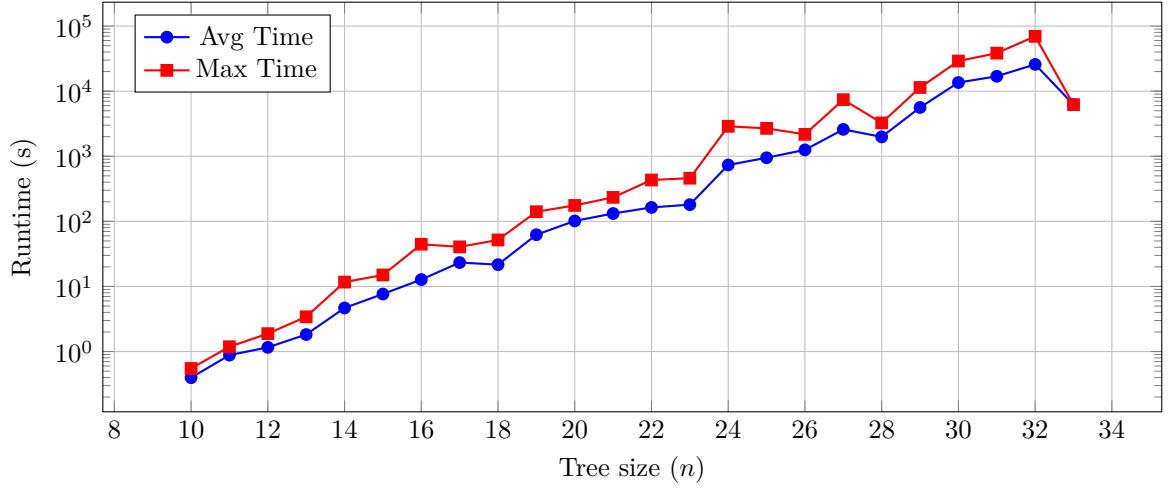


Figure 4.21: Random trees with exponential costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -**Approximation Algorithm**

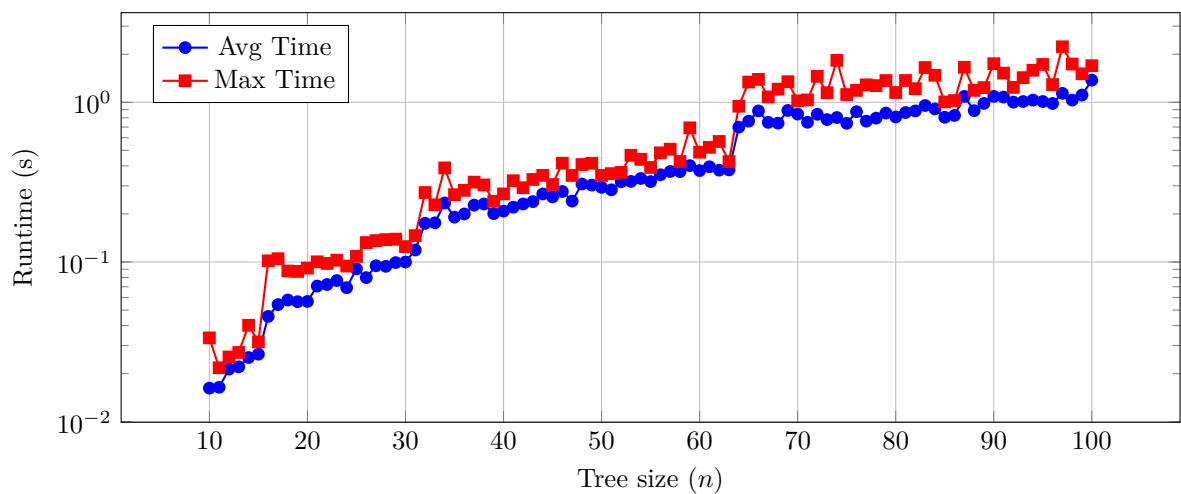


Figure 4.22: Random trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

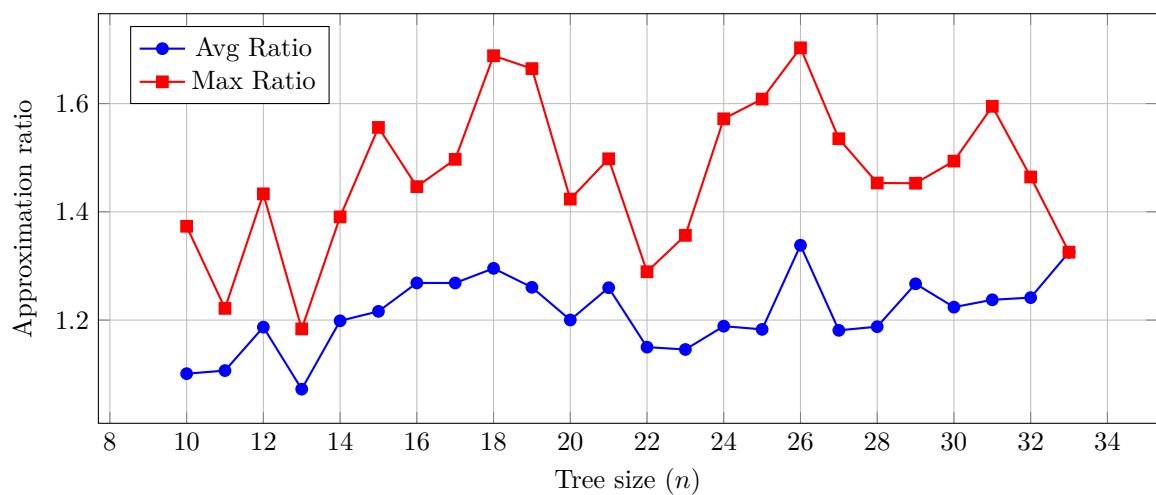


Figure 4.23: Random trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.16: Random trees with exponential cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.016237	0.033495	1.100966	1.373356
11	0.016435	0.021775	1.106673	1.221622
12	0.021375	0.025441	1.186964	1.433260
13	0.022067	0.027149	1.072361	1.183724
14	0.025236	0.040168	1.198633	1.390735
15	0.026477	0.031603	1.216127	1.555999
16	0.045667	0.101752	1.268623	1.446646
17	0.054146	0.104885	1.268534	1.497025
18	0.057846	0.087809	1.295600	1.688485
19	0.056449	0.087378	1.260531	1.664679
20	0.056650	0.091631	1.200211	1.423546
21	0.070721	0.100305	1.259849	1.498037
22	0.072349	0.097988	1.149977	1.289329
23	0.076577	0.102658	1.145669	1.356598
24	0.069105	0.094356	1.188728	1.571916
25	0.090613	0.108517	1.182837	1.608406
26	0.079870	0.132329	1.338275	1.703062
27	0.094799	0.136214	1.181060	1.535153
28	0.093992	0.138174	1.187824	1.453300
29	0.099203	0.139057	1.267033	1.452920
30	0.100028	0.124936	1.223907	1.493783
31	0.118946	0.146275	1.237473	1.594948
32	0.174762	0.272404	1.241535	1.464382
33	0.175950	0.228101	1.325479	1.325479
34	0.234167	0.388441	N/A	N/A
35	0.190970	0.264060	N/A	N/A
36	0.199970	0.281022	N/A	N/A
37	0.227082	0.316417	N/A	N/A
38	0.230939	0.303853	N/A	N/A
39	0.200918	0.240026	N/A	N/A
40	0.208437	0.267012	N/A	N/A
41	0.220411	0.322908	N/A	N/A
42	0.230934	0.292023	N/A	N/A
43	0.238974	0.328827	N/A	N/A
44	0.266803	0.348988	N/A	N/A
45	0.255801	0.305685	N/A	N/A
46	0.276369	0.415777	N/A	N/A
47	0.240607	0.348401	N/A	N/A
48	0.307847	0.407946	N/A	N/A
49	0.302979	0.415478	N/A	N/A
50	0.293423	0.348525	N/A	N/A
51	0.283124	0.357786	N/A	N/A
52	0.317979	0.364042	N/A	N/A
53	0.319823	0.465793	N/A	N/A
54	0.333592	0.439661	N/A	N/A
55	0.318824	0.390928	N/A	N/A
56	0.351867	0.482792	N/A	N/A
57	0.368880	0.508758	N/A	N/A
58	0.368294	0.426094	N/A	N/A
59	0.401352	0.691992	N/A	N/A
60	0.373618	0.487093	N/A	N/A
61	0.394903	0.522432	N/A	N/A
62	0.375968	0.568445	N/A	N/A
63	0.377460	0.425218	N/A	N/A
64	0.699574	0.944739	N/A	N/A
65	0.761989	1.339043	N/A	N/A
66	0.883377	1.391986	N/A	N/A
67	0.748847	1.081355	N/A	N/A
68	0.740471	1.208873	N/A	N/A
69	0.889268	1.346342	N/A	N/A
70	0.843911	1.021090	N/A	N/A
71	0.749992	1.034903	N/A	N/A
72	0.841749	1.453867	N/A	N/A
73	0.779666	1.148637	N/A	N/A
74	0.803237	1.831655	N/A	N/A
75	0.738934	1.118207	N/A	N/A
76	0.873154	1.188425	N/A	N/A
77	0.761878	1.285886	N/A	N/A
78	0.796022	1.271787	N/A	N/A
79	0.856574	1.368408	N/A	N/A
80	0.809499	1.151806	N/A	N/A
81	0.863389	1.369387	N/A	N/A
82	0.883448	1.217195	N/A	N/A
83	0.954642	1.651388	N/A	N/A
84	0.908694	1.476544	N/A	N/A
85	0.805750	1.005292	N/A	N/A
86	0.827420	1.026068	N/A	N/A
87	1.089388	1.657093	N/A	N/A
88	0.886662	1.187690	N/A	N/A
89	0.984090	1.236690	N/A	N/A
90	1.089562	1.745053	N/A	N/A
91	1.078212	1.523559	N/A	N/A
92	1.002155	1.240874	N/A	N/A
93	1.011360	1.427223	N/A	N/A
94	1.032717	1.587759	N/A	N/A
95	1.011799	1.726225	N/A	N/A
96	0.983281	1.291645	N/A	N/A
97	1.137877	2.226232	N/A	N/A
98	1.032485	1.737016	N/A	N/A
99	1.111267	1.508560	N/A	N/A
100	1.376897	1.696145	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.17: Random trees with exponential cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	3.271125	17.785696	1.188598	1.623924
11	3.629943	20.215514	1.107890	1.239986
12	0.136612	0.703763	1.213336	1.595378
13	5.155277	26.768550	1.161811	1.518507
14	5.674060	27.903470	1.212446	1.397901
15	3.337283	15.047687	1.172650	1.433563
16	21.073328	85.748191	1.238863	1.529049
17	25.494491	165.556898	1.214376	1.582861
18	99.869352	406.402671	1.277778	1.553406
19	19.421447	69.304303	1.254327	1.843616
20	22.498086	106.415632	1.221528	1.402355
21	23.358083	72.395053	1.324501	1.669538
22	37.254589	192.448365	1.216357	1.623233
23	66.243801	206.180124	1.221255	1.773604
24	19.672028	104.269739	1.306232	1.571916
25	98.736290	355.952677	1.266486	1.749569
26	56.177485	202.758462	1.370396	1.985201
27	49.095161	221.547670	1.135261	1.346807
28	29.204365	141.400477	1.235612	1.754597
29	26.653767	131.919711	1.262877	1.494846
30	40.418576	102.952930	1.191372	1.459335
31	41.726362	116.834656	1.305434	1.583043
32	18.009437	57.953526	1.210094	1.366504
33	18.721550	65.473680	1.507570	1.507570
34	30.534315	60.208069	N/A	N/A
35	11.796997	24.369075	N/A	N/A
36	32.693106	127.268339	N/A	N/A
37	28.107762	69.320070	N/A	N/A
38	74.705023	297.670002	N/A	N/A
39	28.753502	107.382334	N/A	N/A
40	40.023030	113.296597	N/A	N/A
41	44.548128	84.873195	N/A	N/A
42	617.346981	1712.780073	N/A	N/A
43	3343.745612	25313.002871	N/A	N/A
44	526.433698	833.764386	N/A	N/A
45	1025.802671	2694.859194	N/A	N/A
46	376.982329	798.947795	N/A	N/A
47	182.393146	683.893516	N/A	N/A
48	7359.758340	35448.198180	N/A	N/A
49	2234.354507	8378.426965	N/A	N/A
50	3424.428952	13372.659564	N/A	N/A
51	603.334160	1764.191564	N/A	N/A
52	374.618495	682.347076	N/A	N/A
53	412.175755	1087.737078	N/A	N/A
54	11327.159077	44777.889020	N/A	N/A

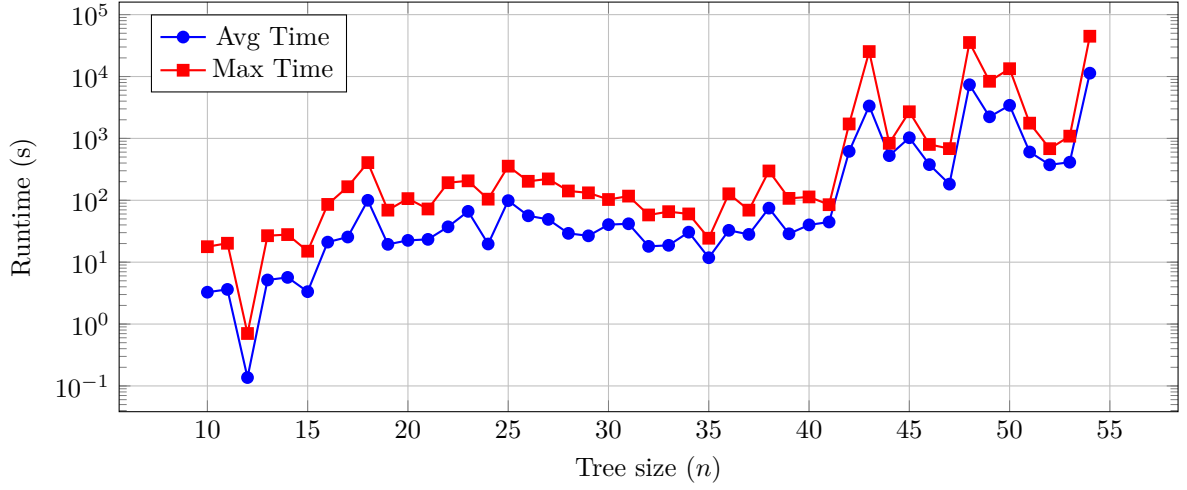


Figure 4.24: Random trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

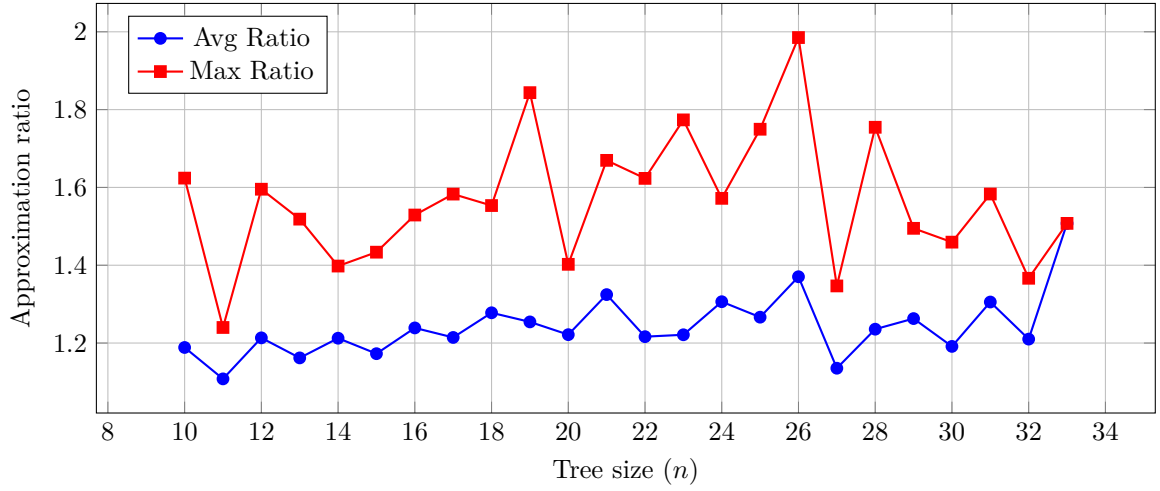


Figure 4.25: Random trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.18: Random trees with exponential cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	2.443456	22.215506	1.363845	1.932925
11	6.154900	57.546679	1.293482	1.512384
12	1151.116577	9780.632542	1.253727	1.674094
13	14.196982	85.125569	1.344387	1.728937
14	198.175667	1789.909311	1.482452	1.941569
15	5862.478971	46840.667775	1.235591	1.615380
16	318.129038	3075.309871	1.332374	1.729848
17	3.516439	21.537107	1.847921	2.528925
18	669.176482	5233.473255	1.829482	2.322135
19	6.937829	25.825382	1.852755	2.402326
20	167.503671	1599.696498	1.604565	2.177329
21	638.992803	5089.416766	1.616518	2.146588
22	152.358166	1272.209907	1.592249	2.216253
23	229.841830	1240.158336	1.621922	2.538157

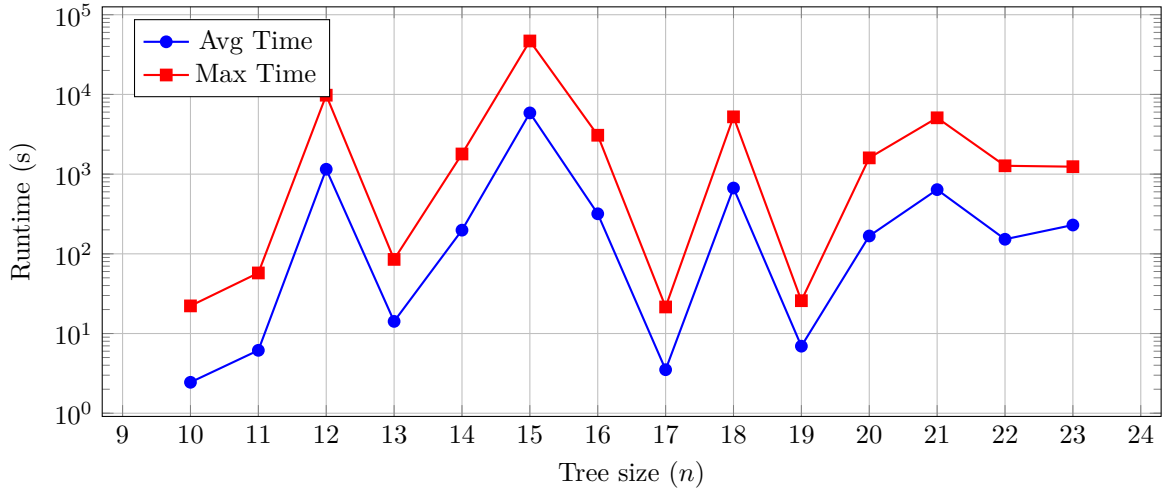


Figure 4.26: Random trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime

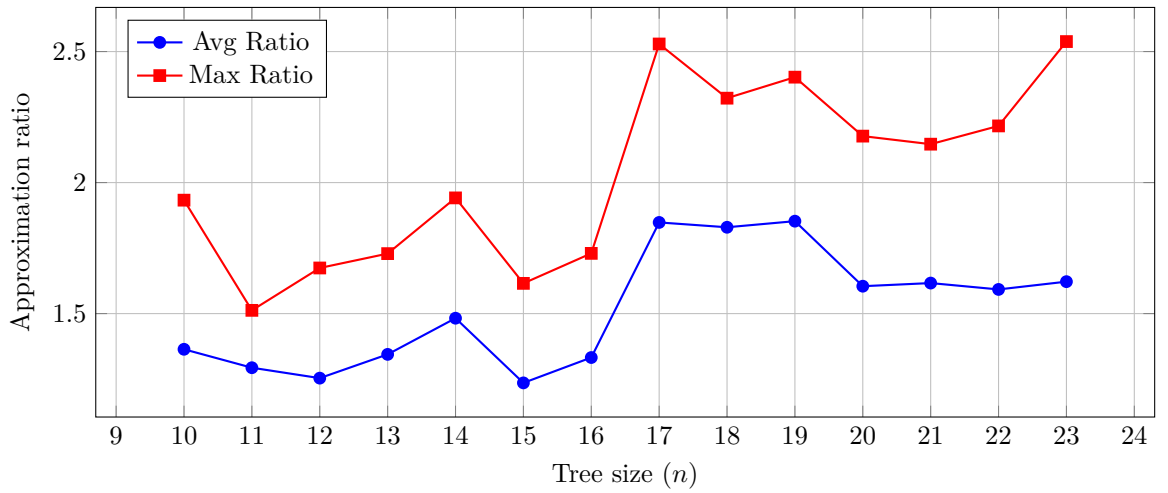


Figure 4.27: Random trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality

We observe that this time the $O(\log \log n)$ -approximation algorithm encountered instances for which it produced decision tree with approximation ratio above 2.5. Therefore we conclude that its performances was the worst among the three algorithms. The $O(\log n / \log \log n)$ -approximation algorithm achieved the best worst approximation ratios while the $O(\sqrt{\log n})$ -approximation algorithm achieved the best average approximation ratios.

4.3.4 Random bounded degree trees with Uniform Cost Distribution ($T_\Delta || V, c \sim \mathcal{U} || C_{max}$)

Exact Algorithm

Table 4.19: Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.569462	0.910251
11	0.947969	1.075459
12	1.732044	2.264132
13	3.526699	5.086343
14	5.781680	8.666455
15	8.839317	13.067036
16	16.529754	30.454387
17	23.270885	30.579978
18	37.895584	53.362680
19	74.208207	107.442290
20	103.980832	189.775445
21	173.465774	281.796656
22	284.843597	428.580711
23	455.351876	665.295764
24	778.535390	1164.126190
25	973.073825	1558.066491
26	1306.403881	2106.394819
27	2427.387178	3590.241069
28	4807.033953	7775.834615
29	7303.134967	12184.796961
30	9092.114544	14593.088346
31	11997.522372	17038.681034

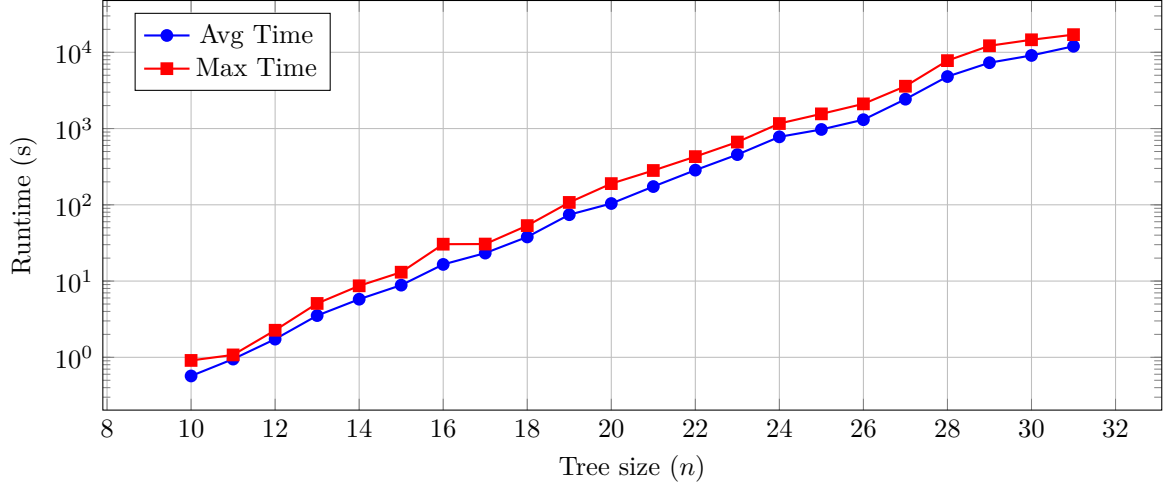


Figure 4.28: Random bounded degree trees with uniform costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

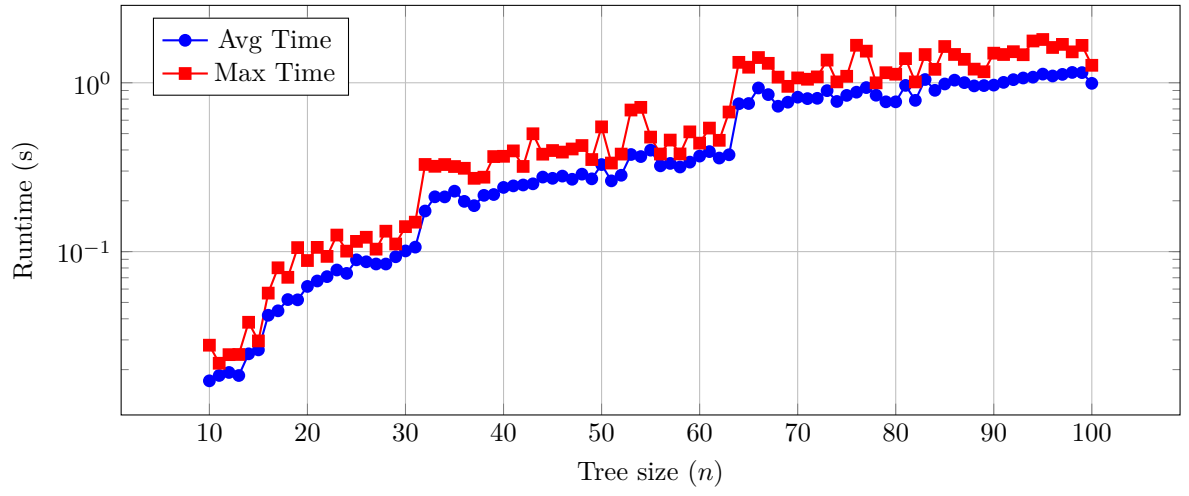


Figure 4.29: Random bounded degree trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

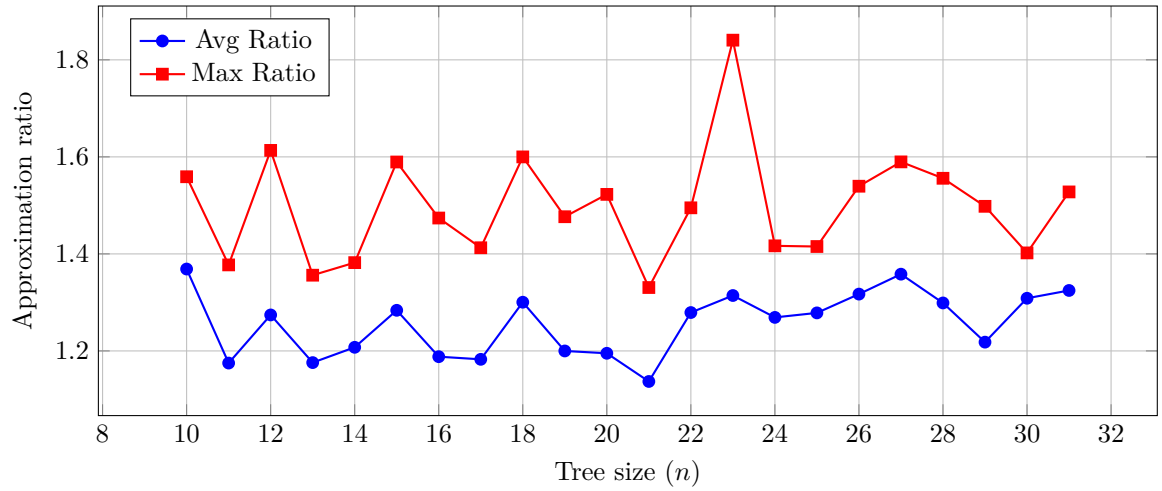


Figure 4.30: Random bounded degree trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.20: Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.017166	0.027907	1.368812	1.559049
11	0.018470	0.021815	1.174985	1.377327
12	0.019244	0.024530	1.274171	1.613368
13	0.018459	0.024588	1.175936	1.356112
14	0.024805	0.038152	1.207483	1.382026
15	0.026180	0.029580	1.283730	1.589454
16	0.041975	0.056785	1.188146	1.473966
17	0.044602	0.080185	1.182686	1.412676
18	0.051988	0.070449	1.300411	1.599961
19	0.051808	0.105617	1.199933	1.476634
20	0.062203	0.088529	1.195159	1.522695
21	0.066990	0.105946	1.137052	1.330699
22	0.071099	0.093698	1.279132	1.495007
23	0.077821	0.125311	1.314228	1.840609
24	0.074299	0.100649	1.269120	1.416625
25	0.089388	0.114975	1.278403	1.415222
26	0.087034	0.121600	1.317146	1.539522
27	0.084461	0.103423	1.358201	1.589700
28	0.084440	0.132099	1.298890	1.555871
29	0.093223	0.110944	1.218117	1.498083
30	0.100934	0.140560	1.308573	1.402066
31	0.106293	0.149435	1.324647	1.527776
32	0.173953	0.328196	N/A	N/A
33	0.211144	0.320116	N/A	N/A
34	0.210901	0.327839	N/A	N/A
35	0.227983	0.319118	N/A	N/A
36	0.198423	0.311573	N/A	N/A
37	0.187135	0.271733	N/A	N/A
38	0.215414	0.275898	N/A	N/A
39	0.217824	0.365530	N/A	N/A
40	0.239979	0.366855	N/A	N/A
41	0.245061	0.394990	N/A	N/A
42	0.248074	0.319521	N/A	N/A
43	0.252062	0.498858	N/A	N/A
44	0.276345	0.377613	N/A	N/A
45	0.271678	0.396937	N/A	N/A
46	0.280002	0.388635	N/A	N/A
47	0.268197	0.405340	N/A	N/A
48	0.287586	0.424867	N/A	N/A
49	0.269772	0.350966	N/A	N/A
50	0.327721	0.548053	N/A	N/A
51	0.262446	0.334615	N/A	N/A
52	0.283252	0.378114	N/A	N/A
53	0.375972	0.689571	N/A	N/A
54	0.365545	0.715186	N/A	N/A
55	0.398877	0.476710	N/A	N/A
56	0.322145	0.378468	N/A	N/A
57	0.332909	0.457751	N/A	N/A
58	0.317050	0.378917	N/A	N/A
59	0.338804	0.511861	N/A	N/A
60	0.367850	0.439049	N/A	N/A
61	0.391132	0.538999	N/A	N/A
62	0.358201	0.456246	N/A	N/A
63	0.374623	0.670986	N/A	N/A
64	0.751215	1.321225	N/A	N/A
65	0.753726	1.236273	N/A	N/A
66	0.931557	1.410607	N/A	N/A
67	0.850978	1.303619	N/A	N/A
68	0.726148	1.079939	N/A	N/A
69	0.767336	0.952349	N/A	N/A
70	0.821550	1.068516	N/A	N/A
71	0.805728	1.049320	N/A	N/A
72	0.808964	1.083351	N/A	N/A
73	0.897976	1.362154	N/A	N/A
74	0.775190	1.012238	N/A	N/A
75	0.841754	1.092495	N/A	N/A
76	0.879476	1.667701	N/A	N/A
77	0.936425	1.539246	N/A	N/A
78	0.843668	0.999127	N/A	N/A
79	0.771374	1.146962	N/A	N/A
80	0.771215	1.123649	N/A	N/A
81	0.964899	1.390082	N/A	N/A
82	0.786860	1.011433	N/A	N/A
83	1.045258	1.469303	N/A	N/A
84	0.902063	1.201233	N/A	N/A
85	0.983766	1.639589	N/A	N/A
86	1.036127	1.471001	N/A	N/A
87	1.001877	1.376682	N/A	N/A
88	0.958278	1.204494	N/A	N/A
89	0.962137	1.162254	N/A	N/A
90	0.968600	1.497351	N/A	N/A
91	1.005362	1.470145	N/A	N/A
92	1.043300	1.527753	N/A	N/A
93	1.067345	1.466230	N/A	N/A
94	1.079613	1.767928	N/A	N/A
95	1.125522	1.804149	N/A	N/A
96	1.096971	1.618471	N/A	N/A
97	1.121245	1.687889	N/A	N/A
98	1.151274	1.524579	N/A	N/A
99	1.148657	1.665216	N/A	N/A
100	0.992137	1.269119	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.21: Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	6.745152	18.628860	1.323526	1.470442
11	0.891391	4.820290	1.358930	1.801952
12	4.650256	18.327222	1.289143	1.527993
13	1.092786	4.693858	1.236346	1.444811
14	4.430810	23.718724	1.310832	1.581653
15	5.028692	20.611170	1.249988	1.589454
16	45.812197	274.837986	1.220340	1.381102
17	10.562579	45.228879	1.183053	1.412676
18	72.619028	424.667823	1.369570	1.677322
19	59.608193	184.134521	1.226645	1.476634
20	145.359836	1086.565355	1.241962	1.678557
21	54.870730	242.676276	1.202372	1.474274
22	18.025357	54.088919	1.312068	1.541131
23	142.777999	909.819114	1.335362	1.515812
24	30.177858	94.455382	1.287965	1.526522
25	213.959363	961.508227	1.299650	1.616968
26	83.539806	279.978287	1.301243	1.539522
27	27.982172	55.013098	1.356035	1.877077
28	311.558299	1681.356424	1.378844	1.797100
29	204.036566	790.608369	1.227950	1.407721
30	265.362686	1567.399140	1.369537	1.528702
31	60.646072	156.607981	1.289125	1.422346
32	65.553018	197.967450	N/A	N/A
33	206.584464	1494.995737	N/A	N/A
34	59.792210	273.676460	N/A	N/A
35	366.157904	2065.520392	N/A	N/A
36	216.624027	1518.987151	N/A	N/A
37	87.541615	236.389548	N/A	N/A
38	57.479844	167.405367	N/A	N/A
39	742.314960	3081.102083	N/A	N/A
40	309.660167	1216.496151	N/A	N/A
41	78.646069	202.845554	N/A	N/A
42	3311.893001	21075.958122	N/A	N/A
43	5813.035363	23120.930214	N/A	N/A
44	919.626538	2211.069650	N/A	N/A
45	1200.194876	5110.575183	N/A	N/A
46	2303.212337	11083.558796	N/A	N/A
47	14020.712690	100646.646954	N/A	N/A
48	776.539038	3377.812620	N/A	N/A
49	881.669483	2599.530711	N/A	N/A

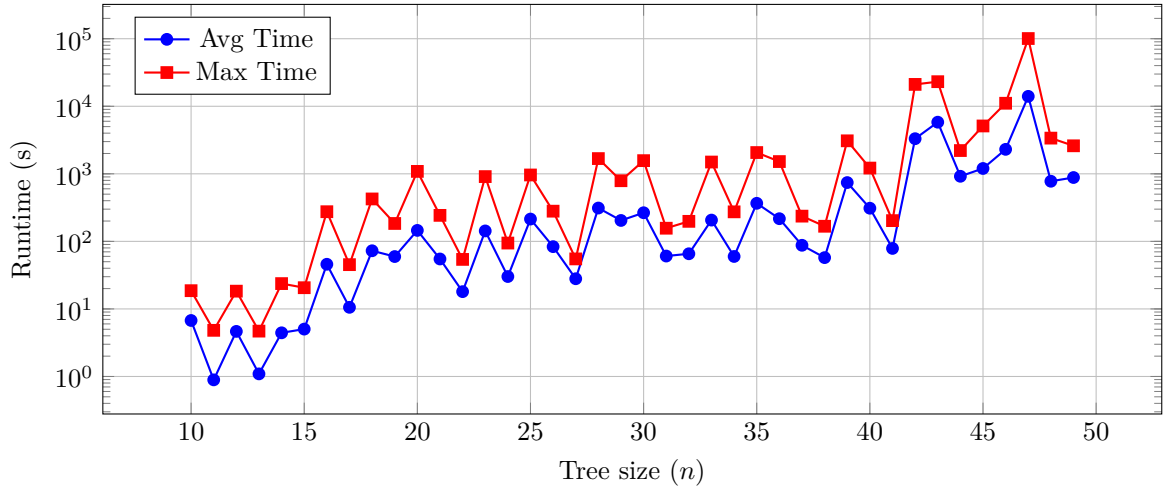


Figure 4.31: Random bounded degree trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

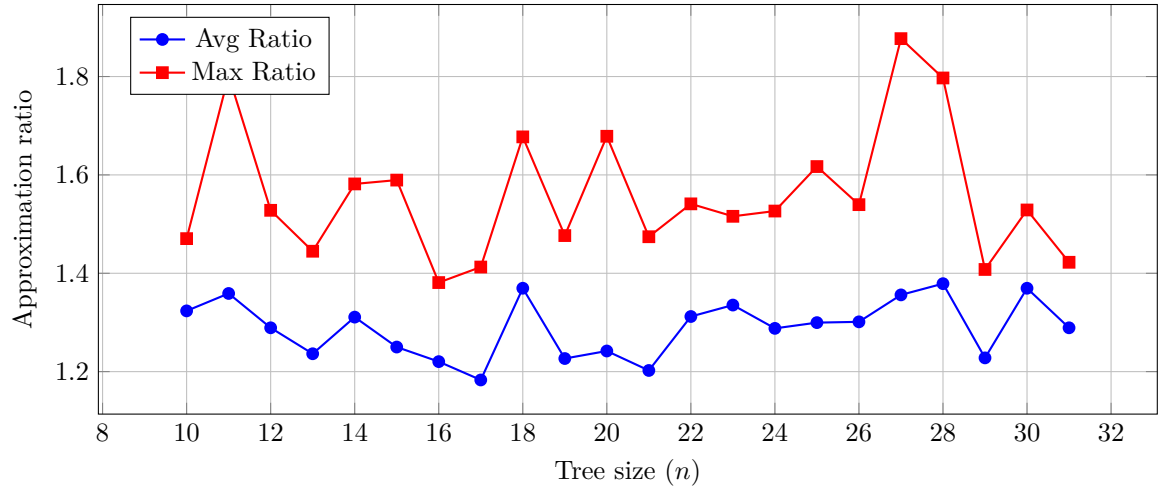


Figure 4.32: Random bounded degree trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.22: Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	22.871485	70.073557	1.248729	1.569975
11	102.598781	606.783683	1.199130	1.752944
12	73.527674	370.151464	1.194871	1.365568

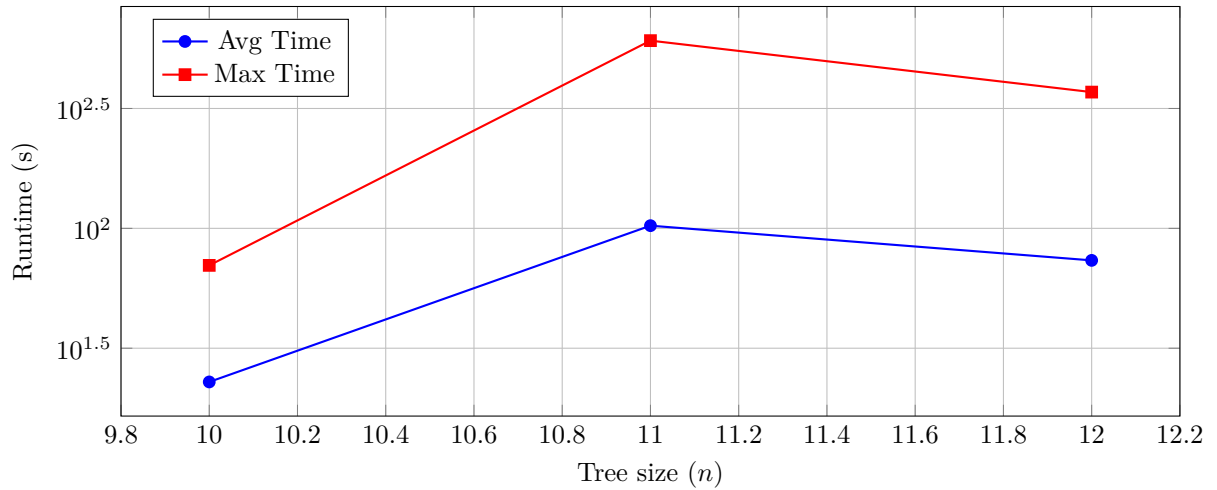


Figure 4.33: Random bounded degree trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime

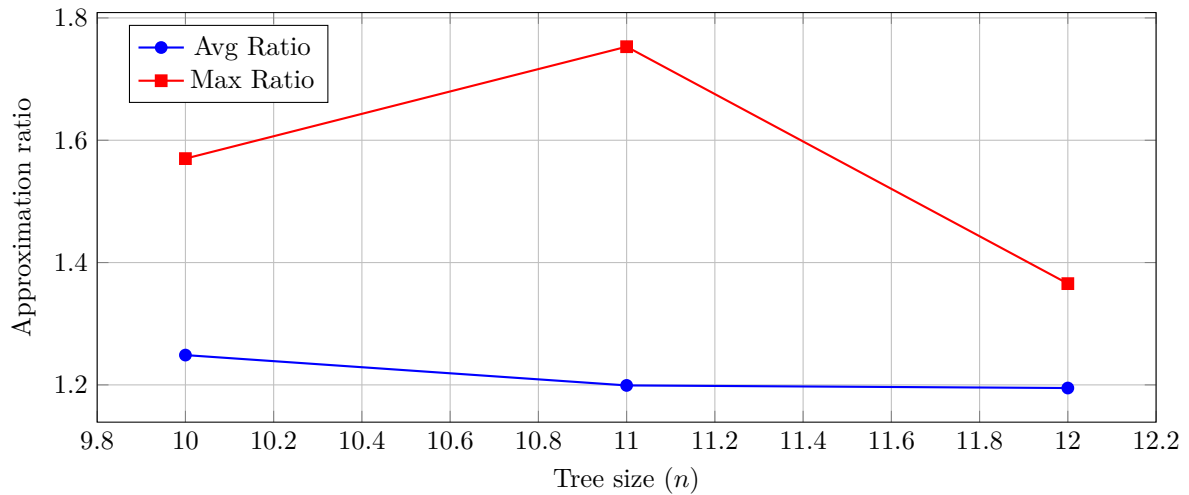


Figure 4.34: Random bounded degree trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality

We observe that:

- The running time of the exact procedure almost strictly follows a linear function on the logarithmic plot. This will also be the case for the other cost distributions for bounded degree instances.
- The $O(\log \log n)$ -approximation algorithm only managed to process instances up to 12 vertices. We have observed that for the first instance having size $n = 13$ which the algorithm encountered, the processing was not completed despite few days of computation. This suggests that the running time of the algorithm is unstable and largely dependent on the input structure.

4.3.5 Random bounded degree trees with Binomial Cost Distribution ($T_\Delta || V, c \sim \mathcal{B} || C_{max}$)

Exact Algorithm

Table 4.23: Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.679129	0.866329
11	1.143846	1.727530
12	2.108556	2.846186
13	2.877116	4.610394
14	5.186298	7.232912
15	11.089588	15.481051
16	17.453376	25.669113
17	22.652498	38.341589
18	34.485252	53.082858
19	64.104993	113.746068
20	109.923283	164.851702
21	147.998538	210.990652
22	199.918885	321.138025
23	385.391925	641.146414
24	575.128529	1069.366728
25	1024.386387	1421.913858
26	1842.750499	2564.884152
27	2745.826810	4357.703549
28	3660.689595	6702.747721
29	4615.927124	8596.952245
30	7846.694749	13389.098644
31	15664.366669	23356.570227

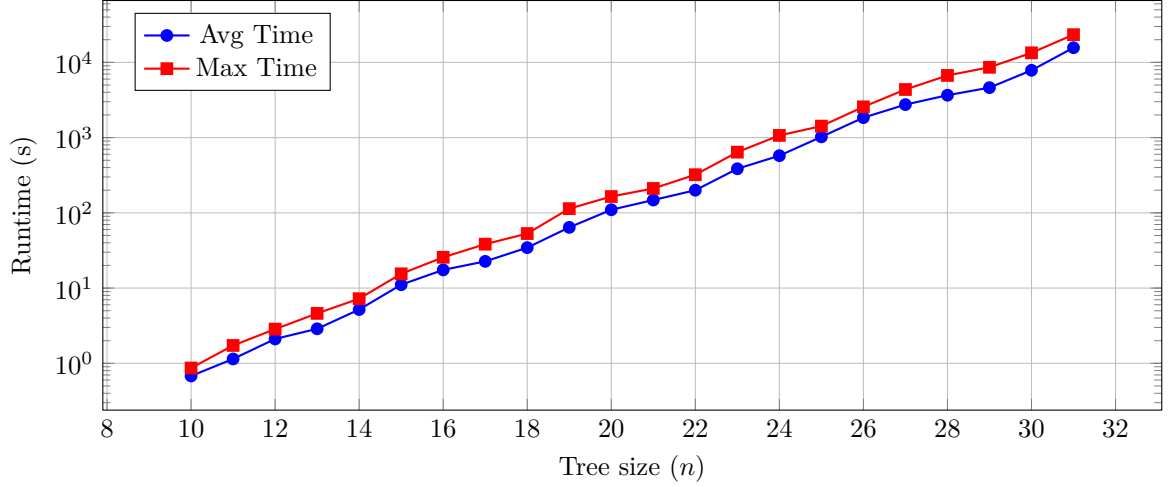


Figure 4.35: Random bounded degree trees with binomial costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

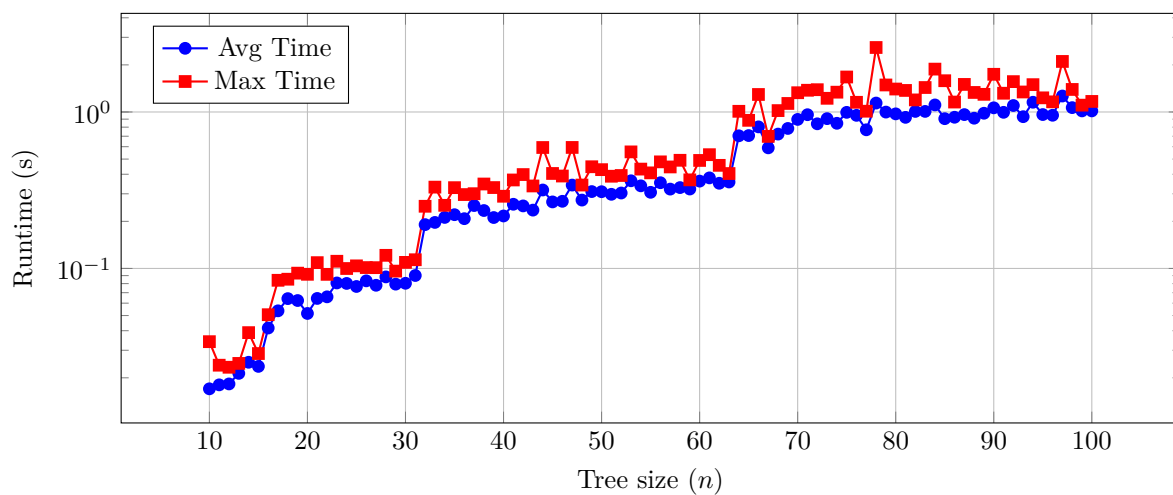


Figure 4.36: Random bounded degree trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

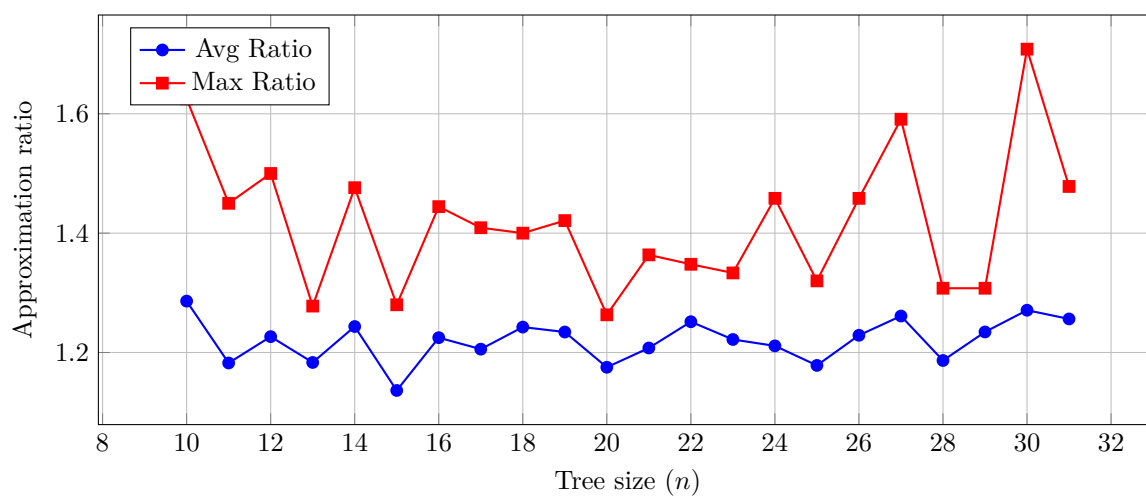


Figure 4.37: Random bounded degree trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.24: Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.017010	0.034047	1.286123	1.625000
11	0.018031	0.024078	1.182418	1.450000
12	0.018282	0.023362	1.226581	1.500000
13	0.021353	0.024717	1.183318	1.277778
14	0.025141	0.038894	1.243542	1.476190
15	0.023657	0.028585	1.136385	1.280000
16	0.041620	0.050626	1.224878	1.444444
17	0.053600	0.084009	1.205608	1.409091
18	0.064152	0.085343	1.242597	1.400000
19	0.062373	0.093314	1.234314	1.421053
20	0.051564	0.091749	1.175276	1.263158
21	0.064321	0.108928	1.207572	1.363636
22	0.065854	0.091667	1.251561	1.347826
23	0.080788	0.111162	1.221814	1.333333
24	0.080213	0.099913	1.211115	1.458333
25	0.076718	0.103894	1.178368	1.320000
26	0.083322	0.101307	1.228844	1.458333
27	0.078075	0.101170	1.261121	1.590909
28	0.088313	0.121297	1.186598	1.307692
29	0.079401	0.096160	1.234421	1.307692
30	0.080396	0.109344	1.270855	1.708333
31	0.090139	0.113654	1.256248	1.478261
32	0.190601	0.249693	N/A	N/A
33	0.196904	0.330911	N/A	N/A
34	0.211871	0.253161	N/A	N/A
35	0.220586	0.328397	N/A	N/A
36	0.208040	0.296657	N/A	N/A
37	0.252311	0.300487	N/A	N/A
38	0.234527	0.346495	N/A	N/A
39	0.211554	0.328821	N/A	N/A
40	0.216473	0.289366	N/A	N/A
41	0.257157	0.367713	N/A	N/A
42	0.251007	0.398224	N/A	N/A
43	0.235938	0.336674	N/A	N/A
44	0.317260	0.592966	N/A	N/A
45	0.266239	0.404586	N/A	N/A
46	0.268650	0.389739	N/A	N/A
47	0.341098	0.593399	N/A	N/A
48	0.273542	0.341317	N/A	N/A
49	0.309979	0.446628	N/A	N/A
50	0.309339	0.428021	N/A	N/A
51	0.297780	0.388546	N/A	N/A
52	0.303485	0.392715	N/A	N/A
53	0.363381	0.555904	N/A	N/A
54	0.337310	0.431521	N/A	N/A
55	0.306649	0.408266	N/A	N/A
56	0.353118	0.480001	N/A	N/A
57	0.321475	0.445371	N/A	N/A
58	0.328714	0.491233	N/A	N/A
59	0.321872	0.368395	N/A	N/A
60	0.361589	0.490116	N/A	N/A
61	0.379219	0.533466	N/A	N/A
62	0.350365	0.456023	N/A	N/A
63	0.356876	0.404184	N/A	N/A
64	0.704425	1.010076	N/A	N/A
65	0.706791	0.884507	N/A	N/A
66	0.803236	1.293810	N/A	N/A
67	0.589984	0.698261	N/A	N/A
68	0.722434	1.020688	N/A	N/A
69	0.784883	1.133728	N/A	N/A
70	0.896407	1.327752	N/A	N/A
71	0.960725	1.377013	N/A	N/A
72	0.840489	1.388138	N/A	N/A
73	0.906631	1.223128	N/A	N/A
74	0.846666	1.338480	N/A	N/A
75	0.994567	1.673440	N/A	N/A
76	0.951723	1.153210	N/A	N/A
77	0.769595	1.011385	N/A	N/A
78	1.140526	2.583047	N/A	N/A
79	0.998550	1.488157	N/A	N/A
80	0.972702	1.401942	N/A	N/A
81	0.924094	1.373113	N/A	N/A
82	1.008332	1.194611	N/A	N/A
83	1.008666	1.433334	N/A	N/A
84	1.109320	1.879651	N/A	N/A
85	0.906292	1.584191	N/A	N/A
86	0.925392	1.157595	N/A	N/A
87	0.962261	1.500324	N/A	N/A
88	0.912527	1.332998	N/A	N/A
89	0.981671	1.296804	N/A	N/A
90	1.063236	1.741295	N/A	N/A
91	0.995132	1.317830	N/A	N/A
92	1.099776	1.563496	N/A	N/A
93	0.933313	1.342385	N/A	N/A
94	1.154195	1.495306	N/A	N/A
95	0.962875	1.231455	N/A	N/A
96	0.952983	1.160954	N/A	N/A
97	1.264992	2.103911	N/A	N/A
98	1.066243	1.393220	N/A	N/A
99	1.016124	1.105883	N/A	N/A
100	1.016175	1.168020	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.25: Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	2.707926	9.689949	1.292343	1.625000
11	3.878902	13.020251	1.232017	1.571429
12	2.915433	7.344573	1.175265	1.500000
13	2.494348	7.083129	1.295623	1.555556
14	3.171852	8.626175	1.293074	1.588235
15	4.012140	13.429746	1.224170	1.333333
16	46.115378	339.579249	1.241891	1.363636
17	18.898404	66.633003	1.196012	1.409091
18	36.211295	117.974062	1.218926	1.400000
19	193.736056	839.828459	1.294498	1.521739
20	24.509002	104.504193	1.182370	1.421053
21	113.817647	717.533866	1.303746	1.520000
22	158.541345	632.801999	1.236339	1.333333
23	201.467341	900.952548	1.295351	1.480000
24	74.417159	296.879322	1.241359	1.666667
25	158.582493	750.312824	1.274272	1.440000
26	26.055754	83.299642	1.216198	1.363636
27	203.830920	637.692105	1.260189	1.384615
28	154.874237	941.820300	1.210050	1.391304
29	76.529289	377.662068	1.278163	1.518519
30	57.540110	170.539340	1.247444	1.370370
31	52.091180	285.796724	1.368920	1.608696
32	69.243253	206.716532	N/A	N/A
33	588.341998	2616.847655	N/A	N/A
34	130.218864	469.809210	N/A	N/A
35	176.424494	1194.214233	N/A	N/A
36	253.396434	1621.865262	N/A	N/A
37	149.194304	758.219895	N/A	N/A
38	64.038932	177.045159	N/A	N/A
39	90.278220	439.358395	N/A	N/A
40	51.339927	96.627891	N/A	N/A
41	174.327404	968.373096	N/A	N/A
42	27559.377885	186210.154356	N/A	N/A
43	935.864694	4425.557972	N/A	N/A
44	375.087963	866.532640	N/A	N/A

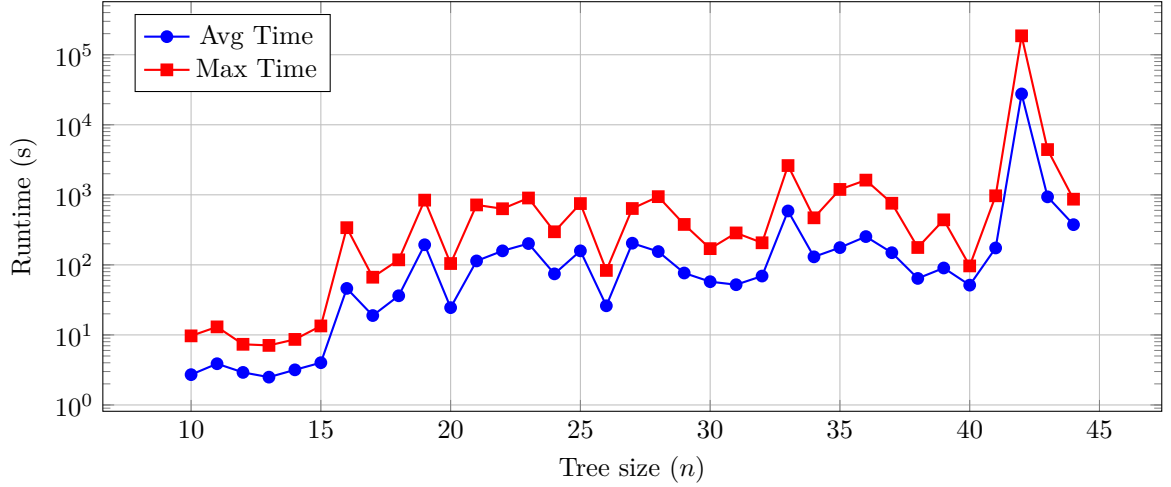


Figure 4.38: Random bounded degree trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

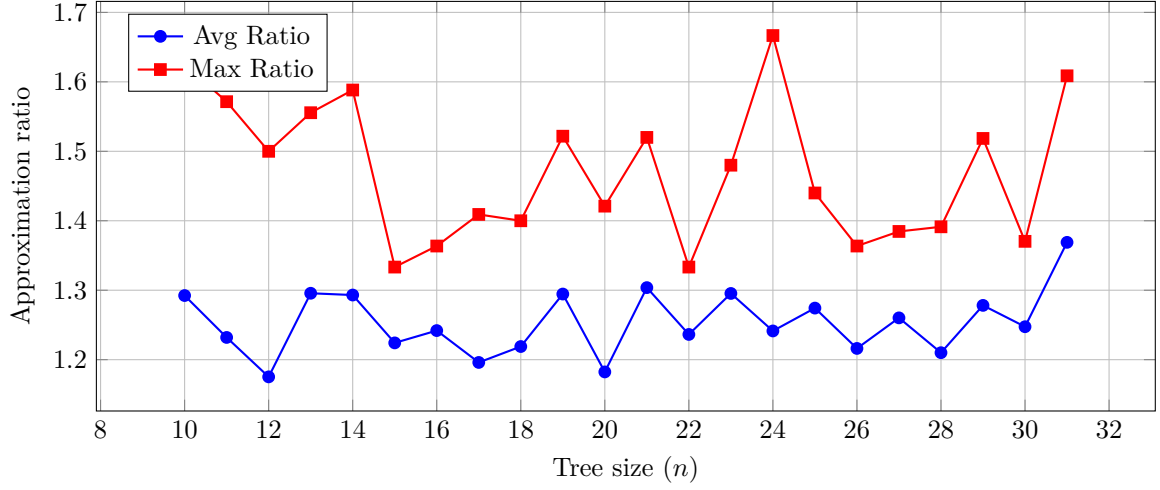


Figure 4.39: Random bounded degree trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.26: Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	14.003681	77.763831	1.206406	1.375000
11	155.893464	1068.826922	1.192894	1.428571
12	727.725333	4558.311441	1.302200	1.500000
13	221.249011	1257.260499	1.243096	1.611111
14	216.184106	1192.845575	1.304222	1.454545
15	1396.084216	5257.675150	1.363265	1.391304

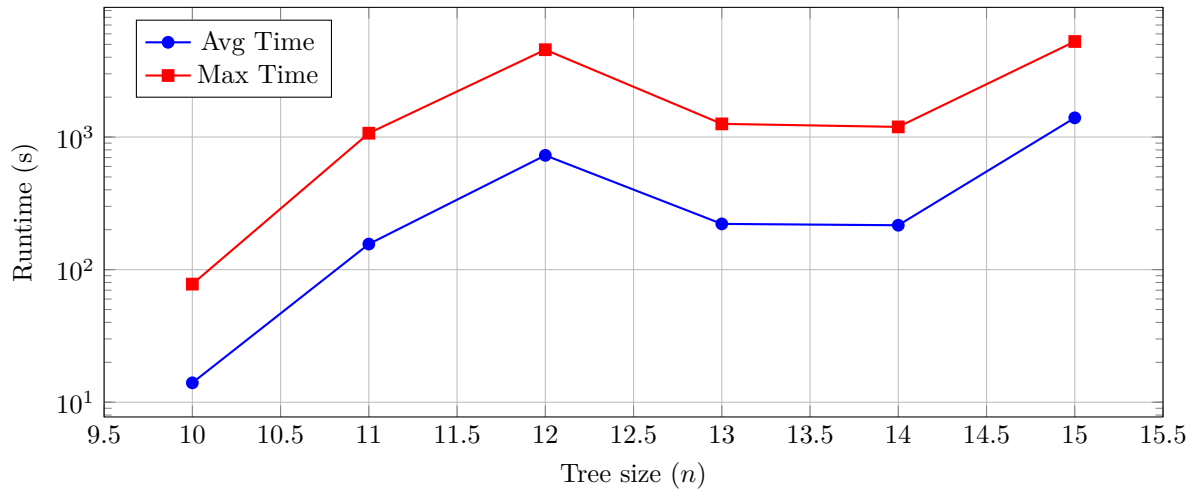


Figure 4.40: Random bounded degree trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime

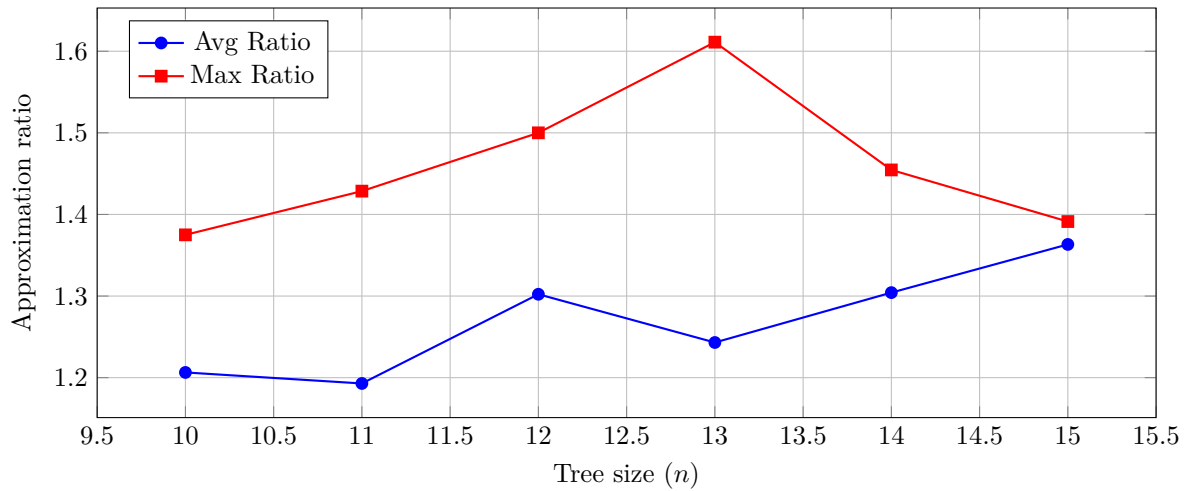


Figure 4.41: Random bounded degree trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality

4.3.6 Random bounded degree trees with Exponential Cost Distribution ($T_\Delta || V, c \sim \mathcal{E} || C_{max}$)

Exact Algorithm

Table 4.27: Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	0.663119	0.825377
11	1.196731	1.791410
12	1.891039	2.513666
13	3.305596	4.702163
14	5.965716	9.199916
15	9.114615	15.714215
16	15.251530	19.425086
17	27.303563	43.274854
18	34.456076	50.955873
19	64.341576	87.757195
20	100.080899	161.331325
21	155.988381	192.563649
22	297.411222	540.406355
23	462.118332	691.724724
24	754.758051	1054.120033
25	1040.417462	1435.313827
26	1872.205580	2273.624595
27	2700.789516	4197.356010
28	3550.437934	5247.668222
29	6667.881853	9457.579550
30	11124.814657	20837.525344
31	14329.817512	25373.898406
32	17600.651535	25193.961912

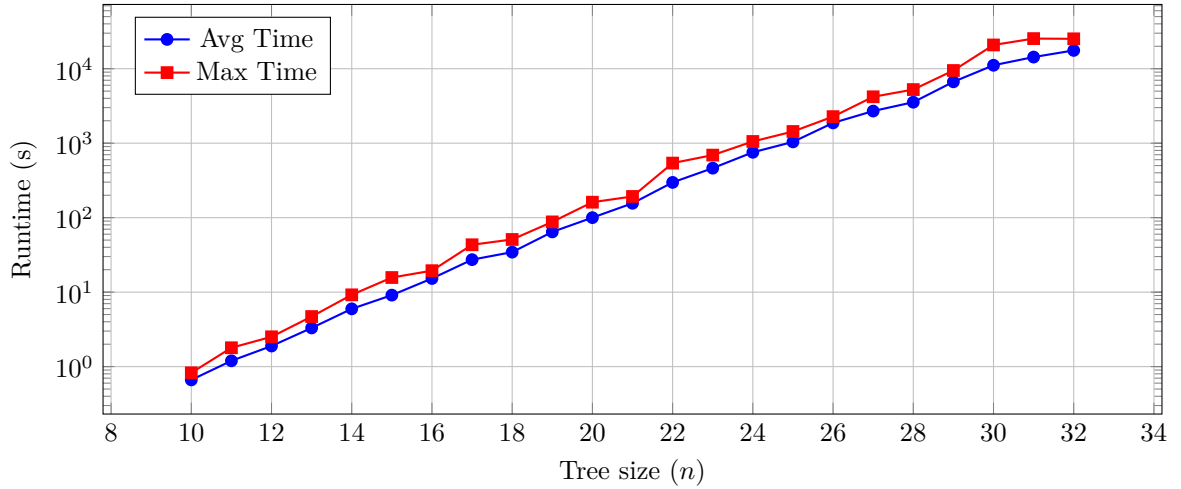


Figure 4.42: Random bounded degree trees with exponential costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

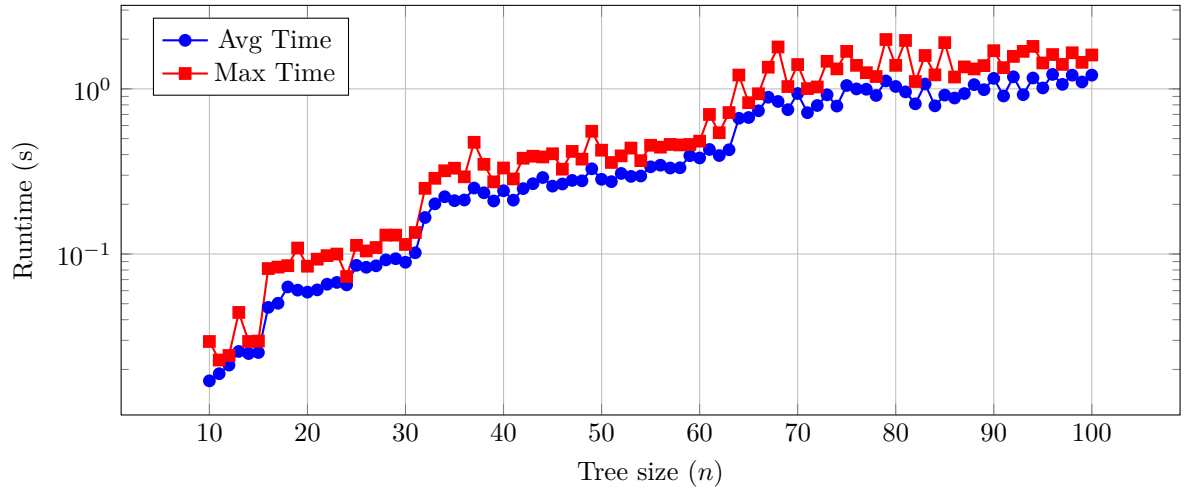


Figure 4.43: Random bounded degree trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

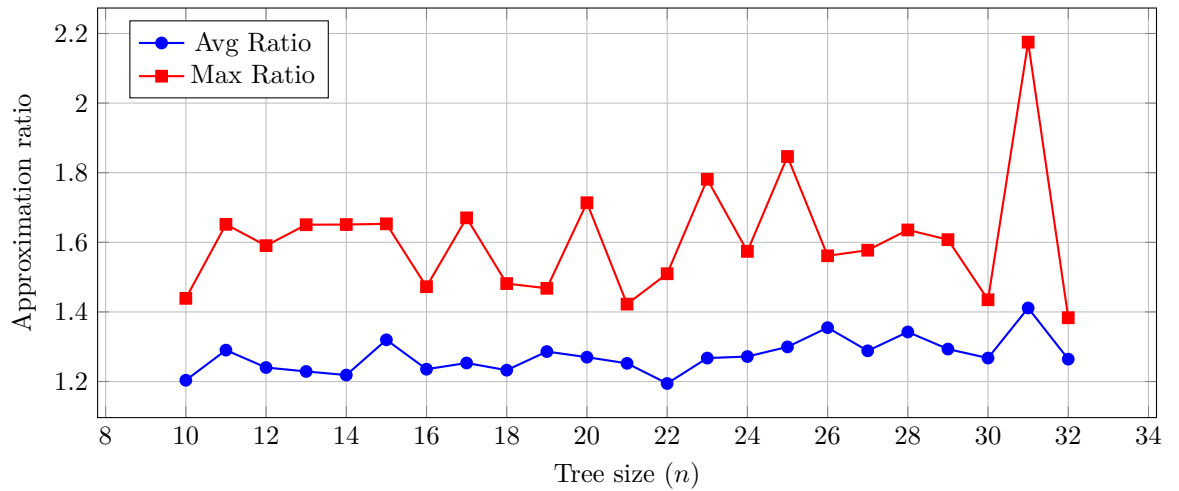


Figure 4.44: Random bounded degree trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.28: Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.017064	0.029529	1.203674	1.438970
11	0.018853	0.022809	1.290255	1.651646
12	0.021250	0.024305	1.240315	1.590435
13	0.025714	0.044285	1.228978	1.650574
14	0.024965	0.029545	1.218572	1.651093
15	0.025329	0.029701	1.319867	1.653139
16	0.047514	0.081636	1.235426	1.472456
17	0.050320	0.083322	1.253268	1.670258
18	0.063168	0.084982	1.232629	1.481480
19	0.060437	0.108687	1.286146	1.467894
20	0.058776	0.084347	1.269993	1.713725
21	0.060658	0.093048	1.252278	1.422471
22	0.065601	0.097741	1.194472	1.509723
23	0.067297	0.099939	1.267425	1.781170
24	0.065035	0.073085	1.271819	1.573995
25	0.085369	0.112901	1.299526	1.846501
26	0.083125	0.104377	1.354881	1.561302
27	0.084848	0.109456	1.288269	1.577357
28	0.092133	0.130192	1.342451	1.635519
29	0.093632	0.130122	1.293282	1.607648
30	0.089140	0.114238	1.267485	1.434932
31	0.101782	0.134960	1.411348	2.170501
32	0.166439	0.249948	1.264328	1.383342
33	0.201353	0.287340	N/A	N/A
34	0.222321	0.319286	N/A	N/A
35	0.210389	0.330792	N/A	N/A
36	0.212352	0.293339	N/A	N/A
37	0.250889	0.474691	N/A	N/A
38	0.234912	0.349340	N/A	N/A
39	0.209537	0.273494	N/A	N/A
40	0.241600	0.331728	N/A	N/A
41	0.211904	0.284885	N/A	N/A
42	0.248906	0.380130	N/A	N/A
43	0.266475	0.391026	N/A	N/A
44	0.290551	0.386946	N/A	N/A
45	0.257645	0.404594	N/A	N/A
46	0.265473	0.326387	N/A	N/A
47	0.279058	0.418024	N/A	N/A
48	0.277432	0.375453	N/A	N/A
49	0.328180	0.553681	N/A	N/A
50	0.283433	0.425298	N/A	N/A
51	0.274254	0.358508	N/A	N/A
52	0.307566	0.393008	N/A	N/A
53	0.295097	0.437961	N/A	N/A
54	0.296799	0.367636	N/A	N/A
55	0.337339	0.455427	N/A	N/A
56	0.344989	0.443624	N/A	N/A
57	0.331748	0.460763	N/A	N/A
58	0.332874	0.457525	N/A	N/A
59	0.393894	0.460473	N/A	N/A
60	0.381587	0.482144	N/A	N/A
61	0.429498	0.699667	N/A	N/A
62	0.394799	0.542583	N/A	N/A
63	0.427788	0.718797	N/A	N/A
64	0.663625	1.212662	N/A	N/A
65	0.670774	0.824984	N/A	N/A
66	0.736604	0.933771	N/A	N/A
67	0.889918	1.352905	N/A	N/A
68	0.839054	1.793355	N/A	N/A
69	0.748483	1.032637	N/A	N/A
70	0.937182	1.403687	N/A	N/A
71	0.717431	1.004171	N/A	N/A
72	0.793108	1.026931	N/A	N/A
73	0.920438	1.468861	N/A	N/A
74	0.786757	1.321461	N/A	N/A
75	1.048187	1.686488	N/A	N/A
76	0.998943	1.388138	N/A	N/A
77	0.994878	1.253292	N/A	N/A
78	0.912260	1.187414	N/A	N/A
79	1.116958	1.990540	N/A	N/A
80	1.033987	1.387972	N/A	N/A
81	0.959086	1.964185	N/A	N/A
82	0.811741	1.107800	N/A	N/A
83	1.069755	1.592601	N/A	N/A
84	0.789396	1.215381	N/A	N/A
85	0.916605	1.905889	N/A	N/A
86	0.880137	1.179485	N/A	N/A
87	0.936402	1.358390	N/A	N/A
88	1.060604	1.320099	N/A	N/A
89	0.988824	1.383169	N/A	N/A
90	1.157446	1.702745	N/A	N/A
91	0.905492	1.344196	N/A	N/A
92	1.183025	1.572788	N/A	N/A
93	0.922568	1.686457	N/A	N/A
94	1.163717	1.807472	N/A	N/A
95	1.012986	1.435413	N/A	N/A
96	1.223400	1.613742	N/A	N/A
97	1.065020	1.406236	N/A	N/A
98	1.211027	1.654624	N/A	N/A
99	1.101084	1.447720	N/A	N/A
100	1.210936	1.604171	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.29: Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.416642	2.304174	1.208981	1.551379
11	1.566125	9.968691	1.235514	1.482922
12	8.060018	23.108846	1.229067	1.696431
13	11.551575	34.749016	1.135136	1.650574
14	3.376439	11.136671	1.136011	1.386714
15	3.714715	27.084303	1.353174	1.711252
16	31.152829	176.503974	1.193502	1.405833
17	45.856546	237.382386	1.208236	1.683934
18	63.446371	295.483772	1.331430	1.506513
19	31.126896	72.799549	1.186911	1.467894
20	20.580559	68.998626	1.293550	1.796065
21	59.362935	201.310459	1.302070	1.598608
22	216.319982	1098.592793	1.187082	1.397604
23	29.765297	178.386978	1.198197	1.379125
24	36.516029	149.678688	1.385947	1.541066
25	72.887743	296.407992	1.302261	1.846501
26	152.003038	1011.778344	1.291703	1.710484
27	29.966320	113.545037	1.248555	1.664102
28	84.513740	174.619332	1.406283	1.663876
29	39.275636	82.207895	1.428571	1.768315
30	54.982348	205.622594	1.251937	1.434932
31	58.428502	178.671214	1.374699	1.656947
32	80.730125	282.358989	1.400324	1.578010
33	56.678606	144.394358	N/A	N/A
34	51.278265	272.960317	N/A	N/A
35	42.485219	137.902753	N/A	N/A
36	344.437207	1758.645433	N/A	N/A
37	45.920929	135.361206	N/A	N/A
38	84.867036	140.305884	N/A	N/A
39	19.364613	39.080111	N/A	N/A
40	110.730643	238.751794	N/A	N/A
41	59.601236	156.672091	N/A	N/A
42	487.209322	1030.513898	N/A	N/A
43	1111.938719	2311.391432	N/A	N/A
44	2781.395112	6759.219824	N/A	N/A
45	1385.200843	3569.999060	N/A	N/A
46	329.113977	618.909292	N/A	N/A
47	16583.554629	65386.335098	N/A	N/A
48	160.878547	401.900421	N/A	N/A
49	160.201559	218.469930	N/A	N/A
50	1356.671227	5457.146720	N/A	N/A

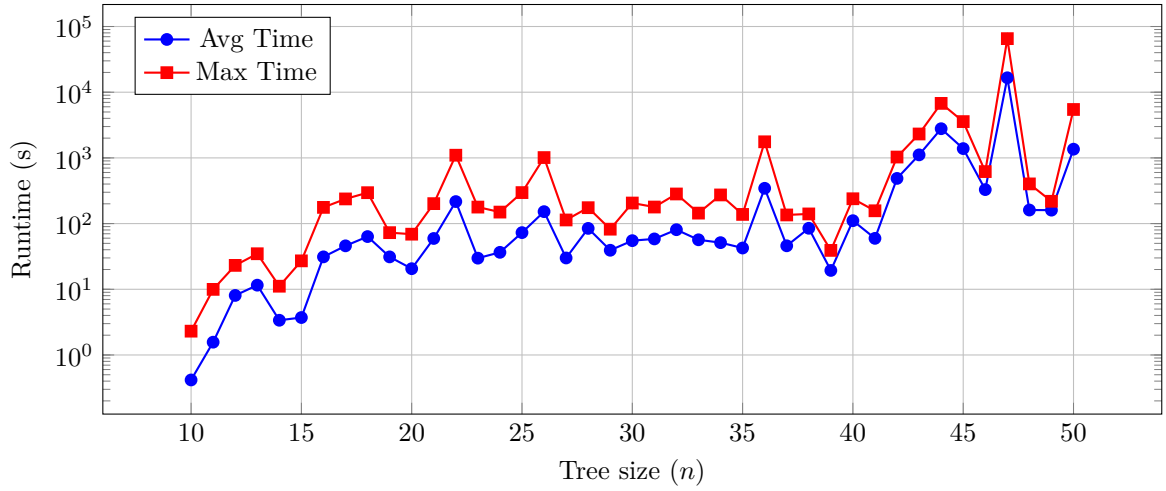


Figure 4.45: Random bounded degree trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

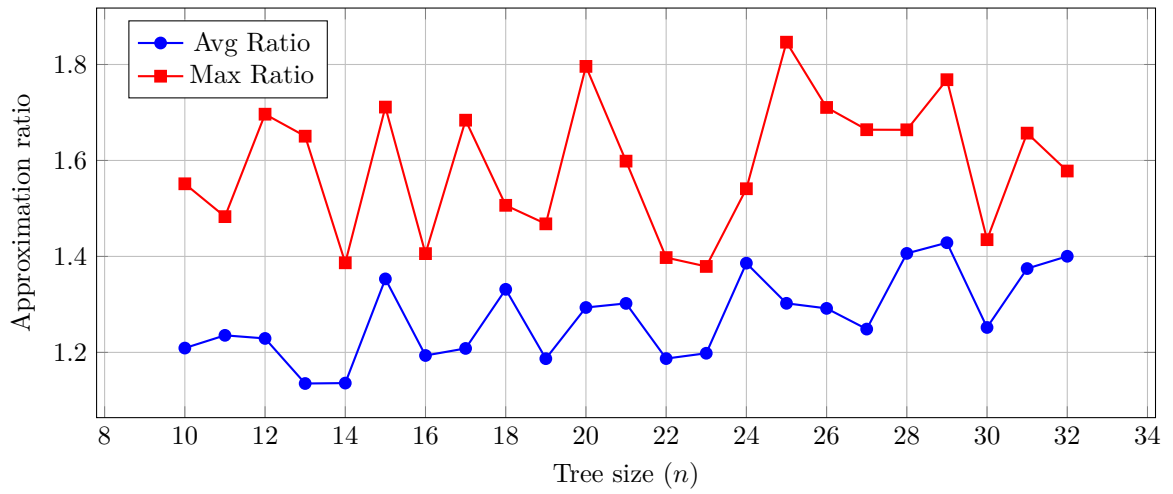


Figure 4.46: Random bounded degree trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.30: Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	8.518412	46.525981	1.157647	1.313390
11	25.215408	104.294252	1.251842	1.680091
12	1702.842947	11831.382128	1.159732	1.345978
13	14.519249	133.357571	1.336236	1.784461
14	11.285647	45.812740	1.357166	2.297747
15	2578.814745	14546.472022	1.376148	2.032323
16	223.922808	2107.476180	1.426391	1.810031

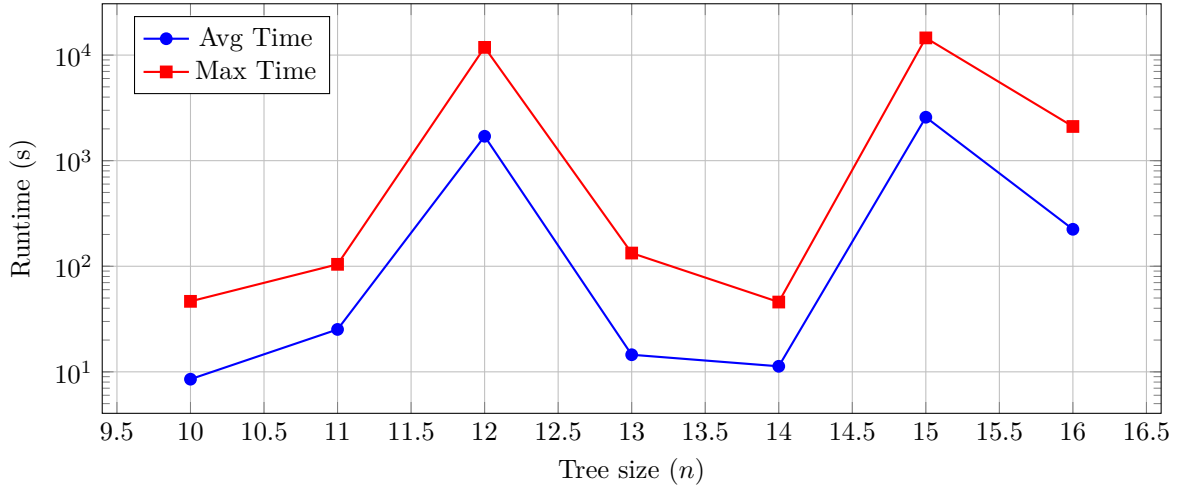


Figure 4.47: Random bounded degree trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime

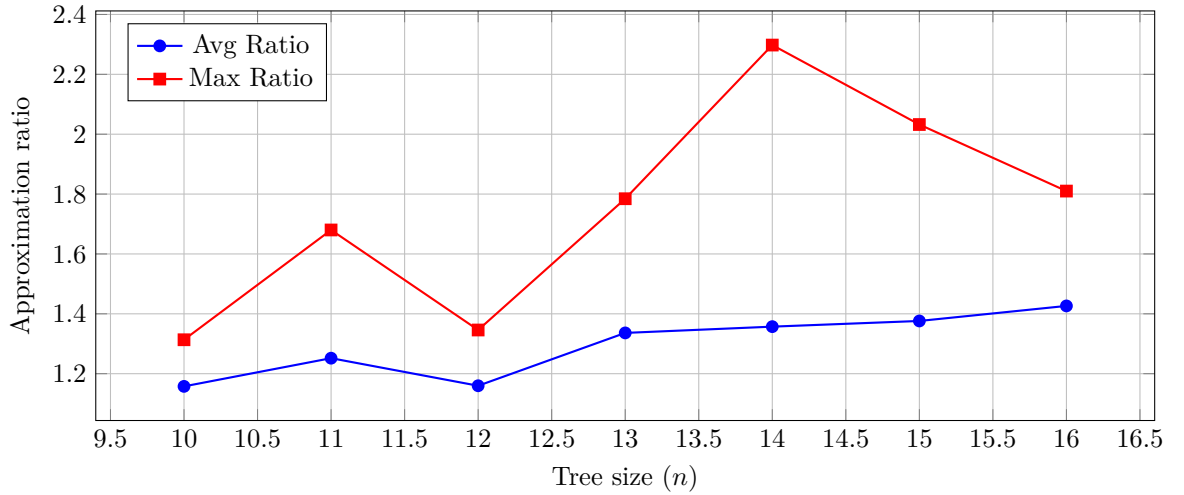


Figure 4.48: Random bounded degree trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality

This time the $O(\sqrt{\log n})$ -approximation algorithm exhibits the best approximation factors never exceeding the value of 2 as opposed to the two other algorithms.

4.3.7 Random bounded diameter trees with Uniform Cost Distribution ($T_D || V, c \sim \mathcal{U} || C_{max}$)

Exact Algorithm

Table 4.31: Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	1.570919	3.689062
11	5.067980	9.054298
12	8.938721	15.989202
13	14.437796	26.267712
14	44.635348	109.074023
15	111.936501	264.827078
16	202.116636	601.761013
17	221.160183	478.393772
18	896.816837	3098.286423
19	2460.687899	6934.493838
20	2373.245927	11474.476462
21	6951.792639	24704.847907

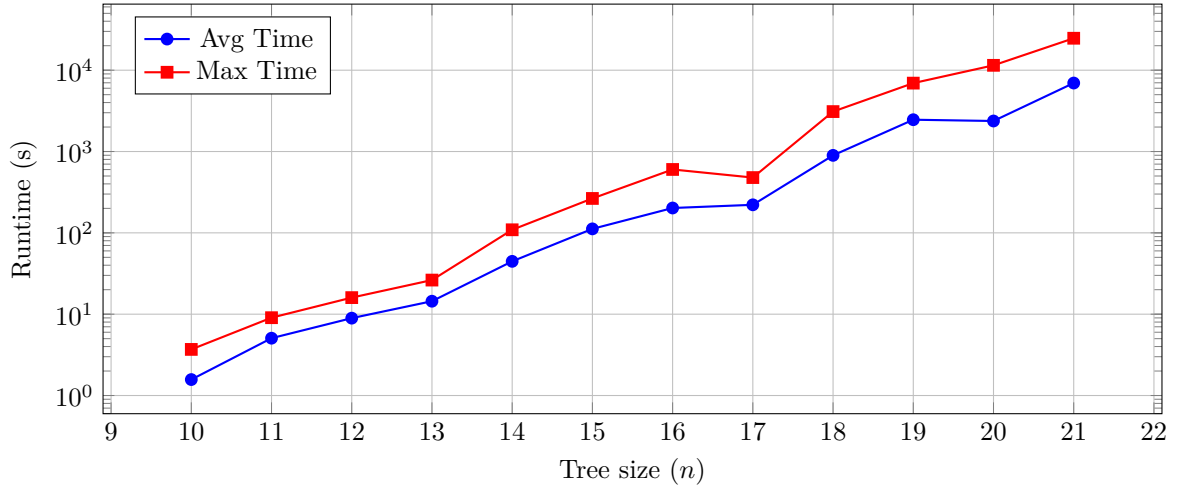


Figure 4.49: Random bounded diameter trees with uniform costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

Table 4.32: Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.015717	0.035868	1.158040	1.524359
11	0.018510	0.022401	1.021099	1.101270
12	0.018348	0.022054	1.058215	1.468638
13	0.022122	0.026800	1.098573	1.324347
14	0.028406	0.044522	1.083748	1.361272
15	0.031735	0.038755	1.122481	1.439557
16	0.039870	0.065156	1.074421	1.303415
17	0.046449	0.060752	1.112277	1.454640
18	0.063156	0.099265	1.040755	1.346087
19	0.063401	0.115019	1.101893	1.278825
20	0.074137	0.109926	1.130023	1.358806
21	0.071196	0.096427	1.112793	1.325175
22	0.083981	0.110521	N/A	N/A
23	0.079816	0.117888	N/A	N/A
24	0.093952	0.136024	N/A	N/A
25	0.103816	0.156663	N/A	N/A
26	0.107550	0.156436	N/A	N/A
27	0.130912	0.173057	N/A	N/A
28	0.102475	0.183071	N/A	N/A
29	0.130074	0.204052	N/A	N/A
30	0.157543	0.234070	N/A	N/A
31	0.177594	0.263206	N/A	N/A
32	0.241548	0.429633	N/A	N/A
33	0.251047	0.381243	N/A	N/A
34	0.238568	0.529830	N/A	N/A
35	0.281972	0.481327	N/A	N/A
36	0.292897	0.508345	N/A	N/A
37	0.290093	0.508398	N/A	N/A
38	0.401070	0.505328	N/A	N/A
39	0.309190	0.470669	N/A	N/A
40	0.384090	0.636493	N/A	N/A
41	0.370625	0.573808	N/A	N/A
42	0.431555	0.601916	N/A	N/A
43	0.416648	0.609130	N/A	N/A
44	0.388849	0.597484	N/A	N/A
45	0.325497	0.834146	N/A	N/A
46	0.655902	0.790465	N/A	N/A
47	0.477656	0.898420	N/A	N/A
48	0.506137	0.870802	N/A	N/A
49	0.602924	0.861005	N/A	N/A
50	0.564607	0.950451	N/A	N/A

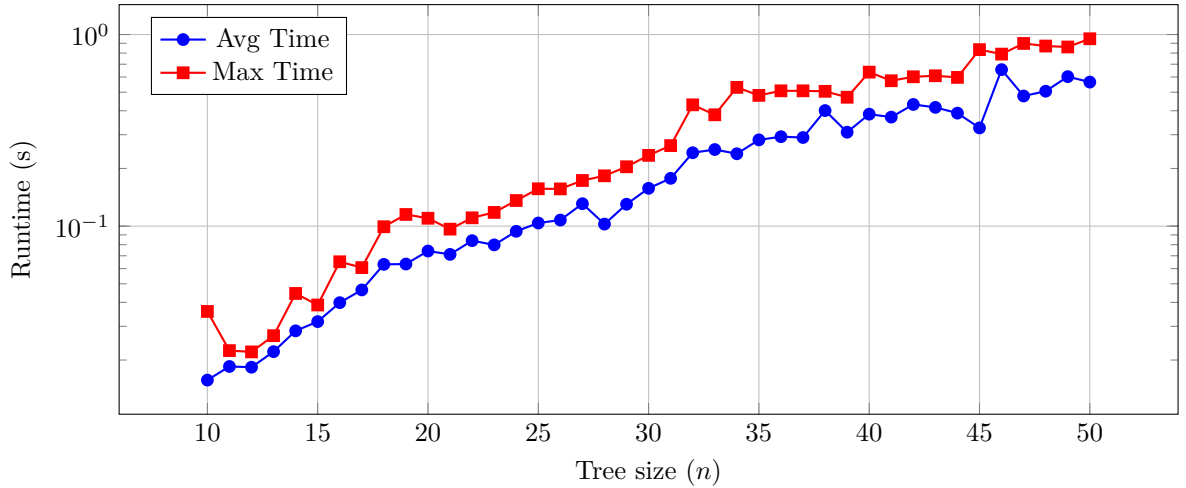


Figure 4.50: Random bounded diameter trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

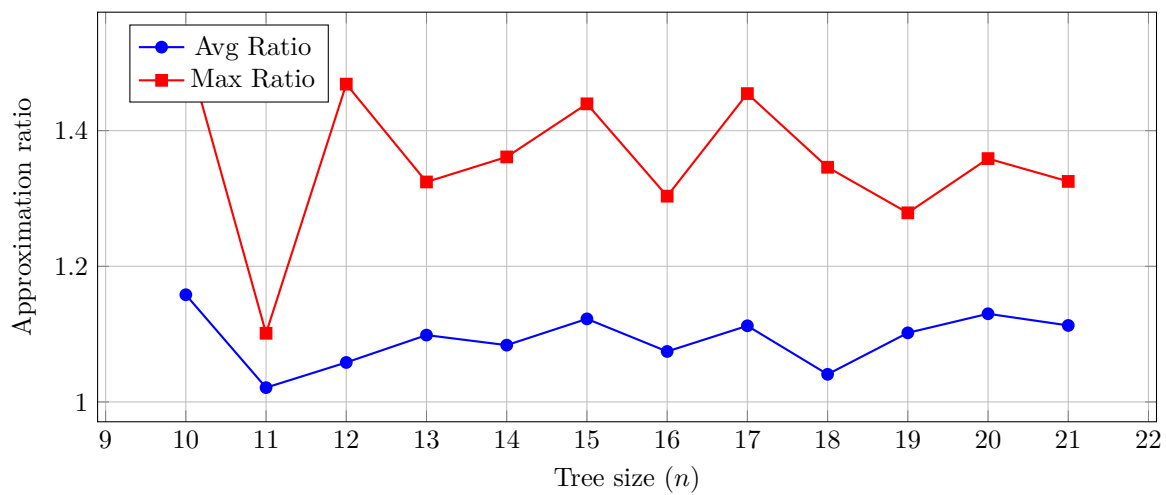


Figure 4.51: Random bounded diameter trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.33: Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	3.287247	28.175828	1.180749	1.524359
11	1.539845	10.784742	1.025939	1.101270
12	0.353198	2.807795	1.037722	1.228049
13	0.868840	7.466320	1.158708	1.705404
14	0.991367	5.981823	1.094554	1.276593
15	0.413340	2.238788	1.126369	1.457840
16	7.500186	36.091991	1.104593	1.617626
17	21.307834	165.633622	1.132573	1.524413
18	10.900548	50.275660	1.116013	1.346087
19	18.460351	130.807672	1.211408	1.575170
20	15.942153	53.129806	1.160428	1.358806
21	15.580031	65.252890	1.100604	1.369118
22	7.948837	27.060065	N/A	N/A
23	121.761659	1079.942826	N/A	N/A
24	56.717800	434.341046	N/A	N/A
25	14.660303	84.954009	N/A	N/A
26	11.105699	44.183038	N/A	N/A
27	6.038337	28.824637	N/A	N/A
28	4.180288	15.623693	N/A	N/A
29	40.721267	212.680349	N/A	N/A
30	9.982068	30.756533	N/A	N/A
31	17.933533	69.616473	N/A	N/A
32	24.270057	152.100059	N/A	N/A
33	64.534191	482.883076	N/A	N/A
34	120.340950	1028.112179	N/A	N/A
35	12.717277	47.877095	N/A	N/A
36	18.365993	128.844212	N/A	N/A
37	40.394286	214.318673	N/A	N/A
38	431.499173	4197.444947	N/A	N/A
39	4.364229	20.584513	N/A	N/A
40	120.698271	913.850650	N/A	N/A
41	13.957164	82.162563	N/A	N/A
42	271.618487	1780.709708	N/A	N/A
43	334.385962	2176.730366	N/A	N/A
44	1049.458907	7044.056830	N/A	N/A
45	1064.419623	8170.099147	N/A	N/A
46	1006.063249	5313.795363	N/A	N/A
47	1432.516379	5728.782735	N/A	N/A
48	528.977412	2043.697435	N/A	N/A
49	68.132585	328.327270	N/A	N/A
50	1487.996698	12471.008062	N/A	N/A

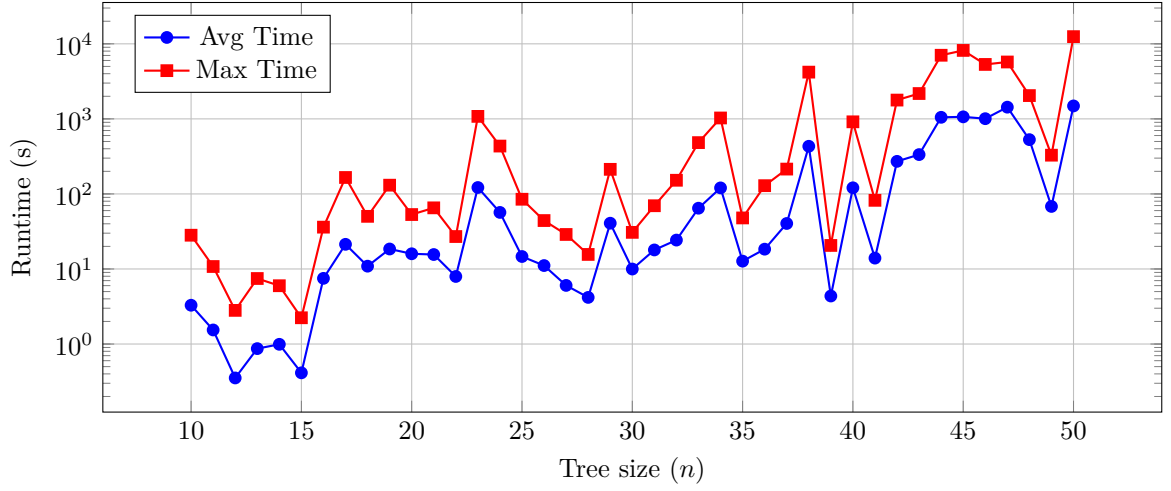


Figure 4.52: Random bounded diameter trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

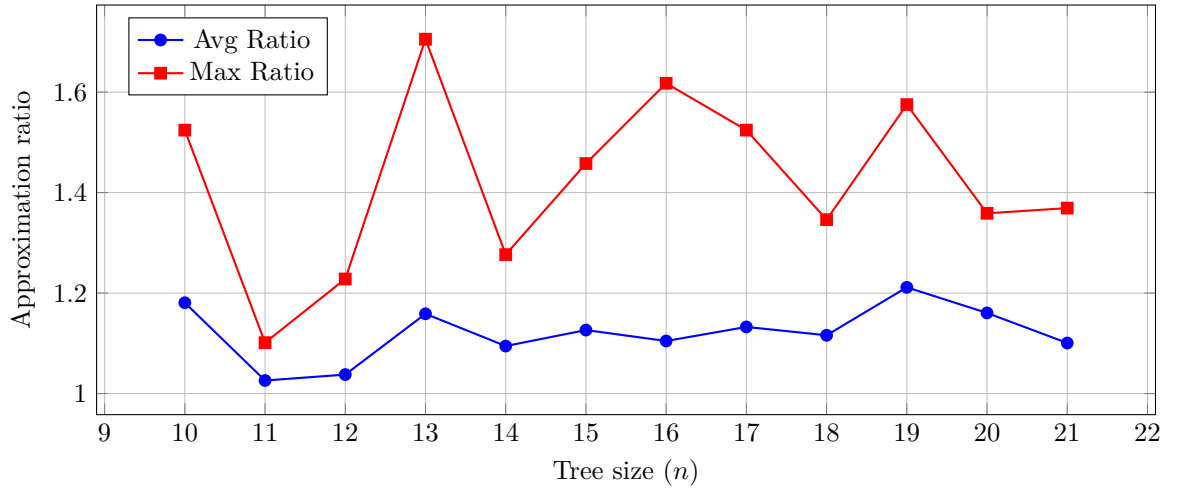


Figure 4.53: Random bounded diameter trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.34: Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	131.778845	1117.149967	1.187313	1.701051
11	81.237545	567.866360	1.155231	1.468990
12	1075.392901	8022.329236	1.055965	1.381036
13	47.522633	259.278009	1.209017	1.398526
14	3738.661589	35462.225328	1.155515	1.497487
15	324.363700	2858.840484	1.195433	1.542564

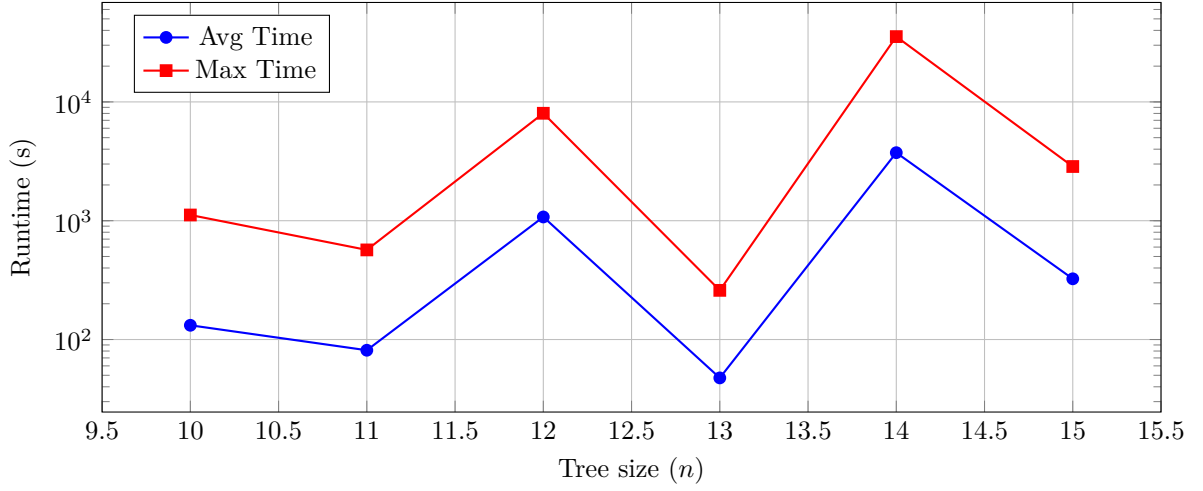


Figure 4.54: Random bounded diameter trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime

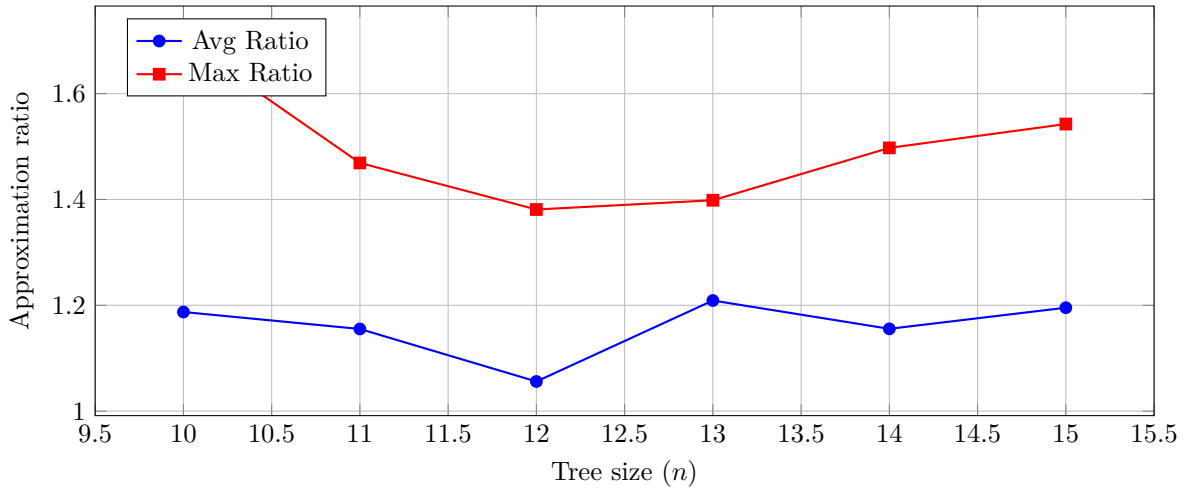


Figure 4.55: Random bounded diameter trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality

We see that all of the algorithms achieve the average approximation ratio of almost 1. This will usually also be the case for the other cost distributions for bounded diameter. Notably the approximation ratio of the $O(\log n / \log \log n)$ -approximation algorithm never exceeds 1.6.

4.3.8 Random bounded diameter trees with Binomial Cost Distribution ($T_D || V, c \sim \mathcal{B} || C_{max}$)

Exact Algorithm

Table 4.35: Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	1.760227	3.878860
11	4.899907	9.239899
12	9.186380	21.423688
13	24.992342	50.947375
14	34.354150	85.870407
15	99.644084	204.957481
16	193.832795	471.235977
17	427.049221	1374.987414
18	932.567403	2802.925454
19	2235.720163	6614.199808
20	4639.727391	14707.450616
21	12912.217062	95781.374143
22	6060.328289	20303.635648
23	14697.500100	28304.439949

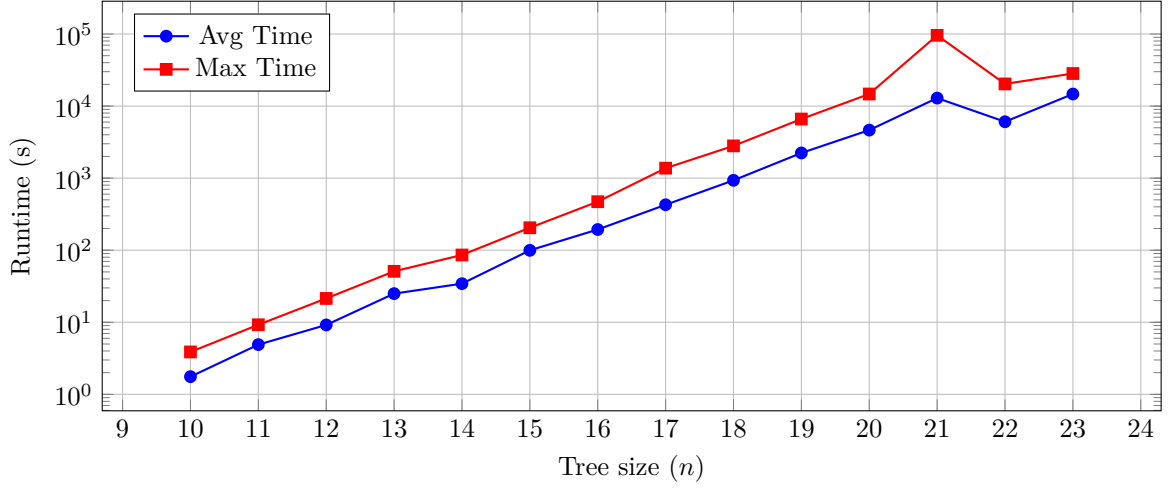


Figure 4.56: Random bounded diameter trees with binomial costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

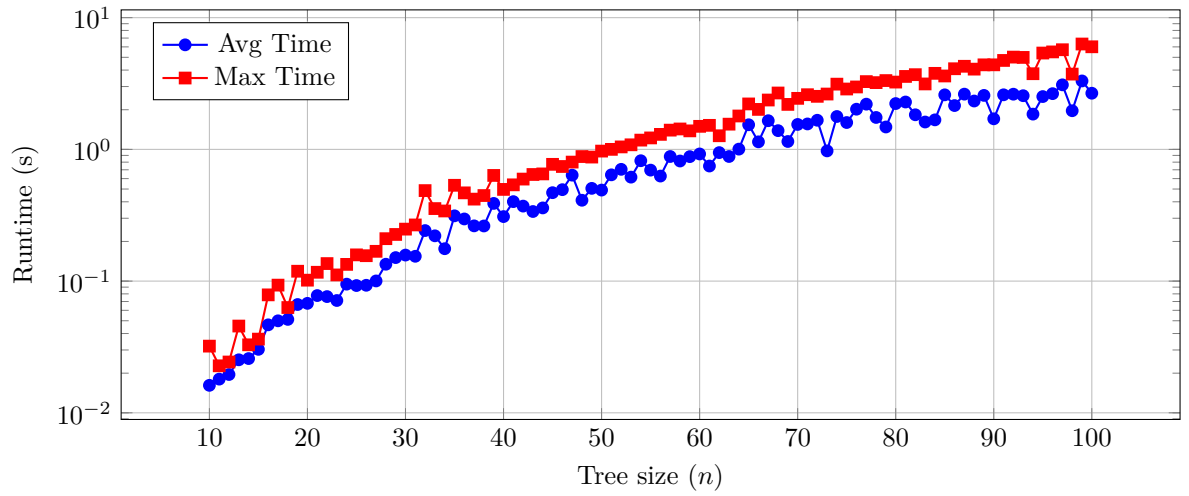


Figure 4.57: Random bounded diameter trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

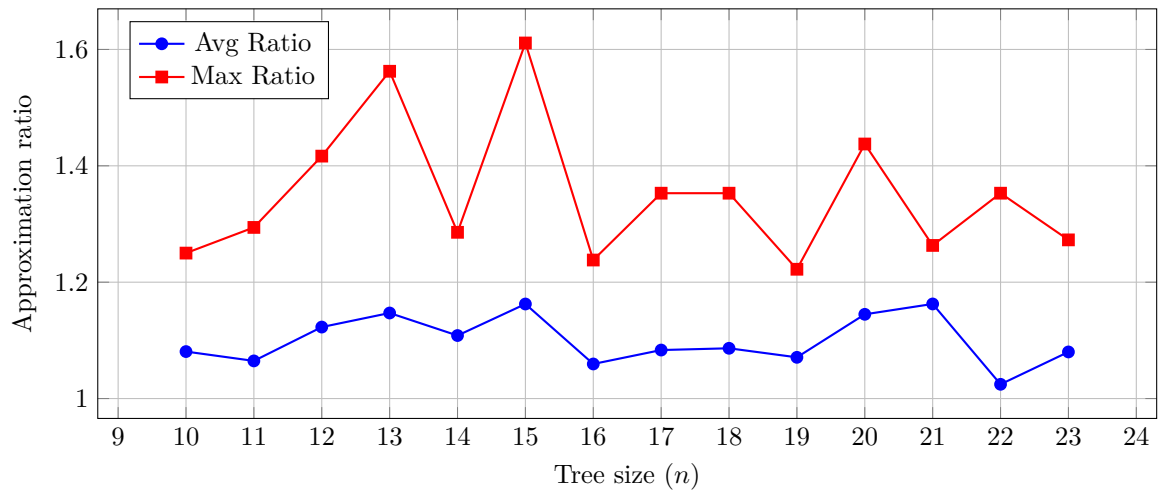


Figure 4.58: Random bounded diameter trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.36: Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.016170	0.032090	1.080752	1.250000
11	0.018023	0.022746	1.064706	1.294118
12	0.019539	0.024310	1.122869	1.416667
13	0.025266	0.045581	1.147044	1.562500
14	0.025816	0.032832	1.108310	1.285714
15	0.030333	0.036272	1.162500	1.611111
16	0.046554	0.078634	1.059447	1.238095
17	0.049943	0.093208	1.083371	1.352941
18	0.051202	0.063002	1.086401	1.352941
19	0.066410	0.118831	1.070857	1.222222
20	0.067775	0.101640	1.144768	1.437500
21	0.077661	0.116701	1.162588	1.263158
22	0.076282	0.135868	1.024494	1.352941
23	0.071245	0.111402	1.080118	1.272727
24	0.094803	0.133947	N/A	N/A
25	0.092541	0.158192	N/A	N/A
26	0.092963	0.155894	N/A	N/A
27	0.100277	0.168299	N/A	N/A
28	0.134316	0.209938	N/A	N/A
29	0.150842	0.225989	N/A	N/A
30	0.157621	0.248060	N/A	N/A
31	0.154327	0.266910	N/A	N/A
32	0.241898	0.486530	N/A	N/A
33	0.220691	0.355357	N/A	N/A
34	0.176181	0.340748	N/A	N/A
35	0.313027	0.533794	N/A	N/A
36	0.296371	0.467690	N/A	N/A
37	0.262639	0.419173	N/A	N/A
38	0.262171	0.446234	N/A	N/A
39	0.388871	0.634181	N/A	N/A
40	0.308942	0.496756	N/A	N/A
41	0.401200	0.538110	N/A	N/A
42	0.371027	0.595056	N/A	N/A
43	0.337597	0.644731	N/A	N/A
44	0.359443	0.651354	N/A	N/A
45	0.469169	0.770009	N/A	N/A
46	0.494499	0.741522	N/A	N/A
47	0.637428	0.800795	N/A	N/A
48	0.410988	0.882099	N/A	N/A
49	0.505863	0.874377	N/A	N/A
50	0.490955	0.970554	N/A	N/A
51	0.640909	1.000332	N/A	N/A
52	0.705589	1.044048	N/A	N/A
53	0.615620	1.083273	N/A	N/A
54	0.820119	1.175789	N/A	N/A
55	0.695847	1.221788	N/A	N/A
56	0.627305	1.299833	N/A	N/A
57	0.882949	1.400036	N/A	N/A
58	0.816052	1.428745	N/A	N/A
59	0.881003	1.382361	N/A	N/A
60	0.923157	1.496155	N/A	N/A
61	0.747629	1.522276	N/A	N/A
62	0.945302	1.269057	N/A	N/A
63	0.883152	1.554010	N/A	N/A
64	1.004425	1.792936	N/A	N/A
65	1.537895	2.213236	N/A	N/A
66	1.141046	2.011576	N/A	N/A
67	1.650844	2.370144	N/A	N/A
68	1.387555	2.679741	N/A	N/A
69	1.148193	2.195346	N/A	N/A
70	1.546593	2.440237	N/A	N/A
71	1.560059	2.600668	N/A	N/A
72	1.663955	2.527978	N/A	N/A
73	0.974159	2.631599	N/A	N/A
74	1.776961	3.128015	N/A	N/A
75	1.599191	2.873196	N/A	N/A
76	2.018583	2.977940	N/A	N/A
77	2.204537	3.270172	N/A	N/A
78	1.747505	3.213140	N/A	N/A
79	1.479354	3.329861	N/A	N/A
80	2.227701	3.243287	N/A	N/A
81	2.289397	3.579549	N/A	N/A
82	1.831228	3.691910	N/A	N/A
83	1.613745	3.139737	N/A	N/A
84	1.675306	3.778127	N/A	N/A
85	2.595014	3.608533	N/A	N/A
86	2.151243	4.094410	N/A	N/A
87	2.626440	4.274898	N/A	N/A
88	2.329371	4.066767	N/A	N/A
89	2.569464	4.383866	N/A	N/A
90	1.706584	4.384948	N/A	N/A
91	2.596474	4.733730	N/A	N/A
92	2.623996	5.018773	N/A	N/A
93	2.555726	4.991155	N/A	N/A
94	1.852433	3.753886	N/A	N/A
95	2.517838	5.388223	N/A	N/A
96	2.649011	5.495581	N/A	N/A
97	3.090944	5.708246	N/A	N/A
98	1.965718	3.728591	N/A	N/A
99	3.308271	6.309167	N/A	N/A
100	2.669058	6.010501	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.37: Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	1.315450	4.699845	1.162861	1.444444
11	0.636294	5.900868	1.088235	1.411765
12	0.140553	0.630252	1.131398	1.400000
13	1.056481	5.165290	1.145640	1.562500
14	1.264377	9.451447	1.128011	1.388889
15	0.071657	0.131132	1.207702	1.611111
16	44.672615	394.039752	1.016527	1.117647
17	6.265333	26.497168	1.090611	1.538462
18	31.365364	220.171019	1.080274	1.222222
19	17.083326	114.709722	1.177091	1.388889
20	14.039402	108.898929	1.157268	1.437500
21	50.435396	313.779091	1.143870	1.235294
22	5.086809	29.426137	1.108209	1.789474
23	18.210017	141.512115	1.047650	1.238095
24	9.227443	39.095663	N/A	N/A
25	12.523250	34.850034	N/A	N/A
26	22.968485	113.524297	N/A	N/A
27	6.417469	22.926064	N/A	N/A
28	6.561036	32.112314	N/A	N/A
29	515.440229	3122.776753	N/A	N/A
30	19.432142	138.441616	N/A	N/A
31	18.417156	109.779732	N/A	N/A
32	69.179037	539.444268	N/A	N/A
33	58.814576	331.901095	N/A	N/A
34	11.658635	35.160866	N/A	N/A
35	12.996855	76.226720	N/A	N/A
36	16.243133	66.497735	N/A	N/A
37	143.753308	948.777878	N/A	N/A
38	13.601047	79.402194	N/A	N/A
39	79.787061	630.501891	N/A	N/A
40	10.829271	71.951620	N/A	N/A
41	145.360365	1271.388779	N/A	N/A
42	230.065454	1707.749052	N/A	N/A
43	427.713947	2233.571819	N/A	N/A
44	283.498826	1387.681610	N/A	N/A
45	711.620931	2755.460815	N/A	N/A
46	410.110760	2031.228997	N/A	N/A
47	63.226483	337.901358	N/A	N/A
48	1466.843111	11486.724226	N/A	N/A
49	574.966115	5326.648618	N/A	N/A
50	413.687725	2192.995060	N/A	N/A

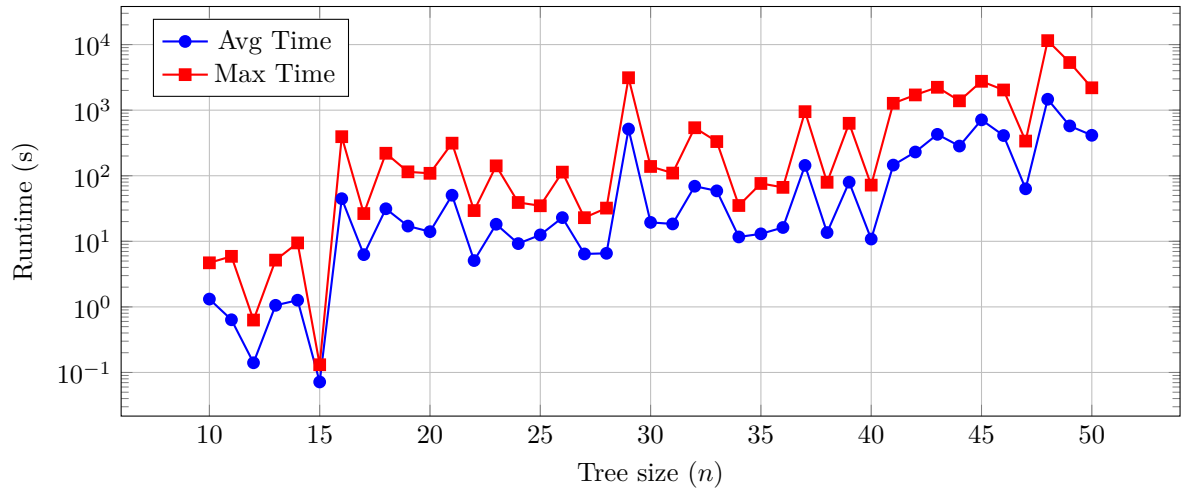


Figure 4.59: Random bounded diameter trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

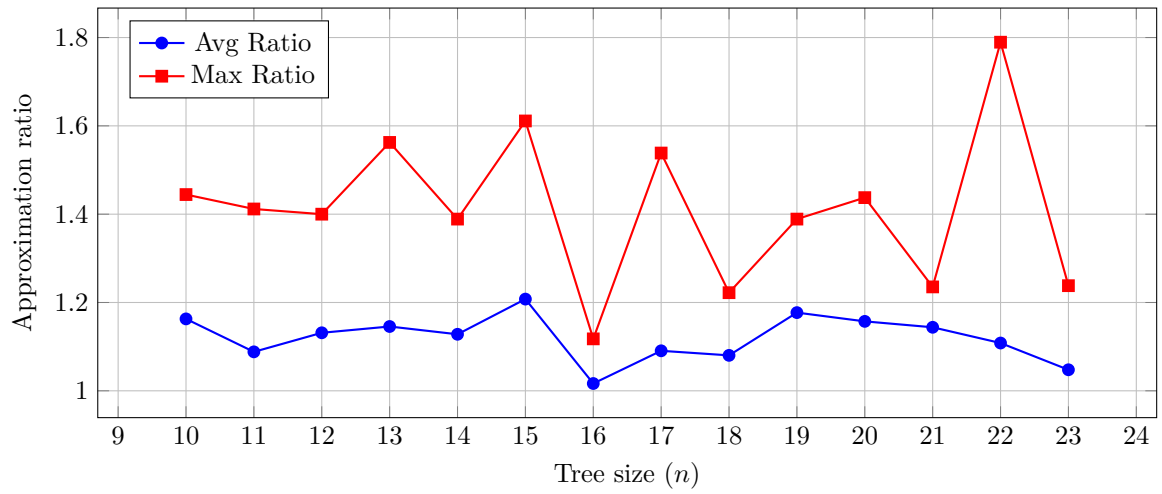


Figure 4.60: Random bounded diameter trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.38: Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	60.290345	466.822105	1.208933	1.687500
11	46.676202	428.006508	1.124510	1.500000
12	2240.480068	13996.713583	1.170216	1.466667
13	2090.532960	13910.421657	1.135247	1.545455
14	12705.560097	25411.102532	1.071429	1.142857

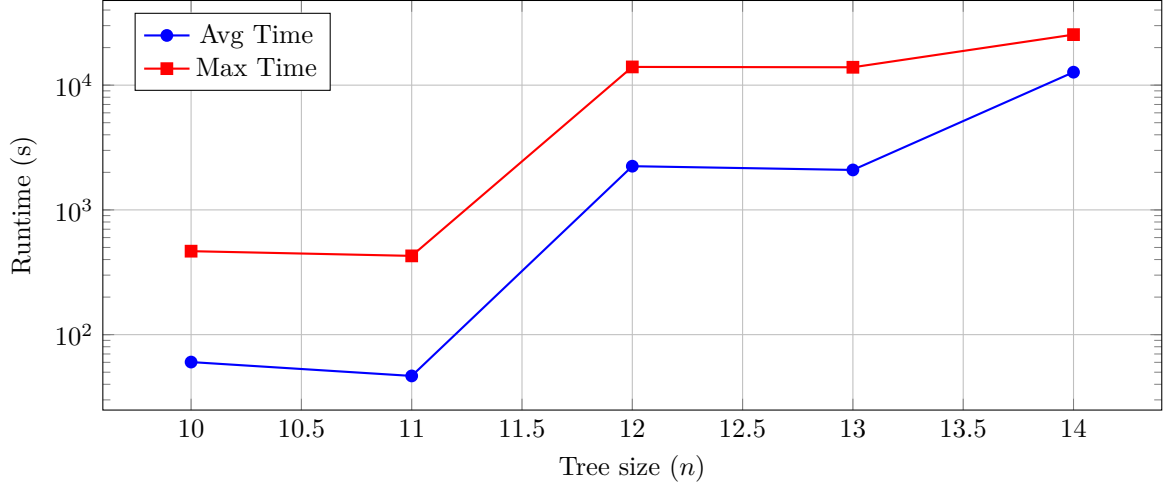


Figure 4.61: Random bounded diameter trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime

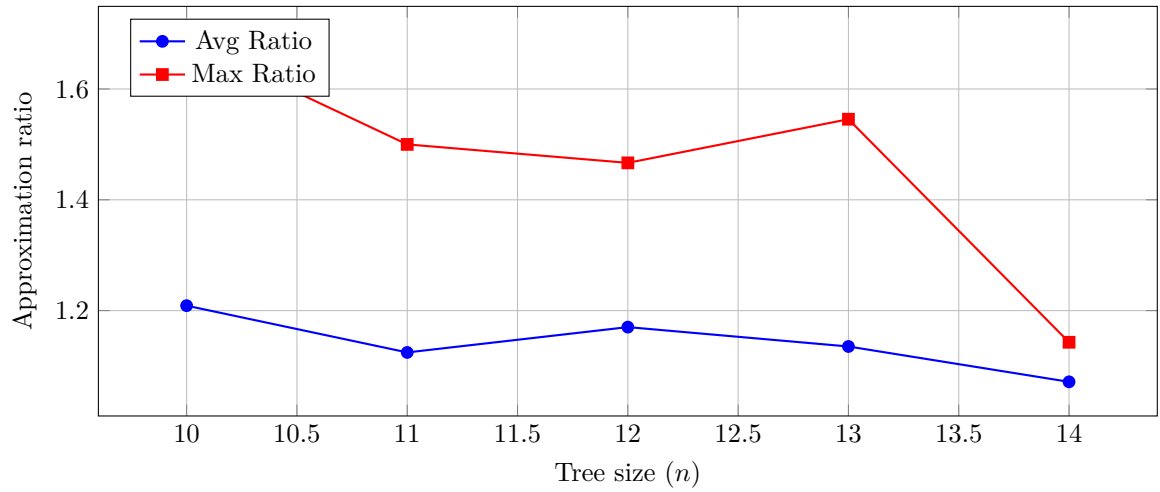


Figure 4.62: Random bounded diameter trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality

4.3.9 Random bounded diameter trees with Exponential Cost Distribution ($T_D || V, c \sim \mathcal{E} || C_{max}$)

Exact Algorithm

Table 4.39: Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - exact algorithm

n	Avg Time (s)	Max Time (s)
10	1.280326	2.232470
11	3.040468	5.115279
12	10.894153	22.382208
13	13.780860	21.255968
14	37.352216	119.960251
15	110.038087	266.557135
16	178.826359	469.600459
17	373.887334	1069.962105
18	982.615131	3046.864656
19	1818.714536	3538.850086
20	5373.141584	15272.695035
21	6451.650789	21614.264220
22	18464.674132	55500.008167
23	9718.348483	34494.313977

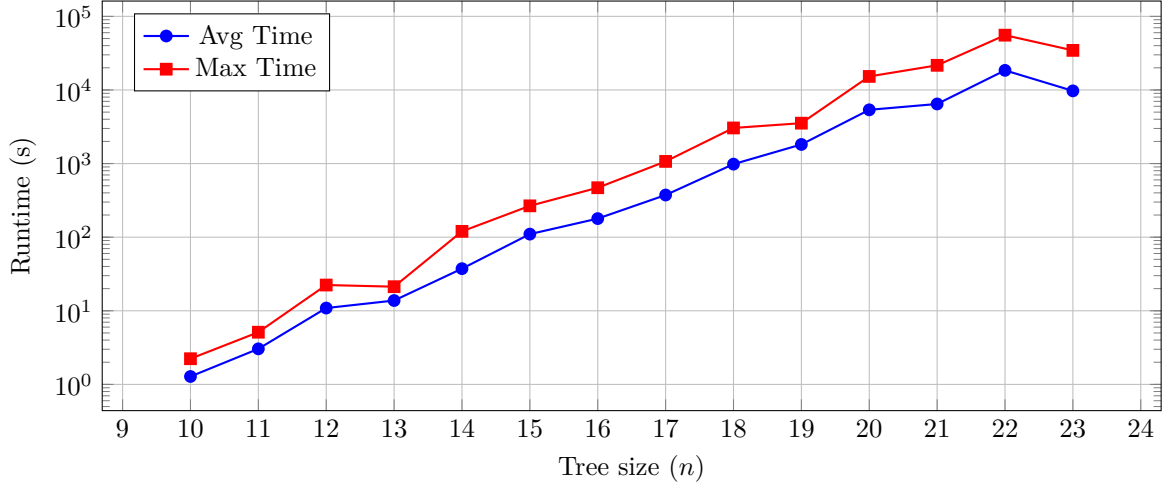


Figure 4.63: Random bounded diameter trees with exponential costs - exact algorithm: runtime comparison

$O(\log n / \log \log n)$ -Approximation Algorithm

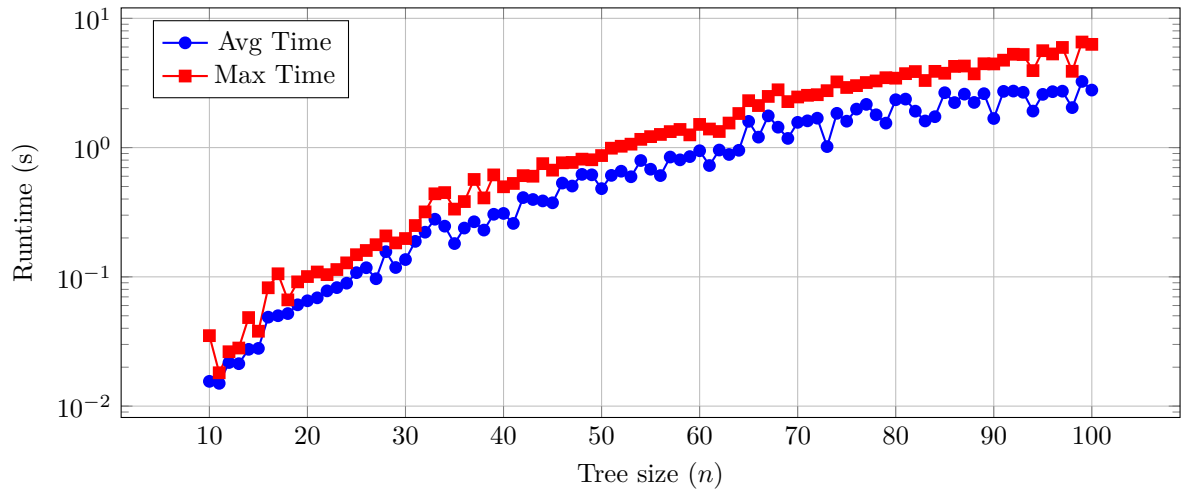


Figure 4.64: Random bounded diameter trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime

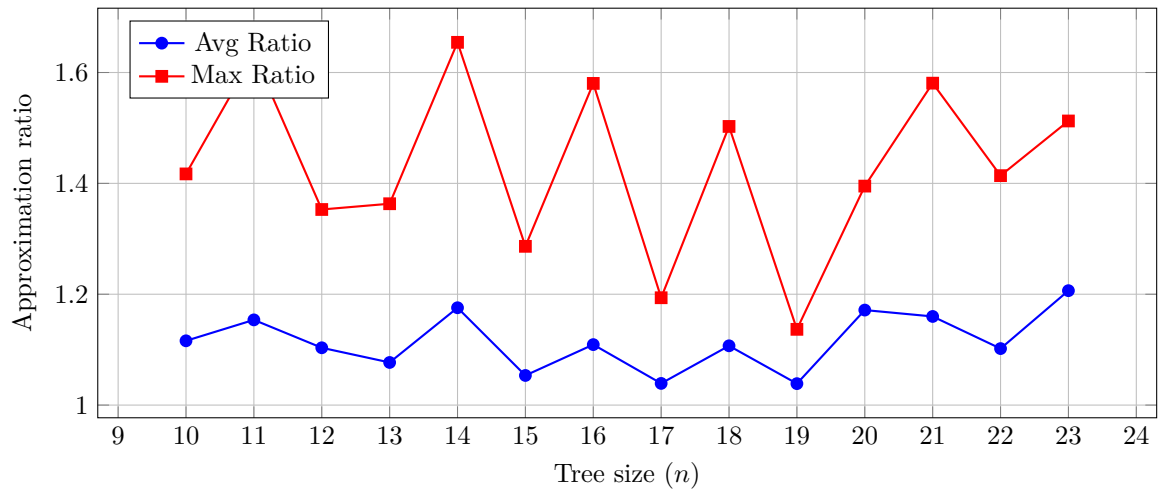


Figure 4.65: Random bounded diameter trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality

Table 4.40: Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\log n / \log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	0.015544	0.035083	1.115963	1.417065
11	0.015002	0.018113	1.153715	1.632655
12	0.021741	0.026340	1.103393	1.352873
13	0.021294	0.028142	1.076891	1.363255
14	0.027492	0.048417	1.175595	1.654453
15	0.027959	0.037970	1.053432	1.286400
16	0.048757	0.082332	1.109097	1.580365
17	0.050042	0.105627	1.039087	1.193657
18	0.051992	0.066350	1.106942	1.502707
19	0.060786	0.091474	1.038682	1.136643
20	0.065284	0.100524	1.171407	1.395069
21	0.068832	0.109037	1.159899	1.580795
22	0.077915	0.104005	1.101927	1.413946
23	0.082563	0.113906	1.206426	1.512570
24	0.089314	0.128303	N/A	N/A
25	0.107731	0.148374	N/A	N/A
26	0.117631	0.159913	N/A	N/A
27	0.096717	0.177339	N/A	N/A
28	0.156546	0.207351	N/A	N/A
29	0.118101	0.183093	N/A	N/A
30	0.136082	0.197905	N/A	N/A
31	0.188196	0.249438	N/A	N/A
32	0.221558	0.318184	N/A	N/A
33	0.279119	0.439034	N/A	N/A
34	0.246677	0.448620	N/A	N/A
35	0.180960	0.334286	N/A	N/A
36	0.238498	0.382393	N/A	N/A
37	0.266818	0.566792	N/A	N/A
38	0.230055	0.408031	N/A	N/A
39	0.304829	0.614860	N/A	N/A
40	0.309389	0.497610	N/A	N/A
41	0.258987	0.527978	N/A	N/A
42	0.410323	0.606614	N/A	N/A
43	0.396682	0.600452	N/A	N/A
44	0.386796	0.751415	N/A	N/A
45	0.374138	0.669208	N/A	N/A
46	0.531448	0.762791	N/A	N/A
47	0.504061	0.767506	N/A	N/A
48	0.620714	0.813997	N/A	N/A
49	0.615761	0.804984	N/A	N/A
50	0.481354	0.867557	N/A	N/A
51	0.609556	0.989398	N/A	N/A
52	0.656396	1.024278	N/A	N/A
53	0.594939	1.062943	N/A	N/A
54	0.794373	1.158195	N/A	N/A
55	0.680189	1.215714	N/A	N/A
56	0.607428	1.263826	N/A	N/A
57	0.844935	1.329423	N/A	N/A
58	0.803902	1.381290	N/A	N/A
59	0.851865	1.254500	N/A	N/A
60	0.945630	1.511116	N/A	N/A
61	0.726750	1.392133	N/A	N/A
62	0.957275	1.331783	N/A	N/A
63	0.884533	1.548206	N/A	N/A
64	0.952991	1.835053	N/A	N/A
65	1.595667	2.306480	N/A	N/A
66	1.204790	2.114547	N/A	N/A
67	1.760421	2.487999	N/A	N/A
68	1.438525	2.799687	N/A	N/A
69	1.177469	2.268392	N/A	N/A
70	1.567740	2.463099	N/A	N/A
71	1.610581	2.538139	N/A	N/A
72	1.688444	2.565555	N/A	N/A
73	1.018243	2.756361	N/A	N/A
74	1.841257	3.223444	N/A	N/A
75	1.600508	2.917482	N/A	N/A
76	1.984953	3.015818	N/A	N/A
77	2.159282	3.186675	N/A	N/A
78	1.793727	3.278087	N/A	N/A
79	1.546021	3.472630	N/A	N/A
80	2.345291	3.444642	N/A	N/A
81	2.372228	3.733331	N/A	N/A
82	1.913488	3.874963	N/A	N/A
83	1.606907	3.303870	N/A	N/A
84	1.734387	3.889234	N/A	N/A
85	2.657834	3.760820	N/A	N/A
86	2.229794	4.249217	N/A	N/A
87	2.591570	4.275775	N/A	N/A
88	2.234963	3.720623	N/A	N/A
89	2.612551	4.451545	N/A	N/A
90	1.679502	4.439206	N/A	N/A
91	2.722348	4.745139	N/A	N/A
92	2.736383	5.273380	N/A	N/A
93	2.677843	5.246852	N/A	N/A
94	1.916391	3.941512	N/A	N/A
95	2.581060	5.618829	N/A	N/A
96	2.707765	5.301700	N/A	N/A
97	2.731590	5.953706	N/A	N/A
98	2.039119	3.893695	N/A	N/A
99	3.249356	6.556551	N/A	N/A
100	2.785821	6.290211	N/A	N/A

$O(\sqrt{\log n})$ -Approximation Algorithm

Table 4.41: Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\sqrt{\log n})$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	1.365923	9.391802	1.084915	1.300456
11	2.001662	7.637198	1.153798	1.329548
12	1.320479	10.062322	1.088531	1.352873
13	0.688580	4.894963	1.123011	1.612684
14	0.994772	2.547614	1.135968	1.511666
15	0.662405	5.515870	1.090916	1.314895
16	22.019609	139.887932	1.072079	1.401290
17	12.723595	33.116942	1.102226	1.476146
18	15.528068	75.330255	1.111326	1.373694
19	6.203363	28.699153	1.141494	1.384143
20	4.669195	14.366929	1.142067	1.676556
21	3.209218	18.381044	1.130766	1.580795
22	5.106489	32.544227	1.084357	1.438644
23	89.171042	526.690770	1.144294	1.512570
24	10.458182	46.943540	N/A	N/A
25	12.202853	67.407558	N/A	N/A
26	23.305728	135.849603	N/A	N/A
27	3.755497	18.823919	N/A	N/A
28	21.037054	108.492413	N/A	N/A
29	4.789940	15.535137	N/A	N/A
30	252.405903	1722.186176	N/A	N/A
31	38.557091	263.716941	N/A	N/A
32	23.927254	156.199183	N/A	N/A
33	20.670348	102.220089	N/A	N/A
34	4.791602	26.495236	N/A	N/A
35	3.359176	8.128729	N/A	N/A
36	8.012382	34.351110	N/A	N/A
37	10.138553	64.400647	N/A	N/A
38	3.488150	11.121605	N/A	N/A
39	12.306544	43.367186	N/A	N/A
40	3.393986	11.358674	N/A	N/A
41	7.703399	47.166723	N/A	N/A
42	952.908057	5333.459129	N/A	N/A
43	618.748675	4144.367910	N/A	N/A
44	297.802471	1335.994804	N/A	N/A
45	60.419330	469.080283	N/A	N/A
46	2374.942395	22413.903396	N/A	N/A
47	102.543395	286.762507	N/A	N/A
48	86.663529	377.738966	N/A	N/A
49	650.867841	3973.997500	N/A	N/A
50	872.015858	5924.478073	N/A	N/A

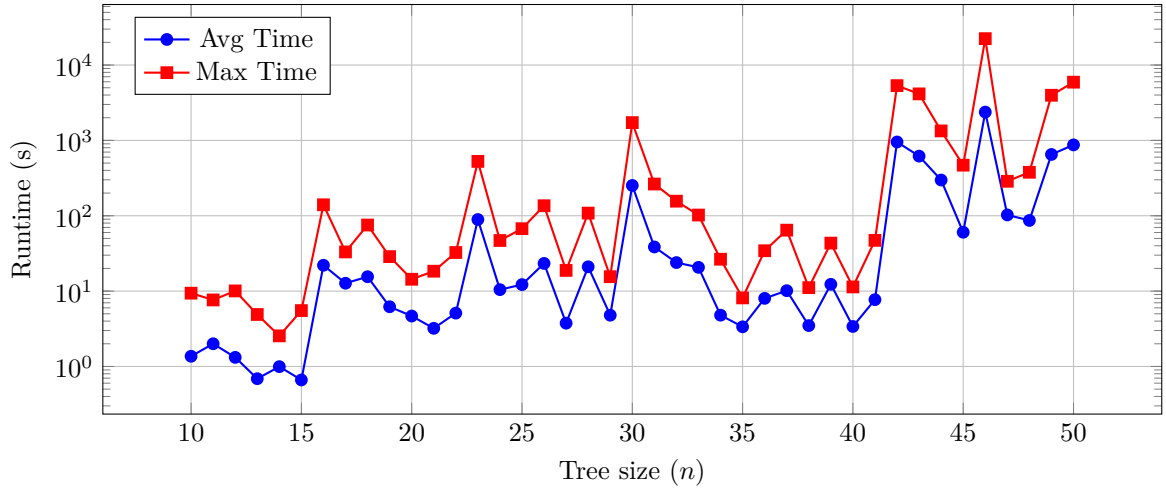


Figure 4.66: Random bounded diameter trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime

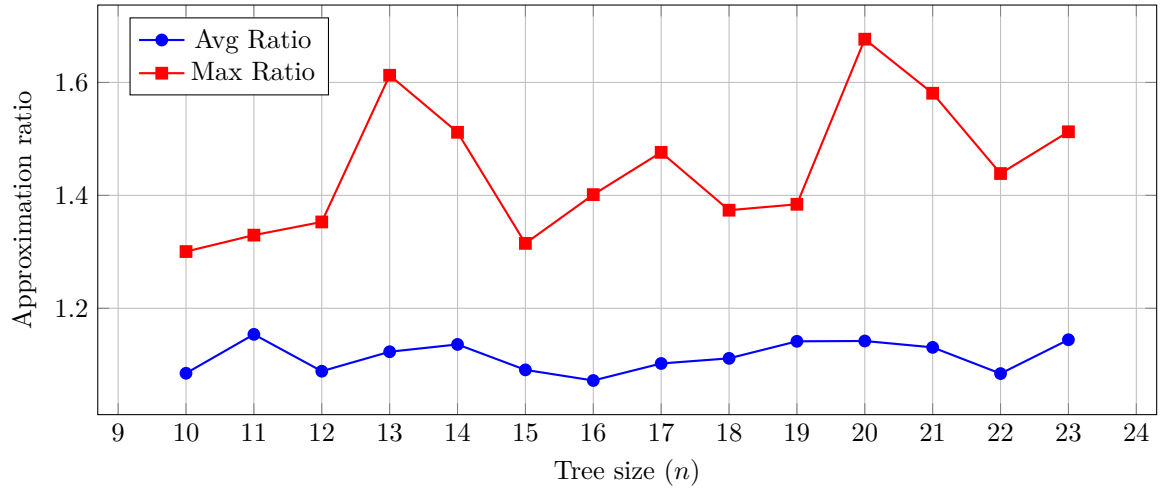


Figure 4.67: Random bounded diameter trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality

$O(\log \log n)$ -Approximation Algorithm

Table 4.42: Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\log \log n)$ -approximation algorithm

n	Avg Time (s)	Max Time (s)	Avg Ratio	Max Ratio
10	1.782975	14.183337	1.312270	1.702688
11	22.072930	131.602413	1.270328	1.686048
12	10.067423	62.703683	1.173534	1.795234
13	10.849034	57.148510	1.328003	1.723272
14	94.129849	852.420933	1.311120	1.991719
15	8.813752	50.991524	1.235900	1.653372
16	277.900910	1992.723006	1.258917	1.657164
17	155.299786	1247.235134	1.653159	2.244013
18	28.139140	257.575366	1.696581	1.966669
19	66.976816	434.965847	1.531624	2.153838
20	288.915824	2850.395236	1.694048	2.383780
21	66.413717	333.190025	1.449795	2.113268
22	3.413461	20.338819	1.692985	2.192194
23	419.496865	3251.748559	1.854477	2.272677
24	228.499456	1467.377345	N/A	N/A
25	207.693467	1005.376580	N/A	N/A
26	809.770890	8034.712137	N/A	N/A
27	611.317996	5832.872161	N/A	N/A
28	1439.597606	5239.430192	N/A	N/A
29	111.139001	1034.176115	N/A	N/A
30	359.993051	2812.259733	N/A	N/A
31	1390.296885	10823.344476	N/A	N/A
32	165.494974	994.756247	N/A	N/A
33	7391.845795	53055.822888	N/A	N/A
34	82.640240	321.645762	N/A	N/A
35	907.067100	3306.168010	N/A	N/A
36	280.030357	1040.227597	N/A	N/A

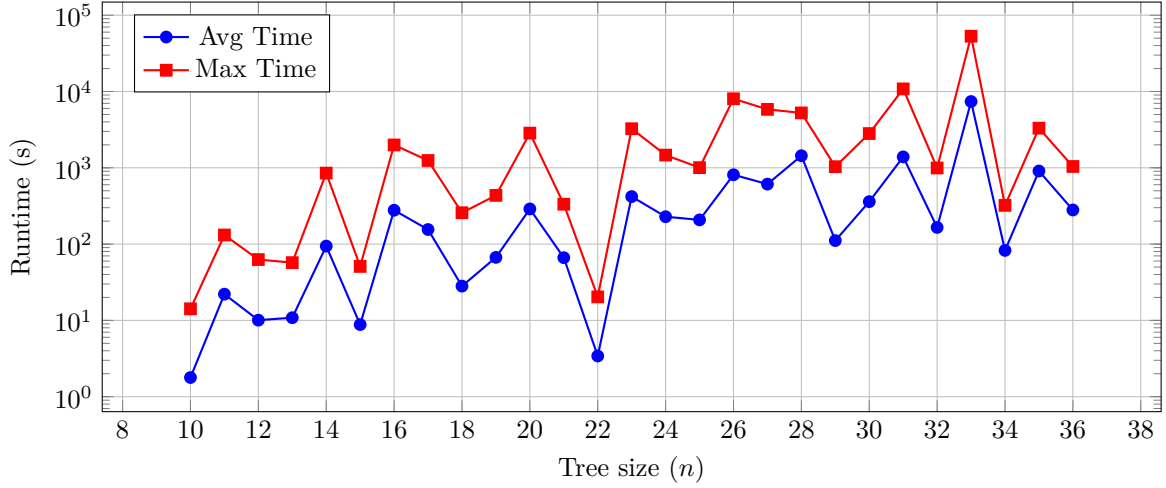


Figure 4.68: Random bounded diameter trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime

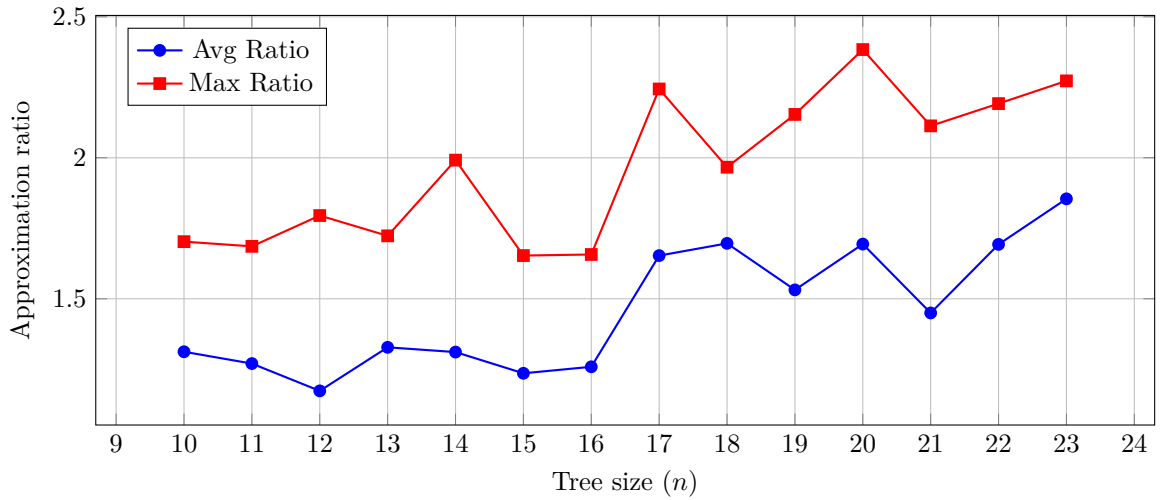


Figure 4.69: Random bounded diameter trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality

We see that the $O(\log \log n)$ -approximation algorithm achieves approximation ratios above 2 and was outperformed by the two other algorithms.

4.3.10 Exponential Runtime Growth Analysis of the Exact Algorithm

During the execution of the exact dynamic programming algorithm from Section 3.2.1, it could be noticed that the running time of the procedure behaved better than $O(2^n)$. We decided to find the value of b such that the function $T(n) = b^n$ best approximates the running time of the procedure. The coefficient of determination R^2 indicates the quality of the exponential fit. It is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where y_i are the observed runtime values, $\hat{y}_i$ are the predicted values from the model $T(n) = b^n$, and $\bar{y}$ is the mean of observed values. Values of R^2 close to 1 suggest that the exponential model accurately describes the runtime growth, while values close to 0 indicate poor fit. We calculated the values of b for which the value of R^2 is maximized:

Table 4.43: Exponential fit for exact DP algorithm runtime

Variant	Distribution	Average CPU Time		Maximum CPU Time	
		b	R^2	b	R^2
Random	Binomial	1.341	0.924	1.381	0.734
	Exponential	1.348	0.606	1.382	0.465
	Uniform	1.350	0.918	1.386	0.598
Random Degree 3	Binomial	1.355	0.918	1.377	0.961
	Exponential	1.358	0.978	1.379	0.940
	Uniform	1.353	0.975	1.373	0.967
Random Diameter 6	Binomial	1.518	0.817	1.600	0.251
	Exponential	1.519	0.717	1.600	0.764
	Uniform	1.509	0.882	1.610	0.949

We observe that:

- The bases b are all significantly smaller than 2, ranging from approximately 1.34 to 1.61, indicating that the algorithm performs better than the worst-case $O(2^n)$ growth in practice for these instances.
- Random trees with bounded diameter (Diameter 6) display the highest growth bases ($b \approx 1.51$ - 1.61).
- Other tree classes exhibited value of b oscillating around $b \approx 1.34$ - 1.39 which is significantly better than the worst case $O(2^n)$ running time.
- For each class of trees the values of b are relatively consistent across different cost distributions (Uniform, Binomial, Exponential), suggesting that the distribution of query costs has a limited impact on the exponential growth rate of the algorithm's runtime and that perhaps the values of b might stabilize for larger input sizes.

Chapter 5

Summary

This thesis presented an experimental analysis of the worst-case Graph Search Problem in tree graphs. We explored various algorithms designed to tackle the Tree Search Problem with both uniform and non-uniform query costs. The main results of this work is that the computational experiments confirm the theoretical claims regarding both the approximation ratios achieved by the presented algorithms as well as their running times. We also showed new theoretical results for the problem including a redesigned QPTAS for the non-uniform query cost model as well as an $O(\log \log n)$ -approximation algorithm running in parametrized time. Moreover, we introduced three field notation enabling unified and sparse way of denoting various Graph Search Problem variants.

For the case of the non-uniform cost model, the costs of the solutions built by all algorithms were relatively low, and no particular algorithm significantly outperformed the others. Importantly, none ever produced an output of cost worse than $3 \cdot \text{OPT}$. This suggests that on random instances most heuristics perform well. However it should be noted that the least algorithmically complex procedure, i. e. the $O(\log n / \log \log n)$ -approximation algorithm usually achieved slightly better quality than the others. This counterintuitive result suggests that on such instances, the most greedy-like algorithms perform best. This is visible in comparison to the $O(\log \log n)$ approximation algorithm achieving the worst approximation ratios which is probably due to the fact that it processes many cost levels and each such level incurs additional $O(\text{OPT})$ cost. Therefore, despite its better asymptotic guarantee, it often performs slightly worse than the $O(\log n / \log \log n)$ and $O(\sqrt{\log n})$ -approximation algorithms.

The other important conclusion drawn from the experimental analysis is that the state-of-the-art approximation algorithms for the non-uniform query cost model are of practical use only for relatively small instances, up to several dozens of vertices. Beyond that, the $O(\log n / \log \log n)$ -approximation algorithm is the only one with acceptable running time. Since all of the tested algorithms provided similar quality of the solutions the $O(\log n / \log \log n)$ -approximation algorithm is to be preferred for practical usage. The $O(\log \log n)$ -approximation algorithm, while theoretically interesting, is impractical for real-world applications due to its high computational complexity. We observe that no particularly hard instances (in terms of approximation quality) were found during the execution of our experiments. This suggests that they might be very scarce. It would be therefore of interest to find such instances and conduct further studies evaluating the quality of the above procedures on them.

Another advantage of the $O(\log n / \log \log n)$ -approximation algorithm is the stability of its running time. No sudden running time spikes were observed for the procedure. In contrast, both of the procedures based on the QPTAS exhibited significant variance in their running times, even on instances of the same size. This is due to the fact that whenever the QPTAS was called for a tree with large value of $\text{rank}(T)$, the running time increased significantly. Note that this value can be large even for trees with very simple structures, for example paths are the trees with the largest possible value of $\text{rank}(T)$ equal to $\lfloor \log(n) \rfloor + 1$. Observing the behaviour of the algorithms manually we noticed that the auxiliary trees for which the QPTAS was called usually had a path-like structure, which significantly increased the running time of the algorithms.

For the uniform query cost model we observe that the naive $O(n \log n)$ implementation of the linear time algorithm based on calculating the ranking coloring performed differently among the three considered classes of trees. On randomly generated and bounded degree instances the observed running times were almost linear while for the bounded diameter instances they matched the upper bound of $O(n \log n)$. On the other hand for the bounded diameter instances the depth of the decision tree was always at most 4. For randomly generated and bounded degree instances the depth of the decision trees produced by the algorithm behaved logarithmically in n . While for bounded degree trees

this is to be expected, for randomly generated instances this behavior of $\text{rank}(T)$ is more surprising. We conjecture that given a random tree T on n vertices, $\text{rank}(T) = \Theta(\log n)$.

The bound $\text{rank}(T) \leq \left\lceil \frac{\text{diam}(T)+1}{2} \right\rceil$ also shows that for trees with bounded diameter, the running time of the QPTAS for $T||V, c||C_{max}$ is in fact polynomial in n and the algorithm becomes a PTAS running in $n^{O(\text{diam}(T)/\epsilon^2)}$. It would be interesting to verify whether a similar result can be obtained for trees with bounded degree as well. Since for such trees the ranking is always of order $\Theta(\log n)$, a new technique would have to be designed. However, it is possible that other structural properties of such trees can be exploited to obtain constant factor approximation for such trees.

Another interesting observation is the surprisingly good performance of the $O(2^n n)$ time exact algorithm. Despite its exponential time complexity, it was able to solve instances with up to 33 vertices within reasonable time limits. By performing a generic regression of the running time of the algorithm we observe that its practical running time tightly matched an exponential function with base $b \approx 1.34$ - 1.61 . Upon careful examination of the exact algorithm one may observe that in fact the upper bound on its running time is the number of connected subtrees of T . Since we generated our random trees using Prüfer sequences, by [KP19] we know that the expected number of connected subtrees of a tree generated in this way is of order $O(1.4196^n)$ which matches the experimentally obtained values. Although this is still exponential, it is significantly better than the worst-case bound of $O(2^n)$. This suggests that the exact algorithm may be practical for small to medium-sized instances, especially when the input trees have certain structural properties that limit the number of connected subtrees. In recent years many NP-hard problems, such as the Traveling Salesman Problem or the Steiner Tree Problem, have seen significant improvements in their exact exponential time algorithms [Gas10]. It is therefore an interesting research direction to investigate whether similar improvements can be obtained for the Tree Search Problem as well.

The non-uniform cost model enforces the use of complex algorithmic techniques, and non-polynomial time subroutines in order to achieve satisfying approximation guarantees. For practical usage this provides a two-fold challenge: on the one hand, it makes the algorithms hard to implement and debug, on the other hand, it limits their practical usability beyond small instances. Due to this, the experimental analysis conducted in this thesis required significant effort. This was particularly noticeable during the implementation of the QPTAS for $T||V, c||C_{max}$ both in terms of the used data structures with operations on them as well as the algorithm itself, consisting of few cases of a complex dynamic programming procedure. It is interesting, whether it is possible to derive similar results, using simpler techniques. In particular, a further research direction would be to simplify the QPTAS both in terms of the large constant in the exponent and the algorithm itself. A central question in this line of research is whether it is possible to derive constant factor approximation guarantee for the non-uniform query cost model or ideally, a PTAS. As of right now, no inapproximability results are known for this problem, besides a trivial non-existence of an absolute approximation algorithm. Therefore, this problem is included among the specific list of problems for which an $O(\sqrt{\log n})$ -approximation is known, but there is no lower bound matching this result. A similar issue is prevalent among many separation problems such as treewidth, vertex separators and edge cuts [ARV09; KNS; FHL05] for which an $O(\sqrt{\log n})$ -approximation algorithms are known, but no lower bounds beyond constant factor inapproximability are known. Interestingly, these problems are connected to Tree Search Problem, since they can be used to derive relatively simple constant factor approximation algorithms for $T||V, c, w||\sum C_i$ and $T||E, c, w||\sum C_i$ [Szy25a; Das16; CC17].

A future way to decrease the approximation ratio would be to devise a dynamic program which stores less information for each box of the boxed decision tree. As of right now each of $\text{rank}(T)$ boxes require enumerating $O(n)$ values of load. Reducing this amount to $\text{poly}(\log n)$ would immediately yield a polynomial time algorithm with approximation ratio of $O(\log \log n)$. However, we think that this is a very challenging task, since the load values are crucial to ensure that the merging and rotating operations are performed correctly. A different way to achieve such approximation ratio would be to design an algorithm similar to the one used for the k -up modular cost function, recursively splitting the cost levels into $O(\log \log n)$ levels and processing each separately. However, it is not clear how to perform merging of such strategies in polynomial (or almost polynomial) time simultaneously ensuring that the cost incurred at each level is $O(\text{OPT})$.

Bibliography

- [Ang18] Haris Angelidakis. “Shortest path queries, graph partitioning and covering problems in worst and beyond worst case settings”. In: *ArXiv* abs/1807.09389 (2018). URL: <https://api.semanticscholar.org/CorpusID:51718679>.
- [ARV09] Sanjeev Arora, Satish Rao, and Umesh Vazirani. “Expander flows, geometric embeddings and graph partitioning”. In: *J. ACM* 56.2 (Apr. 2009). ISSN: 0004-5411. DOI: 10.1145/1502793.1502794. URL: <https://doi.org/10.1145/1502793.1502794>.
- [BBS12] Gowtham Bellala, Suresh K. Bhavnani, and Clayton Scott. “Group-Based Active Query Selection for Rapid Diagnosis in Time-Critical Situations”. In: *IEEE Transactions on Information Theory* 58.1 (2012), pp. 459–478. DOI: 10.1109/TIT.2011.2169296.
- [BDO23] Piotr Borowiecki, Dariusz Dereniowski, and Dorota Osula. “The complexity of bicriteria tree-depth”. In: *Theoretical Computer Science* 947 (2023), p. 113682. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2022.12.032>. URL: <https://www.sciencedirect.com/science/article/pii/S0304397522007666>.
- [Ber+22] Benjamin Berendsohn et al. *Fast approximation of search trees on trees with centroid trees*. Sept. 2022. DOI: 10.48550/arXiv.2209.08024.
- [Ber24] Benjamin Aram Berendsohn. “Search trees on graphs”. Dissertation. 2024. URL: <http://dx.doi.org/10.17169/refubium-45704>.
- [BH08] Michael Ben-Or and Avinatan Hassidim. “The Bayesian Learner is Optimal for Noisy Binary Search (and Pretty Good for Quantum as Well)”. In: Oct. 2008, pp. 221–230. DOI: 10.1109/FQCS.2008.58.
- [BK22] Benjamin Berendsohn and László Kozma. “Splay trees on trees”. In: Jan. 2022, pp. 1875–1900. ISBN: 978-1-61197-707-3. DOI: 10.1137/1.9781611977073.75.
- [Bod+95] H.L. Bodlaender et al. “Approximating Treewidth, Pathwidth, Frontsize, and Shortest Elimination Tree”. In: *Journal of Algorithms* 18.2 (1995), pp. 238–255. ISSN: 0196-6774. DOI: <https://doi.org/10.1006/jagm.1995.1009>. URL: <https://www.sciencedirect.com/science/article/pii/S0196677485710097>.
- [Bod+98] Hans L. Bodlaender et al. “Rankings of Graphs”. In: *SIAM Journal on Discrete Mathematics* 11.1 (1998), pp. 168–181. DOI: 10.1137/S0895480195282550. eprint: <https://doi.org/10.1137/S0895480195282550>. URL: <https://doi.org/10.1137/S0895480195282550>.
- [Car+04] Renato Carmo et al. “Searching in Random Partially Ordered Sets”. In: vol. 321. June 2004, pp. 41–57. ISBN: 978-3-540-43400-9. DOI: 10.1007/3-540-45995-2_27.
- [CC17] Moses Charikar and Vaggos Chatziafratis. “Approximate hierarchical clustering via sparsest cut and spreading metrics”. In: *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA ’17. Barcelona, Spain: Society for Industrial and Applied Mathematics, 2017, pp. 841–854.
- [CDL22] Julien Courtiel, Paul Dorbec, and Romain Lecoq. “Theoretical Analysis of git bisect”. In: *LATIN 2022: Theoretical Informatics*. Ed. by Armando Castañeda and Francisco Rodríguez-Henríquez. Cham: Springer International Publishing, 2022, pp. 157–171. ISBN: 978-3-031-20624-5.
- [Cha+21] Hong Chang et al. “Conflict-free connection of trees”. In: *Journal of Combinatorial Optimization* 42.3 (2021), pp. 340–353. DOI: 10.1007/s10878-018-0363-x.
- [Cic+12] Ferdinando Cicalese et al. “The binary identification problem for weighted trees”. In: *Theoretical Computer Science* 459 (2012), pp. 100–112. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2012.06.023>.

- [Cic+14] Ferdinando Cicalese et al. “Improved Approximation Algorithms for the Average-Case Tree Searching Problem”. In: *Algorithmica* 68 (Apr. 2014). DOI: 10.1007/s00453-012-9715-6.
- [Cic+16] Ferdinando Cicalese et al. “On the tree search problem with non-uniform costs”. In: *Theoretical Computer Science* 647 (2016), pp. 22–32. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2016.07.019>.
- [CLS14] Ferdinando Cicalese, Eduardo Laber, and Aline Medeiros Saettler. “Diagnosis determination: decision trees optimizing simultaneously worst and expected testing cost”. In: *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32*. ICML’14. Beijing, China: JMLR.org, 2014, pp. I–414–I–422.
- [CLS16] Ferdinando Cicalese, Eduardo Laber, and Aline Saettler. “Decision Trees for Function Evaluation: Simultaneous Optimization of Worst and Expected Cost”. In: *Algorithmica* 79 (Oct. 2016). DOI: 10.1007/s00453-016-0225-9.
- [Das04] Sanjoy Dasgupta. “Analysis of a greedy active learning strategy”. In: *Advances in Neural Information Processing Systems*. Ed. by L. Saul, Y. Weiss, and L. Bottou. Vol. 17. MIT Press, 2004. URL: https://proceedings.neurips.cc/paper_files/paper/2004/file/c61fbef63df5ff317aecdc3670094472-Paper.pdf.
- [Das16] Sanjoy Dasgupta. “A cost function for similarity-based hierarchical clustering”. In: *Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing*. STOC ’16. Cambridge, MA, USA: Association for Computing Machinery, 2016, pp. 118–127. ISBN: 9781450341325. DOI: 10.1145/2897518.2897527. URL: <https://doi.org/10.1145/2897518.2897527>.
- [Der+17] Dariusz Dereniowski et al. “Approximation Strategies for Generalized Binary Search in Weighted Trees”. In: *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*. Ed. by Ioannis Chatzigiannakis et al. Vol. 80. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017, 84:1–84:14. ISBN: 978-3-95977-041-5. DOI: 10.4230/LIPIcs.ICALP.2017.84.
- [Der00] Dariusz Dereniowski. “Minimum edge ranking spanning trees of threshold graphs”. In: *Discrete Applied Mathematics* 103.1-3 (2000), pp. 35–40. DOI: 10.1016/S0166-218X(99)00212-7.
- [Der01] Dariusz Dereniowski. “On minimum edge ranking spanning trees”. In: *Theoretical Computer Science* 262.1-2 (2001), pp. 411–425. DOI: 10.1016/S0304-3975(00)00415-6.
- [Der03] Dariusz Dereniowski. “Cholesky factorization of matrices with a given vertex ranking”. In: *Linear Algebra and its Applications* 365 (2003), pp. 215–239. DOI: 10.1016/S0024-3795(02)00522-9.
- [Der06] Dariusz Dereniowski. “Edge ranking of weighted trees”. In: *Discrete Applied Mathematics* 154.8 (2006), pp. 1198–1209. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2005.11.005>.
- [Der08] Dariusz Dereniowski. “Edge ranking and searching in partial orders”. In: *Discrete Applied Mathematics* 156.13 (2008). Fifth International Conference on Graphs and Optimization, pp. 2493–2500. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2008.03.007>.
- [DGP23] Dariusz Dereniowski, Przemysław Gordinowicz, and Paweł Prałat. “Edge and Pair Queries—Random Graphs and Complexity”. In: *The Electronic Journal of Combinatorics* 30.2 (2023). DOI: 10.37236/11159. URL: <https://www.combinatorics.org/ojs/index.php/eljc/article/view/v30i2p34>.
- [DGW24] Dariusz Dereniowski, Przemysław Gordinowicz, and Karolina Wróbel. “On multidimensional generalization of binary search”. In: *ArXiv abs/2404.13193* (2024). URL: <https://api.semanticscholar.org/CorpusID:269293685>.
- [DK06] Dariusz Dereniowski and Marek Kubale. “Efficient Parallel Query Processing by Graph Ranking”. In: *Fundam. Inform.* 69 (Feb. 2006), pp. 273–285. DOI: 10.3233/FUN-2006-69302.

- [DK97] Dariusz Dereniowski and Marek Kubale. “Parallel assembly of modular products”. In: *International Journal of Production Research* 35.3 (1997), pp. 675–689. DOI: 10.1080/002075497195515.
- [DL05] Sanjoy Dasgupta and Philip M. Long. “Hierarchical clustering: Objective functions and algorithms”. In: *Journal of Computer and System Sciences*. Vol. 70. 4. Elsevier, 2005, pp. 555–569.
- [DLU21] Dariusz Dereniowski, Aleksander Łukasiewicz, and Przemysław Uznański. “An Efficient Noisy Binary Search in Graphs via Median Approximation”. In: *Combinatorial Algorithms*. Ed. by Paola Flocchini and Lucia Moura. Cham: Springer International Publishing, 2021, pp. 265–281. ISBN: 978-3-030-79987-8.
- [DLU25] Dariusz Dereniowski, Aleksander Łukasiewicz, and Przemysław Uznański. “Noisy (Binary) Searching: Simple, Fast and Correct”. In: *42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025)*. Ed. by Olaf Beyersdorff et al. Vol. 327. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 29:1–29:18. ISBN: 978-3-95977-365-2. DOI: 10.4230/LIPIcs.STACS.2025.29. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2025.29>.
- [DMS19] Argyrios Deligkas, George B. Mertzios, and Paul G. Spirakis. “Binary Search in Graphs Revisited”. In: *Algorithmica* 81.5 (May 2019), pp. 1757–1780. ISSN: 1432-0541. DOI: 10.1007/s00453-018-0501-y.
- [DN06] Dariusz Dereniowski and Adam Nadolski. “Vertex rankings of chordal graphs and weighted trees”. In: *Information Processing Letters* 98.3 (2006), pp. 96–100. ISSN: 0020-0190. DOI: <https://doi.org/10.1016/j.ipl.2005.12.006>.
- [DW22] Dariusz Dereniowski and Izajasz Wrosz. “Constant-Factor Approximation Algorithm for Binary Search in Trees with Monotonic Query Times”. In: *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022)*. Ed. by Stefan Szeider, Robert Ganian, and Alexandra Silva. Vol. 241. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 42:1–42:15. ISBN: 978-3-95977-256-3. DOI: 10.4230/LIPIcs.MFCS.2022.42.
- [DW24] Dariusz Dereniowski and Izajasz Wrosz. *Searching in trees with monotonic query times*. 2024. arXiv: 2401.13747 [cs.DS]. URL: <https://arxiv.org/abs/2401.13747>.
- [EKS16] Ehsan Emamjomeh-Zadeh, David Kempe, and Vikrant Singhal. “Deterministic and probabilistic binary search in graphs”. In: June 2016, pp. 519–532. DOI: 10.1145/2897518.2897656.
- [FHL05] Uriel Feige, MohammadTaghi Hajiaghayi, and James R. Lee. “Improved approximation algorithms for minimum-weight vertex separators”. In: *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. STOC ’05. Baltimore, MD, USA: Association for Computing Machinery, 2005, pp. 563–572. ISBN: 1581139608. DOI: 10.1145/1060590.1060674. URL: <https://doi.org/10.1145/1060590.1060674>.
- [Gas10] Serge Gaspers. *Exponential Time Algorithms - Structures, Measures, and Bounds*. July 2010. ISBN: 978-3-639-21825-1.
- [GHT12] Archontia C. Giannopoulou, Paul Hunter, and Dimitrios M. Thilikos. “LIFO-search: A min–max theorem and a searching game for cycle-rank and tree-depth”. In: *Discrete Applied Mathematics* 160.15 (2012), pp. 2089–2097. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2012.03.015>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X12001199>.
- [GKR10] Daniel Golovin, Andreas Krause, and Debajyoti Ray. “Near-optimal Bayesian active learning with noisy observations”. In: *Proceedings of the 24th International Conference on Neural Information Processing Systems - Volume 1*. NIPS’10. Vancouver, British Columbia, Canada: Curran Associates Inc., 2010, pp. 766–774.

- [GNR10] Anupam Gupta, Viswanath Nagarajan, and R. Ravi. “Approximation Algorithms for Optimal Decision Trees and Adaptive TSP Problems”. In: *Automata, Languages and Programming*. Ed. by Samson Abramsky et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 690–701. ISBN: 978-3-642-14165-2.
- [Gra69] R. L. Graham. “Bounds on Multiprocessing Timing Anomalies”. In: *SIAM Journal on Applied Mathematics* 17.2 (1969), pp. 416–429. DOI: 10.1137/0117039. eprint: <https://doi.org/10.1137/0117039>. URL: <https://doi.org/10.1137/0117039>.
- [Høg+21] Svein Høgemo et al. “On Dasgupta’s Hierarchical Clustering Objective and Its Relation to Other Graph Parameters”. In: *Fundamentals of Computation Theory*. Ed. by Evripidis Bampis and Aris Pagourtzis. Cham: Springer International Publishing, 2021, pp. 287–300. ISBN: 978-3-030-86593-1.
- [Høg24] Svein Høgemo. “Tight Approximation Bounds on a Simple Algorithm for Minimum Average Search Time in Trees”. In: *ArXiv abs/2402.05560* (2024). URL: <https://api.semanticscholar.org/CorpusID:267547530>.
- [Jac+10] Tobias Jacobs et al. “On the Complexity of Searching in Trees: Average-Case Minimization”. In: *Automata, Languages and Programming*. Ed. by Samson Abramsky et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 527–539. ISBN: 978-3-642-14165-2.
- [KMS95] Meir Katchalski, William McCuaig, and Suzanne Seager. “Ordered colourings”. In: *Discrete Mathematics* 142.1 (1995), pp. 141–154. ISSN: 0012-365X. DOI: [https://doi.org/10.1016/0012-365X\(93\)E0216-Q](https://doi.org/10.1016/0012-365X(93)E0216-Q). URL: <https://www.sciencedirect.com/science/article/pii/0012365X93E0216Q>.
- [KNS] Robert Krauthgamer, Joseph (Seffi) Naor, and Roy Schwartz. “Partitioning Graphs into Balanced Components”. In: *Proceedings of the 2009 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 942–949. DOI: 10.1137/1.9781611973068.102. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611973068.102>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611973068.102>.
- [Knu73] Donald Knuth. *The Art Of Computer Programming, vol. 3: Sorting And Searching*. Addison-Wesley, 1973, pp. 391–392.
- [KP19] Bogumił Kamiński and Paweł Prałat. “Sub-trees of a random tree”. In: *Discrete Applied Mathematics* 268 (2019), pp. 119–129. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2019.05.003>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X19302549>.
- [KPB99] S. Rao Kosaraju, Teresa M. Przytycka, and Ryan Borgstrom. “On an Optimal Split Tree Problem”. In: *Algorithms and Data Structures*. Ed. by Frank Dehne et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 157–168. ISBN: 978-3-540-48447-9.
- [KZ13] Amin Karbasi and Morteza Zadimoghaddam. “Constrained Binary Identification Problems”. In: *30th International Symposium on Theoretical Aspects of Computer Science (STACS)*. Vol. 20. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2013, pp. 550–561. DOI: 10.4230/LIPIcs.STACS.2013.550. URL: <https://drops.dagstuhl.de/opus/volltexte/2013/3942>.
- [Leu89] Joseph Y-T. Leung. “Bin packing with restricted piece sizes”. In: *Information Processing Letters* 31.3 (1989), pp. 145–149. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(89\)90223-8](https://doi.org/10.1016/0020-0190(89)90223-8). URL: <https://www.sciencedirect.com/science/article/pii/0020019089902238>.
- [LLM] Ray Li, Percy Liang, and Stephen Mussmann. “A Tight Analysis of Greedy Yields Subexponential Time Approximation for Uniform Decision Tree”. In: *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 102–121. DOI: 10.1137/1.9781611975994.7. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611975994.7>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611975994.7>.

- [LN04] Eduardo S. Laber and Loana Tito Nogueira. “On the hardness of the minimum height decision tree problem”. In: *Discrete Applied Mathematics* 144.1 (2004). Discrete Mathematics and Data Mining, pp. 209–212. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2004.06.002>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X04001787>.
- [LY98] Tak Wah Lam and Fung Ling Yue. “Edge ranking of graphs is hard”. In: *Discrete Applied Mathematics* 85.1 (1998), pp. 71–86. ISSN: 0166-218X. DOI: [https://doi.org/10.1016/S0166-218X\(98\)00029-8](https://doi.org/10.1016/S0166-218X(98)00029-8).
- [MOW08] Shay Mozes, Krzysztof Onak, and Oren Weimann. “Finding an optimal tree searching strategy in linear time”. In: *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2008, San Francisco, California, USA, January 20-22, 2008*. Ed. by Shang-Hua Teng. SIAM, 2008, pp. 1096–1105. URL: <http://dl.acm.org/citation.cfm?id=1347082.1347202>.
- [NO06] Jaroslav Nešetřil and Patrice Ossona de Mendez. “Tree-depth, subgraph coloring and homomorphism bounds”. In: *European Journal of Combinatorics* 27.6 (2006), pp. 1022–1041. ISSN: 0195-6698. DOI: <https://doi.org/10.1016/j.ejc.2005.01.010>. URL: <https://www.sciencedirect.com/science/article/pii/S0195669805000570>.
- [OP06] Krzysztof Onak and Pawel Parys. “Generalization of Binary Search: Searching in Trees and Forest-Like Partial Orders”. In: *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*. 2006, pp. 379–388. DOI: 10.1109/FOCS.2006.32.
- [Pot88] Alex Pothen. *The Complexity of Optimal Elimination Trees*. Technical Report CS-88-16. Penn State University CS-88-16. Pennsylvania State University, Department of Computer Science, Apr. 1988.
- [Riv+75] Ronald L. Rivest et al. “Optimal Search in Trees”. In: *SIAM Journal on Computing* 4.4 (1975), pp. 542–547. DOI: 10.1137/0204045.
- [Sah76] Sartaj K. Sahni. “Algorithms for Scheduling Independent Tasks”. In: *J. ACM* 23.1 (Jan. 1976), pp. 116–127. ISSN: 0004-5411. DOI: 10.1145/321921.321934. URL: <https://doi.org/10.1145/321921.321934>.
- [Sch89] Alejandro A. Schäffer. “Optimal node ranking of trees in linear time”. In: *Information Processing Letters* 33.2 (1989), pp. 91–96. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(89\)90161-0](https://doi.org/10.1016/0020-0190(89)90161-0).
- [SDG92] Arunabha Sen, Haiyong Deng, and Sumanta Guha. “On a graph partition problem with application to VLSI layout”. In: *Information Processing Letters* 43.2 (1992), pp. 87–94. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(92\)90017-P](https://doi.org/10.1016/0020-0190(92)90017-P). URL: <https://www.sciencedirect.com/science/article/pii/002001909290017P>.
- [SLC14] Aline Saettler, Eduardo Laber, and Ferdinando Cicalese. “Trading off Worst and Expected Cost in Decision Tree Problems and a Value Dependent Model”. In: (June 2014).
- [Szy25a] Michał Szyfelbein. *Approximating the Average-Case Graph Search Problem with Non-Uniform Costs*. 2025. arXiv: 2511.06564 [cs.DS]. URL: <https://arxiv.org/abs/2511.06564>.
- [Szy25b] Michał Szyfelbein. *Searching in trees with k -up-modular cost functions*. 2025. arXiv: 2504.17887 [cs.DS]. URL: <https://arxiv.org/abs/2504.17887>.

List of Figures

1.1	Binary search	7
1.2	Sample tree	8
1.3	Vertex decision tree for a tree	8
1.4	Edge decision tree for a tree	8
1.5	Tree and decision trees for it	8
3.1	Sample tree with uniform costs	19
3.2	Sample tree with uniform costs	19
3.3	Edge decision tree for a tree	19
3.4	Tree and decision trees for it	19
3.5	Example tree $\mathcal{T}$ with sets $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$	23
3.6	Auxiliary tree $\mathcal{T}_{\mathcal{Z}}$ built from vertices of set $\mathcal{Z}$	23
3.7	Auxiliary tree $\mathcal{T}_{\mathcal{Z}}$ construction and the structure of the decision tree $D_{\mathcal{Z}}$	23
3.8	Heavy module contraction	27
3.9	Input tree T	30
3.10	Example boxed decision tree.	30
3.11	Structure of a boxed decision tree	30
3.12	Boxline B	34
3.13	Bipartition of B	34
3.14	Decision tree D_1	34
3.15	Boxline B_2	34
3.16	Decision tree D_2	34
3.17	Rehanging step	34
3.18	Resulting decision tree D	34
3.19	Basic steps of DPTimelines procedure	34
3.20	k -up modularity	35
3.21	Example tree $\mathcal{T}$ with sets $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$	39
3.22	Auxiliary tree $\mathcal{T}_{\mathcal{Z}}$ built from vertices of set $\mathcal{Z}$	39
3.23	Auxiliary tree $\mathcal{T}_{\mathcal{Z}}$ construction and the structure of the decision tree $D_{\mathcal{Z}}$	39
4.1	Decision tree depth for random trees (uniform query costs)	47
4.2	Runtime for random trees (uniform query costs)	48
4.3	Decision tree depth for bounded degree trees (uniform query costs)	49
4.4	Runtime for bounded degree trees (uniform query costs)	49
4.5	Decision tree depth for bounded diameter trees (uniform query costs)	50
4.6	Runtime for bounded diameter trees (uniform query costs)	50
4.7	Random trees with uniform costs - exact algorithm: runtime	52
4.8	Random trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	53
4.9	Random trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	53
4.10	Random trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	56
4.11	Random trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	56
4.12	Random trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime	57
4.13	Random trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality	57
4.14	Random trees with binomial costs - exact algorithm: runtime comparison	58
4.15	Random trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	59

4.16	Random trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	59
4.17	Random trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	61
4.18	Random trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	62
4.19	Random trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime . . .	63
4.20	Random trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality	63
4.21	Random trees with exponential costs - exact algorithm: runtime comparison	64
4.22	Random trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	65
4.23	Random trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	65
4.24	Random trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime . .	68
4.25	Random trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	68
4.26	Random trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime .	69
4.27	Random trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality	69
4.28	Random bounded degree trees with uniform costs - exact algorithm: runtime comparison	71
4.29	Random bounded degree trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	72
4.30	Random bounded degree trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	72
4.31	Random bounded degree trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	74
4.32	Random bounded degree trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	75
4.33	Random bounded degree trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime	76
4.34	Random bounded degree trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality	76
4.35	Random bounded degree trees with binomial costs - exact algorithm: runtime comparison	77
4.36	Random bounded degree trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	78
4.37	Random bounded degree trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	78
4.38	Random bounded degree trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	80
4.39	Random bounded degree trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	81
4.40	Random bounded degree trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime	82
4.41	Random bounded degree trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality	82
4.42	Random bounded degree trees with exponential costs - exact algorithm: runtime comparison	83
4.43	Random bounded degree trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	84
4.44	Random bounded degree trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	84
4.45	Random bounded degree trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	87

4.46	Random bounded degree trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	87
4.47	Random bounded degree trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime	88
4.48	Random bounded degree trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality	88
4.49	Random bounded diameter trees with uniform costs - exact algorithm: runtime comparison	90
4.50	Random bounded diameter trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	91
4.51	Random bounded diameter trees with uniform costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	92
4.52	Random bounded diameter trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	94
4.53	Random bounded diameter trees with uniform costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	94
4.54	Random bounded diameter trees with uniform costs - $O(\log \log n)$ -approximation algorithm: runtime	95
4.55	Random bounded diameter trees with uniform costs - $O(\log \log n)$ -approximation algorithm: approximation quality	95
4.56	Random bounded diameter trees with binomial costs - exact algorithm: runtime comparison	97
4.57	Random bounded diameter trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	98
4.58	Random bounded diameter trees with binomial costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	98
4.59	Random bounded diameter trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	101
4.60	Random bounded diameter trees with binomial costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	101
4.61	Random bounded diameter trees with binomial costs - $O(\log \log n)$ -approximation algorithm: runtime	102
4.62	Random bounded diameter trees with binomial costs - $O(\log \log n)$ -approximation algorithm: approximation quality	102
4.63	Random bounded diameter trees with exponential costs - exact algorithm: runtime comparison	103
4.64	Random bounded diameter trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: runtime	104
4.65	Random bounded diameter trees with exponential costs - $O(\log n / \log \log n)$ -approximation algorithm: approximation quality	104
4.66	Random bounded diameter trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: runtime	107
4.67	Random bounded diameter trees with exponential costs - $O(\sqrt{\log n})$ -approximation algorithm: approximation quality	107
4.68	Random bounded diameter trees with exponential costs - $O(\log \log n)$ -approximation algorithm: runtime	108
4.69	Random bounded diameter trees with exponential costs - $O(\log \log n)$ -approximation algorithm: approximation quality	108

List of Tables

1.1	Sample values for the three field notation for the Graph Search Problem.	11
3.1	Summary of analyzed algorithms with theoretical guarantees	18
4.1	Tree generation methods and their properties	44
4.2	Probability distributions for vertex costs	45
4.3	Experiment coverage: tree generation methods vs. problem variants and cost distributions	46
4.4	Performance on random trees with uniform query costs	47
4.5	Performance on bounded degree trees with uniform query costs	48
4.6	Performance on bounded diameter trees with uniform query costs	50
4.7	Random trees with uniform cost distribution - exact algorithm	52
4.8	Random trees with uniform cost distribution - $O(\log n / \log \log n)$ -approximation algorithm	54
4.9	Random trees with uniform cost distribution - $O(\sqrt{\log n})$ -approximation algorithm . . .	55
4.10	Random trees with uniform cost distribution - $O(\log \log n)$ -approximation algorithm . .	56
4.11	Random trees with binomial cost distribution - exact algorithm	58
4.12	Random trees with binomial cost distribution - $O(\log n / \log \log n)$ -approximation algo- rithm	60
4.13	Random trees with binomial cost distribution - $O(\sqrt{\log n})$ -approximation algorithm . .	61
4.14	Random trees with binomial cost distribution - $O(\log \log n)$ -approximation algorithm . .	62
4.15	Random trees with exponential cost distribution - exact algorithm	64
4.16	Random trees with exponential cost distribution - $O(\log n / \log \log n)$ -approximation al- gorithm	66
4.17	Random trees with exponential cost distribution - $O(\sqrt{\log n})$ -approximation algorithm .	67
4.18	Random trees with exponential cost distribution - $O(\log \log n)$ -approximation algorithm	69
4.19	Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - exact algorithm	71
4.20	Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\log n / \log \log n)$ - approximation algorithm	73
4.21	Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\sqrt{\log n})$ - approximation algorithm	74
4.22	Random bounded degree trees ($\Delta(T) = 3$) with uniform cost distribution - $O(\log \log n)$ - approximation algorithm	75
4.23	Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - exact algo- rithm	77
4.24	Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\log n / \log \log n)$ - approximation algorithm	79
4.25	Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\sqrt{\log n})$ - approximation algorithm	80
4.26	Random bounded degree trees ($\Delta(T) = 3$) with binomial cost distribution - $O(\log \log n)$ - approximation algorithm	81
4.27	Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - exact algorithm	83
4.28	Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\log n / \log \log n)$ - approximation algorithm	85
4.29	Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\sqrt{\log n})$ - approximation algorithm	86
4.30	Random bounded degree trees ($\Delta(T) = 3$) with exponential cost distribution - $O(\log \log n)$ - approximation algorithm	88

4.31	Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - exact algorithm	90
4.32	Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\log n / \log \log n)$ -approximation algorithm	91
4.33	Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\sqrt{\log n})$ -approximation algorithm	93
4.34	Random bounded diameter trees ($\text{diam}(T) = 6$) with uniform cost distribution - $O(\log \log n)$ -approximation algorithm	95
4.35	Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - exact algorithm	97
4.36	Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - $O(\log n / \log \log n)$ -approximation algorithm	99
4.37	Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - $O(\sqrt{\log n})$ -approximation algorithm	100
4.38	Random bounded diameter trees ($\text{diam}(T) = 6$) with binomial cost distribution - $O(\log \log n)$ -approximation algorithm	102
4.39	Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - exact algorithm	103
4.40	Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\log n / \log \log n)$ -approximation algorithm	105
4.41	Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\sqrt{\log n})$ -approximation algorithm	106
4.42	Random bounded diameter trees ($\text{diam}(T) = 6$) with exponential cost distribution - $O(\log \log n)$ -approximation algorithm	108
4.43	Exponential fit for exact DP algorithm runtime	109